annunciator User Manual

ANN-UM-001, Issue 3, software version 1.1.1

Tom Scott

2026-11-13

CONTENTS

1 Document identification **5**
- 1.1 Identification data . . . 5
- 1.2 Change record . . . 5
- 1.3 Applicability . . . 5
- 1.4 Copyright and restrictions . . . 6
- 1.5 Forms of this manual . . . 6

2 Information for use of this manual **7**
- 2.1 Purpose of the manual . . . 7
- 2.2 Audience . . . 7
- 2.3 Scope . . . 7
- 2.4 How to use the manual . . . 8
- 2.5 Conventions . . . 8
- 2.6 Critical information . . . 8
- 2.7 Language . . . 9
- 2.8 Comments on the manual . . . 9

3 Concept of operations **11**
- 3.1 Function of the library . . . 11
- 3.2 Flow of data . . . 11
- 3.3 Parts of the library . . . 12
- 3.4 Severity . . . 13
- 3.5 Message codes . . . 13
- 3.6 Coded errors and coded warnings . . . 13
- 3.7 Error policy . . . 15
- 3.8 Functions for long runs and for evidence . . . 15
- 3.9 Relation to the standard `logging` module . . . 16

4 Installation **17**
- 4.1 Necessary software . . . 17
- 4.2 Install the library . . . 17
- 4.3 Make sure that the installation is correct . . . 18
- 4.4 Install the library for development . . . 18
- 4.5 Upgrade the library . . . 18
- 4.6 Remove the library . . . 19
- 4.7 Build this manual . . . 19

5 Procedures **21**
- 5.1 Record your first events . . . 21
- 5.2 Configure the library . . . 22
  - 5.2.1 Configure the library in the program . . . 23
  - 5.2.2 Configure the library from a file . . . 23
  - 5.2.3 Configure the library from the environment . . . 24
  - 5.2.4 Stop the library . . . 24
- 5.3 Record events . . . 25

5.3.1 Record a message . . . 25
5.3.2 Add context data . . . 26
5.3.3 Keep secret data out of the log . . . 26
5.4 Define a message catalogue . . . 27
5.4.1 Define the messages in a module . . . 27
5.4.2 Record a message from the catalogue . . . 28
5.4.3 Keep the catalogue in a file . . . 28
5.4.4 Show the definition of a code . . . 28
5.5 Raise and catch coded errors . . . 29
5.5.1 Raise a coded error . . . 29
5.5.2 Make your own error class . . . 29
5.5.3 Catch a coded error . . . 30
5.5.4 Add context to an error that goes up . . . 30
5.6 Issue coded warnings . . . 31
5.6.1 Issue a warning . . . 31
5.6.2 Change a warning into an error in tests . . . 32
5.6.3 Mark a function as deprecated . . . 32
5.7 Apply an error policy . . . 32
5.7.1 Record the exceptions of a function . . . 33
5.7.2 Record the exceptions of a block . . . 33
5.7.3 Do an operation again . . . 34
5.7.4 Collect many errors . . . 34
5.7.5 Record durations and calls . . . 35
5.8 Record unhandled exceptions . . . 35
5.8.1 Install the hooks . . . 35
5.8.2 Record the errors of an `asyncio` event loop . . . 36
5.9 Examine and summarise a log . . . 36
5.9.1 Make a summary at the end of a program . . . 36
5.9.2 Examine a log file in the terminal . . . 37
5.9.3 Examine events in a program . . . 38
5.9.4 Send events to a different system . . . 38
5.10 Document your messages with Sphinx . . . 38
5.11 Monitor a long run . . . 39
5.11.1 Keep detailed records for failures only . . . 40
5.11.2 Record repeated messages . . . 41
5.11.3 Record the progress of an operation . . . 41
5.11.4 Find an operation that stopped . . . 42
5.11.5 Count the different failures . . . 42
5.12 Record and compare runs . . . 43
5.12.1 Record the manifest of a run . . . 43
5.12.2 Make a tamper-evident log file . . . 44
5.12.3 Examine the integrity of a log file . . . 44
5.12.4 Compare two runs . . . 45
5.13 Make the records robust . . . 45
5.13.1 Write one log from many processes . . . 45
5.13.2 Keep the local variables of a failure . . . 46
5.13.3 Examine data with soft checks . . . 47
5.14 Examine your code and share results . . . 48
5.14.1 Examine the message codes in your source code . . . 48
5.14.2 Examine events in a test . . . 49
5.14.3 Make an HTML report of a run . . . 50

6 Reference information **53**
6.1 Severities . . . 53
6.2 Configuration options . . . 53
6.2.1 General options . . . 54
6.2.2 Console options . . . 54
6.2.3 File options . . . 55

6.2.4 Text options . . . 55
6.2.5 Filter options . . . 55
6.2.6 Diagnostic options . . . 56
6.2.7 Format of environment variables . . . 56
6.3 Error classes and warning classes . . . 57
6.3.1 Coded errors . . . 57
6.3.2 Attributes of a coded error . . . 57
6.3.3 Coded warnings . . . 58
6.4 Record formats and file formats . . . 58
6.4.1 Plain text format . . . 58
6.4.2 JSON Lines format . . . 58
6.4.3 Catalogue file format . . . 60
6.4.4 CSV format . . . 60
6.4.5 Run manifest . . . 60
6.4.6 HTML report . . . 61
6.5 Command-line interface . . . 61
6.5.1 Syntax . . . 61
6.5.2 General options . . . 61
6.5.3 Commands . . . 61
6.5.4 Selection options . . . 63
6.5.5 Exit status . . . 64
6.6 Programming interface . . . 64
6.6.1 Severities (`annunciator.levels`) . . . 64
6.6.2 Structured loggers (`annunciator.logger`) . . . 66
6.6.3 Context data (`annunciator.context`) . . . 72
6.6.4 Message catalogues (`annunciator.catalogue`) . . . 73
6.6.5 Coded errors (`annunciator.errors`) . . . 78
6.6.6 Coded warnings (`annunciator.warns`) . . . 84
6.6.7 Events (`annunciator.events`) . . . 87
6.6.8 Configuration (`annunciator.config`) . . . 90
6.6.9 Handlers (`annunciator.handlers`) . . . 94
6.6.10 Formatters (`annunciator.formatters`) . . . 97
6.6.11 Filters (`annunciator.filters`) . . . 98
6.6.12 Handling tools (`annunciator.handling`) . . . 102
6.6.13 Hooks (`annunciator.hooks`) . . . 108
6.6.14 Reports (`annunciator.report`) . . . 109
6.6.15 Flight recorder (`annunciator.recorder`) . . . 114
6.6.16 Progress and watchdog (`annunciator.watch`) . . . 115
6.6.17 Runs (`annunciator.provenance`) . . . 118
6.6.18 Integrity (`annunciator.integrity`) . . . 121
6.6.19 Multiprocess tools (`annunciator.multiprocess`) . . . 123
6.6.20 Soft checks (`annunciator.checks`) . . . 125
6.6.21 Lint (`annunciator.lint`) . . . 128
6.6.22 Test tools (`annunciator.testing`) . . . 129
6.6.23 HTML report (`annunciator.htmlreport`) . . . 132
6.6.24 Command-line interface (`annunciator.cli`) . . . 132
6.6.25 Sphinx extension (`annunciator.sphinxext`) . . . 133

7 Messages and problem resolution **135**
7.1 Find the remedy for a message . . . 135
7.1.1 How to read a message code . . . 135
7.2 Message reference . . . 136
7.2.1 ANN-E-0001 . . . 136
7.2.2 ANN-E-0002 . . . 136
7.2.3 ANN-E-0003 . . . 136
7.2.4 ANN-E-0004 . . . 137
7.2.5 ANN-D-0010 . . . 137
7.2.6 ANN-E-0101 . . . 137

7.2.7 ANN-E-0102 . . . 138
7.2.8 ANN-E-0103 . . . 138
7.2.9 ANN-E-0104 . . . 138
7.2.10 ANN-E-0105 . . . 139
7.2.11 ANN-C-0201 . . . 139
7.2.12 ANN-C-0202 . . . 139
7.2.13 ANN-E-0203 . . . 140
7.2.14 ANN-E-0204 . . . 140
7.2.15 ANN-W-0205 . . . 140
7.2.16 ANN-W-0301 . . . 141
7.2.17 ANN-E-0302 . . . 141
7.2.18 ANN-E-0303 . . . 141
7.2.19 ANN-E-0304 . . . 142
7.2.20 ANN-W-0305 . . . 142
7.2.21 ANN-D-0306 . . . 142
7.2.22 ANN-D-0307 . . . 143
7.2.23 ANN-I-0308 . . . 143
7.2.24 ANN-W-0309 . . . 143
7.2.25 ANN-E-0310 . . . 143
7.2.26 ANN-I-0320 . . . 144
7.2.27 ANN-I-0330 . . . 144
7.2.28 ANN-I-0331 . . . 144
7.2.29 ANN-W-0332 . . . 145
7.2.30 ANN-I-0333 . . . 145
7.2.31 ANN-I-0340 . . . 145
7.2.32 ANN-N-0401 . . . 146
7.2.33 ANN-N-0402 . . . 146
7.2.34 ANN-E-0403 . . . 146
7.2.35 ANN-E-0501 . . . 147
7.2.36 ANN-E-0510 . . . 147
7.2.37 ANN-E-0511 . . . 147
7.2.38 ANN-E-0601 . . . 148
7.2.39 ANN-E-0602 . . . 148
7.2.40 ANN-E-0603 . . . 148
7.2.41 ANN-E-0604 . . . 149
7.2.42 ANN-E-0605 . . . 149
7.2.43 ANN-E-0606 . . . 149
7.2.44 ANN-W-0611 . . . 150
7.2.45 ANN-W-0612 . . . 150
7.2.46 ANN-W-0613 . . . 150
7.2.47 ANN-W-0614 . . . 151
7.2.48 ANN-W-0615 . . . 151
7.2.49 ANN-W-0616 . . . 151
7.3 Problems without a message code . . . 152
7.4 Report a problem . . . 153

8 Glossary **155**
8.1 Abbreviations . . . 157

9 Related information **159**
9.1 Standards . . . 159
9.2 Python documentation . . . 159
9.3 Other documentation . . . 159
9.4 Files in the software package . . . 160

10 Conformity to standards **161**
10.1 Structure and content: ISO/IEC/IEEE 26514:2022 . . . 161
10.2 Language: ASD-STE100 Issue 9 . . . 162
10.3 Limits of the conformity statement . . . 162

Python Module Index 163

Index 165

LIST OF FIGURES

3.1 Flow of data in the library 11
3.2 Structure of a message code 13
3.3 Classes of coded errors and coded warnings 14
3.4 Policy of `handle_errors` and `guard` 15

5.1 Operation of the flight recorder 40

LIST OF TABLES

1.1 Identification data of this manual . . . 5
1.2 Change record . . . 5

2.1 Where to find information . . . 8
2.2 Typographic conventions . . . 8
2.3 Technical names . . . 9
2.4 Technical verbs . . . 9

3.1 Parts of the library . . . 12
3.2 Use of the severities . . . 13

4.1 Necessary software . . . 17

5.1 Procedures . . . 21
5.2 Selection of the tool for a warning . . . 31
5.3 Selection of the handling tool . . . 32
5.4 Policy parameters . . . 33
5.5 Events of the hooks . . . 36
5.6 Selection of the tool for a long run . . . 40
5.7 Soft checks . . . 47
5.8 Values of the parameter `on_fail` . . . 48
5.9 Types of findings of the command `lint` . . . 49
5.10 Conditions of `assert_logged` and `assert_not_logged` . . . 50
5.11 Sections of the HTML report . . . 50

6.1 Severities . . . 53
6.2 General options . . . 54
6.3 Console options . . . 54
6.4 File options . . . 55
6.5 Text options . . . 55
6.6 Filter options . . . 56
6.7 Diagnostic options . . . 56
6.8 Values of environment variables . . . 56
6.9 Classes of coded errors . . . 57
6.10 Attributes of coded errors and coded warnings . . . 57
6.11 Classes of coded warnings . . . 58
6.12 Parts of the primary line . . . 58
6.13 Keys of an event object . . . 59
6.14 Keys of the object `exception` . . . 59
6.15 Keys of the object `chain` . . . 59
6.16 Keys of a catalogue file . . . 60
6.17 Keys of a run manifest . . . 61
6.18 General options of the command-line interface . . . 61
6.19 Options of the command `catalogue` . . . 62

6.20 Options of the command `report` . . . 62
6.21 Options of the command `view` . . . 62
6.22 Options of the command `verify` . . . 63
6.23 Options of the command `lint` . . . 63
6.24 Selection options . . . 63
6.25 Exit status of the command-line interface . . . 64

7.1 Severity letters and necessary action . . . 135
7.2 Number ranges of the library messages . . . 136
7.3 Problems, probable causes and remedies . . . 152

8.1 Abbreviations . . . 157

9.1 Standards . . . 159
9.2 Python documentation . . . 159
9.3 Other documentation . . . 159
9.4 Files in the software package . . . 160

10.1 Information components and their position in this manual . . . 161
10.2 Application of the writing rules . . . 162

Document number
ANN-UM-001

Issue
3

Issue date
2026-11-13

Applicable software
annunciator, version 1.1.1

Author
Tom Scott

`annunciator` is a Python library for error, warning and information handling and for logs. This manual tells you how to install the library, how to use it and how to correct problems.

If you are a new user, read *Information for use of this manual* first. Then do the procedure *Record your first events*.

CHAPTER

ONE

DOCUMENT IDENTIFICATION

1.1 Identification data

Table 1.1: Identification data of this manual

Item	Value
Title	annunciator User Manual
Document number	ANN-UM-001
Issue	3
Issue date	2026-11-13
Applicable software	annunciator, version 1.1.1
Applicable environment	Python 3.9 or later on Windows, Linux and macOS
Author and issuer	Tom Scott
Licence of the software	MIT licence
Language	English (ASD-STE100 Simplified Technical English, Issue 9)

1.2 Change record

Table 1.2: Change record

Issue	Date	Software version	Change
1	2026-11-12	1.0.0	First issue.
2	2026-11-12	1.1.0	Added four procedures: *Monitor a long run*, *Record and compare runs*, *Make the records robust* and *Examine your code and share results*. Added the related options, commands, formats, messages and terms.
3	2026-11-13	1.1.1	Changed the description of the HTML report in *Examine your code and share results*. The report now has the style of a printed industrial report.

1.3 Applicability

This issue applies only to version 1.1.1 of the software. To find the version of your software, do this command:

```
annunciator --version
```

If the version is different, use the issue of the manual that applies to your version.

1.4 Copyright and restrictions

Copyright 2026, Tom Scott. The MIT licence applies to the software and to this manual. The file `LICENSE` in the software package contains the text of the licence.

The software has no warranty. The licence text gives the conditions.

1.5 Forms of this manual

The manual is available in two forms. The two forms have the same contents.

- The HTML form is for use on a screen. It has a navigation panel, a search function and an index.
- The PDF form is for a printed copy. It has a table of contents, a list of figures, a list of tables and an index.

CHAPTER

TWO

INFORMATION FOR USE OF THIS MANUAL

2.1 Purpose of the manual

This manual gives the instructions and the reference data for the `annunciator` library. It tells you how to do these tasks:

- install the library,
- record information, warnings and errors,
- define messages that have a code, a cause and a remedy,
- handle exceptions with a standard policy,
- examine log files,
- monitor long runs and compare runs,
- show that a log file did not change,
- find the cause of a message and correct the problem.

2.2 Audience

The manual is for two groups of users.

Developers
: Persons who write Python programs that use the library. You must know how to write Python functions and classes. You must know how to use exceptions. Knowledge of the standard `logging` module helps, but it is not necessary.

Operators
: Persons who run a program that uses the library and who read its messages and its log files. You must know how to use a terminal. Knowledge of Python is not necessary. Read *Messages and problem resolution* and *Command-line interface*.

2.3 Scope

The manual includes the full function of version 1.1.1 of the library. It does not give instructions for Python, for the standard `logging` module or for Sphinx. *Related information* tells you where to find those instructions.

2.4 How to use the manual

Table 2.1: Where to find information

If you must	Read
Know what the library does and how its parts operate together	*Concept of operations*
Install, upgrade or remove the library	*Installation*
Do a task for the first time	*Procedures*
Find the permitted values of an option, a command or a parameter	*Reference information*
Know the meaning of a message code and the corrective action	*Messages and problem resolution*
Know the meaning of a term	*Glossary*

Each procedure has the same structure:

Purpose
The result that the procedure gives.

Before you start
The conditions that are necessary before the first step.

Procedure
The steps, in a numbered list. Do the steps in the given sequence.

Result
The condition that shows that the procedure was successful.

2.5 Conventions

Table 2.2: Typographic conventions

Convention	Meaning
`monospace text`	Text that you type, text that the computer shows, and names in the program code.
`UPPER_CASE` in a command	A value that you must replace, for example `FILE`.
`[text]` in a command	An optional part of a command.
`>>>`	The prompt of the interactive Python interpreter. Do not type it.
`$` or a line without prompt in a `console` block	A command for the terminal.

2.6 Critical information

The manual uses two types of critical information. The use of this software cannot cause injury to persons. Thus the manual contains no warnings.

Caution

A caution tells you that data loss, disclosure of secret data or an incorrect result is possible. Obey the instruction in the caution.

Note

A note gives you information that helps you to do a task.

A caution or a note that applies to a step comes immediately before that step.

2.7 Language

The text of this manual obeys the writing rules of ASD-STE100 Simplified Technical English, Issue 9. The spelling is British English.

The names in Table 2.3 are technical names in this manual. The verbs in Table 2.4 are technical verbs. *Glossary* gives the definitions of the technical names.

Table 2.3: Technical names

Category	Technical names
Parts of the library	catalogue, coded error, coded warning, context, event, event store, field, filter, formatter, handler, hook, logger, message code, record, report, severity
Functions for runs	chain, fingerprint, flight recorder, manifest, run, soft check, stall, watchdog
Python terms	argument, attribute, class, context manager, coroutine, decorator, exception, fixture, function, local variable, method, module, package, parameter, process, queue, subclass, thread, traceback, warning
Computer terms	browser, command, console, directory, environment variable, file, hash, HTML, JSON, JSON Lines, key, log, option, path, prefix, seed, terminal, TOML

Table 2.4: Technical verbs

Verb	Meaning in this manual
call	To start a function or a method.
catch	To stop an exception in an `except` block.
configure	To set the options of the library.
format	To change a record into text.
import	To load a Python module.
raise	To start an exception.
return	To give a value to the code that called a function.
run	To start a program or a command.
type	To enter text with the keyboard.

2.8 Comments on the manual

If you find an error in this manual, send a report to the issuer. Include the document number, the issue and the section number.

CHAPTER

THREE

CONCEPT OF OPERATIONS

3.1 Function of the library

A program must tell its user what occurs. Some information is about normal progress. Some information tells the user that a result is possibly incorrect. Some information tells the user that an operation failed.

`annunciator` gives one system for all this information. The system has three primary functions:

Record
: The library records each event with a severity, a time, a text and structured data. It writes the events to the console, to a log file and to an event store in memory.

Explain
: Each message can have a message code. The code identifies a definition in a message catalogue. The definition gives the probable cause and the remedy. Thus the user knows what to do.

Handle
: The library gives tools that apply a standard policy to exceptions. The tools record an exception, change it into a different exception, try the operation again or collect it.

The library uses only the Python standard library. It operates together with the standard `logging` module. Records from standard loggers go through the same handlers.

3.2 Flow of data

Figure 3.1 shows how an event goes from your program to its destinations.

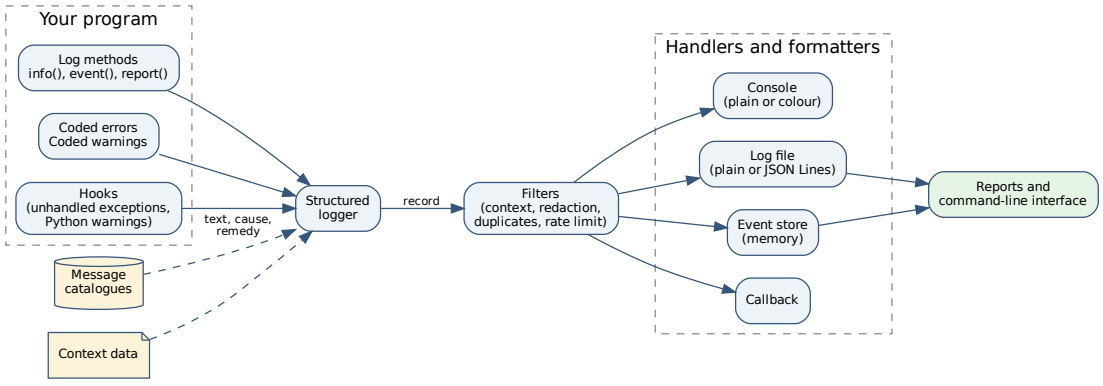


Figure 3.1: Flow of data in the library

The flow has these stages:

1. Your program calls a method of a structured logger. A coded error, a coded warning or a hook can also start an event.
2. If the event has a message code, the logger gets the text, the cause and the remedy from the catalogue.
3. The logger adds the context data and makes a record.
4. The filters of each handler examine the record. A filter can accept, reject or change the record.
5. The formatter of each handler changes the record into text.
6. The handler writes the text to its destination.
7. The report functions and the command-line interface read the events. The source is the event store or a JSON Lines log file.

3.3 Parts of the library

Table 3.1: Parts of the library

Part	Function	Module
Severity	Gives the importance of an event.	`annunciator.levels`
Structured logger	Records events with codes and fields.	`annunciator.logger`
Context	Adds the same data to all events of one operation.	`annunciator.context`
Message catalogue	Contains the definitions of the messages.	`annunciator.catalogue`
Coded error	An exception that has a code, a cause and a remedy.	`annunciator.errors`
Coded warning	A Python warning that has a code, a cause and a remedy.	`annunciator.warns`
Event	The structured form of one record.	`annunciator.events`
Handler	Sends records to a destination.	`annunciator.handlers`
Formatter	Changes a record into plain text, colour text or JSON.	`annunciator.formatters`
Filter	Accepts, rejects or changes records.	`annunciator.filters`
Configuration	Installs the handlers, the formatters, the filters and the hooks.	`annunciator.config`
Handling tools	Apply a policy to exceptions.	`annunciator.handling`
Hooks	Record exceptions that no code caught, and Python warnings.	`annunciator.hooks`
Reports	Read and select events, and make summaries.	`annunciator.report`
Flight recorder	Keeps detailed records and writes them when a failure occurs.	`annunciator.recorder`
Progress and watchdog	Record the progress of long operations and find stalls.	`annunciator.watch`
Runs	Record the manifest and the result of each run.	`annunciator.provenance`
Integrity	Makes and examines tamper-evident log files.	`annunciator.integrity`
Multiprocess tools	Send the records of many processes to one writer.	`annunciator.multiprocess`
Soft checks	Examine data and give coded messages.	`annunciator.checks`
Lint	Finds incorrect message codes in source code.	`annunciator.lint`
Test tools	Collect events in tests.	`annunciator.testing`
HTML report	Writes a report of a run for a browser.	`annunciator.htmlreport`
Command-line interface	Shows catalogues and examines log files.	`annunciator.cli`
Sphinx extension	Writes the message reference of your documentation.	`annunciator.sphinxext`

3.4 Severity

Each event has one of seven severities. Table 3.2 gives the use of each severity. *Severities* gives the full data.

Table 3.2: Use of the severities

Severity	Type	Use
`TRACE`, `DEBUG`	Information	Data for a developer who looks for the cause of a fault.
`INFO`, `NOTICE`	Information	Normal progress. `NOTICE` is for events that are important to the user.
`WARNING`	Warning	The program continues, but a result is possibly incorrect.
`ERROR`	Error	An operation did not complete. The program can continue with other operations.
`CRITICAL`	Error	The program cannot continue.

3.5 Message codes

A message code identifies one message definition. Figure 3.2 shows the three parts of a code.

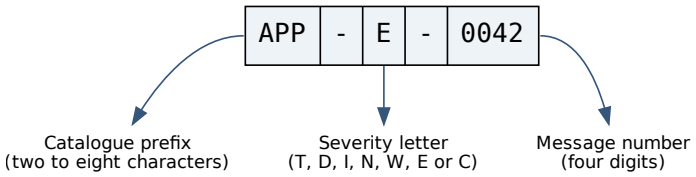


Figure 3.2: Structure of a message code

The prefix identifies the catalogue. Usually, one program or one package has one catalogue. The severity letter is the first letter of the severity name. The message number is unique in the catalogue.

A message definition contains four items:

- the code,
- the message text, with optional replacement fields such as `{path}`,
- the probable cause,
- the remedy.

The same definition supplies the text of the log record, the text of the exception and the text of the user documentation. Thus these three texts always agree.

3.6 Coded errors and coded warnings

A coded error is an exception with a message code. A coded warning is a Python warning with a message code. Figure 3.3 shows the classes that the library supplies. You can make your own subclasses.

Use an error when the operation cannot continue. Use a warning when the operation continues but the user must know about a condition.

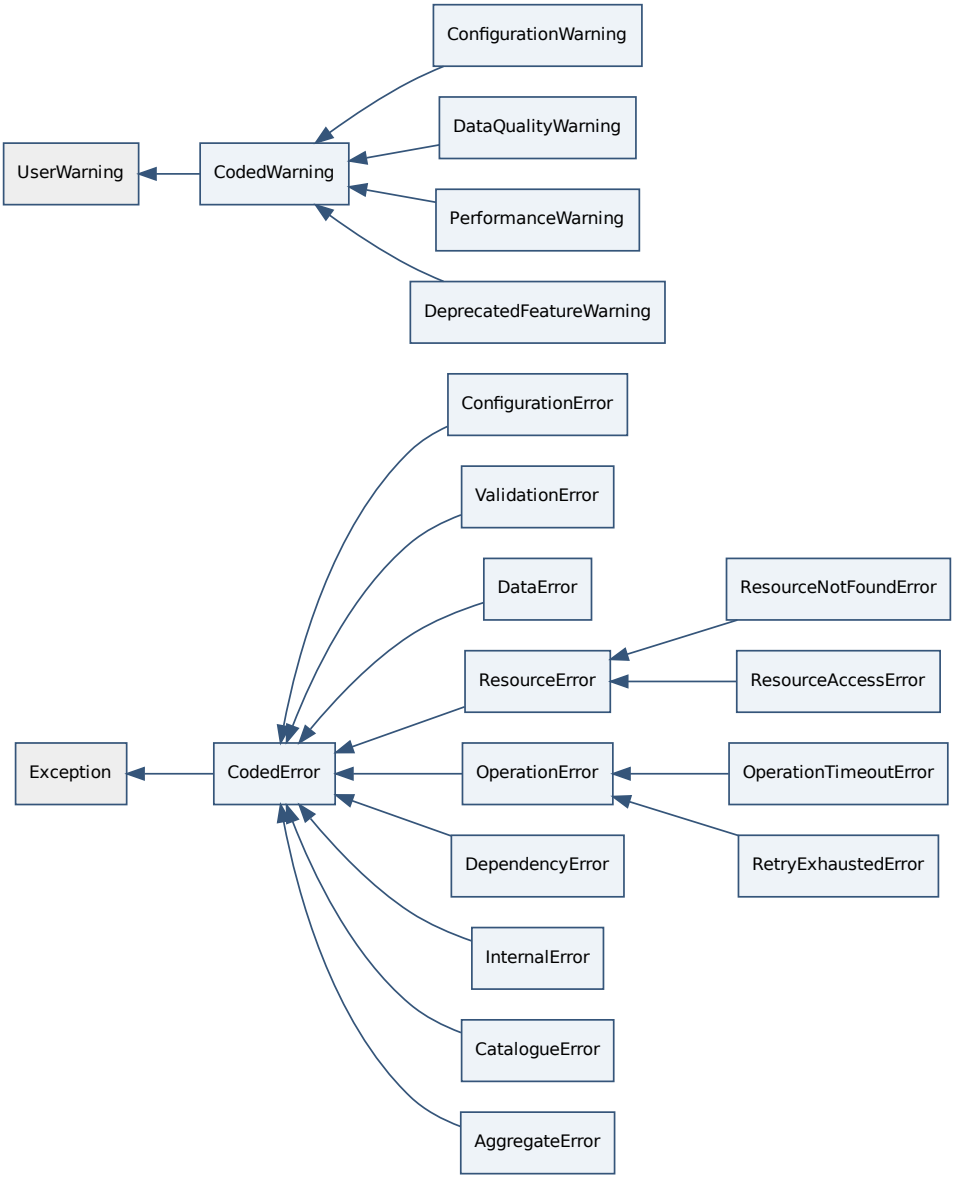


Figure 3.3: Classes of coded errors and coded warnings

3.7 Error policy

The handling tools apply the same policy to each exception. Figure 3.4 shows the policy of the `handle_errors` decorator and the `guard` context manager.

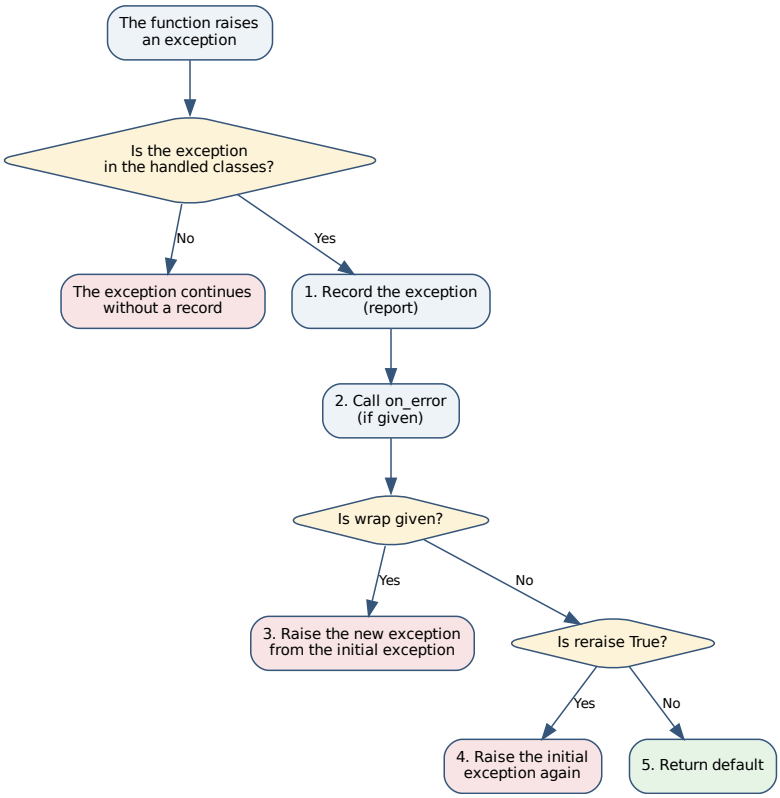


Figure 3.4: Policy of `handle_errors` and `guard`

The library records each exception object only one time. If an exception goes through two handling tools, the log contains one record.

3.8 Functions for long runs and for evidence

Version 1.1.0 added functions that standard log tools do not usually have. They answer four questions.

What occurred before the failure?
The flight recorder keeps the last detailed records in memory. It writes them only when a failure occurs. Refer to Figure 5.1.

Does the run continue?
Progress records show the rate and the remaining time. A watchdog records a warning when an operation makes no progress.

Can a different person do the same run again?
A run manifest records the software versions, the source revision, the configuration and the seed.

Is the log file the same as when the program wrote it?
A chain of hashes connects the records. A change, a removal or an addition breaks the chain.

Monitor a long run and *Record and compare runs* give the procedures.

3.9 Relation to the standard `logging` module

A structured logger is a thin layer on a standard logger with the same name. Thus these conditions apply:

- Modules that use `logging.getLogger` send their records through the handlers of the library.
- Handlers from other packages receive the records of structured loggers.
- The severities have the same numeric values as the standard levels.

The library does not change the root logger before you call `configure`.

CHAPTER

FOUR

INSTALLATION

4.1 Necessary software

Table 4.1: Necessary software

Item	Necessary condition
Python	Version 3.9 or later
Operating system	Windows, Linux or macOS
Other packages	None for the library. Python 3.9 and 3.10 must have the package `tomli` to read TOML files.
To build this manual	The packages `sphinx` and `furo`. A LaTeX system is necessary for the PDF form.

4.2 Install the library

Purpose

To install the library in your Python environment.

Before you start

- Make sure that Python 3.9 or later is installed.
- Get the software package `annunciator`. The package is a directory or a `.whl` file.

Procedure

1. Open a terminal.
2. If you use a virtual environment, activate it.
3. Go to the directory that contains the file `pyproject.toml`.
4. Install the library:

   ```
   python -m pip install .
   ```

   If you have a `.whl` file, give the path of the file as a replacement for the full stop.
5. If your Python version is 3.9 or 3.10 and you use TOML files, install the extra `toml`:

   ```
   python -m pip install ".[toml]"
   ```

Result

The library is installed. Do the procedure *Make sure that the installation is correct*.

4.3 Make sure that the installation is correct

Purpose

To make sure that Python can find the library and that the library operates.

Procedure

1. Show the version:

```
annunciator --version
```

2. Show an example of each severity:

```
annunciator demo
```

Result

Step 1 shows `annunciator` and the version number. Step 2 shows seven lines, one for each severity, and one line with the code `ANN-E-0001`.

If the terminal cannot find the command `annunciator`, use `python -m annunciator` as a replacement. Refer to *Problems without a message code*.

4.4 Install the library for development

Purpose

To install the library in a mode that uses the source files directly. Then a change in a source file has an immediate effect.

Procedure

1. Go to the directory that contains the file `pyproject.toml`.
2. Install the library in the editable mode, with the tools for the manual:

```
python -m pip install -e ".[docs]"
```

Result

The command `python -m pip show annunciator` shows the path of your source directory.

4.5 Upgrade the library

Procedure

1. Get the new software package.
2. Go to the directory that contains the new file `pyproject.toml`.
3. Install the new version:

```
python -m pip install --upgrade .
```

4. Read the file `CHANGELOG.md` for changes that have an effect on your program.

Result

The command `annunciator --version` shows the new version number.

4.6 Remove the library

Procedure

1. Remove the library:

```
python -m pip uninstall annunciator
```

2. Type `y` when the command asks for confirmation.

Result

The command `python -m pip show annunciator` shows that the package is not installed.

Note

This procedure does not remove your log files and your configuration files.

4.7 Build this manual

Before you start

Install the library for development. That procedure installs `sphinx` and `furo`.

Procedure

1. Go to the directory that contains the file `pyproject.toml`.
2. Build the HTML form:

```
sphinx-build -b html docs docs/_build/html
```

3. If you must have the PDF form, build it:

```
sphinx-build -M latexpdf docs docs/_build
```

4. Examine the text against the writing rules:

```
python tools/ste_check.py docs
```

Result

The file `docs/_build/html/index.html` is the first page of the HTML form. The file `docs/_build/latex/ANN-UM-001.pdf` is the PDF form. Step 4 shows the number of findings.

CHAPTER

FIVE

PROCEDURES

This chapter contains the procedures for the usual tasks. Do the procedure *Record your first events* first. You can do the other procedures in any sequence.

Table 5.1: Procedures

Task	Procedure
Record your first events	*Record your first events*
Select the destinations and the formats	*Configure the library*
Record information, warnings and errors	*Record events*
Give a code, a cause and a remedy to each message	*Define a message catalogue*
Raise and catch errors that have a code	*Raise and catch coded errors*
Tell the user about a condition without an exception	*Issue coded warnings*
Apply a standard policy to exceptions	*Apply an error policy*
Record exceptions that no code caught	*Record unhandled exceptions*
Examine and summarise a log	*Examine and summarise a log*
Write the message reference of your own manual	*Document your messages with Sphinx*
Get the necessary information from a run of many hours	*Monitor a long run*
Record a manifest, protect a log file and compare runs	*Record and compare runs*
Use many processes, keep local variables and examine data	*Make the records robust*
Find incorrect codes, write tests and make an HTML report	*Examine your code and share results*

5.1 Record your first events

Purpose

To write a small program that records information, a warning and an error.

Before you start

Install the library. Refer to *Installation*.

Procedure

1. Make a file with the name `first.py`.
2. Put this text in the file:

Example 5.1: A first program

```
from annunciator import configure, get_logger

configure(level="INFO")
log = get_logger("first")

log.info("The measurement started.", sample="CeO2-04", live_time=600)
log.warning("The dead time is high.", dead_time=0.31)
try:
    value = 1 / 0
except ZeroDivisionError as error:
    log.report(error, operation="ratio")
```

3. Run the program:

```
python first.py
```

Result

The terminal shows three events. The text is almost the same as this example. The times and the paths are different.

```
2026-11-12T10:15:02.120Z INFO     first The measurement started. | sample=CeO2-04␣
↪live_time=600
2026-11-12T10:15:02.121Z WARNING  first The dead time is high. | dead_time=0.31
2026-11-12T10:15:02.121Z ERROR    first [ANN-E-0303] The operation 'ratio' failed:␣
↪ZeroDivisionError: division by zero | operation=ratio exc_type=ZeroDivisionError
    Cause:  The operation raised an exception that does not have a message code.
    Remedy: Read the traceback in the log and correct the cause.
    Traceback (most recent call last):
      File "first.py", line 9, in <module>
        value = 1 / 0
    ZeroDivisionError: division by zero
```

Each line has the time, the severity, the logger name and the message. The text after the vertical bar shows the structured fields.

Subsequent steps

- To write the events to a file, do *Configure the library*.
- To give your own codes to messages, do *Define a message catalogue*.

5.2 Configure the library

Purpose

To select the minimum severity, the destinations and the formats of the events.

Before you start

Read *Configuration options*. It gives all options and their permitted values.

5.2.1 Configure the library in the program

Procedure

1. Import the function `configure`.
2. Call the function one time, near the start of the program. Give the options as keyword arguments:

Example 5.2: Configuration in the program

```
from annunciator import configure

configure(
    level="DEBUG",
    console_level="INFO",
    file="logs/run.jsonl",
    file_format="json",
    rotate="size",
    max_bytes=5_000_000,
    backup_count=3,
)
```

3. To change the configuration, call `configure` again with the new options.

Result

The console shows events of the severity `INFO` and higher. The file `logs/run.jsonl` contains events of the severity `DEBUG` and higher, one JSON object on each line. The library makes the directory `logs` if it does not exist.

> **Note**
>
> A second call of `configure` removes the handlers of the first call. It does not remove handlers that other code installed.

5.2.2 Configure the library from a file

Procedure

1. Make a TOML file, for example `annunciator.toml`:

Example 5.3: Configuration file

```
[annunciator]
level = "INFO"
file = "logs/run.log"
rotate = "time"
when = "midnight"
backup_count = 14
duplicate_window = 10.0

[annunciator.logger_levels]
"matplotlib" = "WARNING"
```

2. Examine the file:

```
annunciator check-config annunciator.toml
```

3. If the command shows a message code, refer to *Messages and problem resolution*. Correct the file and do step 2 again.
4. Give the path of the file to `configure`:

```
from annunciator import configure

configure("annunciator.toml")
```

Result

Step 2 shows the text `The configuration file is correct` and the value of each option. The program then uses these values.

> **Note**
>
> You can also put the options in the table `[tool.annunciator]` of the file `pyproject.toml`.

5.2.3 Configure the library from the environment

Use this procedure to change an option for one run without a change to the program.

Procedure

1. In the program, read the environment before you call `configure`:

```
from annunciator import LogConfig, configure

base = LogConfig(level="INFO", file="logs/run.log")
configure(LogConfig.from_env(base=base))
```

2. In the terminal, set a variable. The name is `ANNUNCIATOR_` and the option name in capital letters.

   On Windows (PowerShell):

```
$env:ANNUNCIATOR_LEVEL = "DEBUG"
```

   On Linux and macOS:

```
export ANNUNCIATOR_LEVEL=DEBUG
```

3. Run the program.

Result

The program uses the value of the variable for the option `level`. It uses the values of `base` for all other options.

5.2.4 Stop the library

Procedure

1. At the end of the program, call `shutdown`:

```
from annunciator import shutdown

shutdown()
```

Result

The library writes all records, closes the log file and removes its handlers and hooks.

> **Note**
>
> The library also calls `shutdown` automatically when Python stops.

5.3 Record events

Purpose

To record information, warnings and errors with structured fields and context data.

Before you start

Configure the library. Refer to *Configure the library*.

5.3.1 Record a message

Procedure

1. Get a logger in each module. Use the name of the module:

```
from annunciator import get_logger

log = get_logger(__name__)
```

2. Call the method that has the name of the severity. Give the data as keyword arguments:

Example 5.4: Messages with structured fields

```
log.debug("The spectrum has %d channels.", 8192)
log.info("The fit completed.", chi_square=1.07, iterations=14)
log.notice("The calibration is complete.")
log.warning("The peak is near the edge of the range.", channel=8101)
log.error("The file is not available.", path="run_17.spe")
log.critical("The detector lost its high voltage.")
```

3. In an `except` block, use `exception` or `report` to record the traceback:

```
try:
    spectrum = load("run_17.spe")
except OSError as error:
    log.report(error, operation="load spectrum", path="run_17.spe")
    raise
```

Result

Each call makes one event. The keyword arguments are the structured fields of the event. In a JSON Lines file, the fields are in the object `fields`.

> **Note**
>
> Put values in fields, not in the message text. Then the message text stays constant and you can find all events of one type.

5.3.2 Add context data

Context data are fields that the library adds to all events of one operation.

Procedure

1. To add data for a block of code, use the context manager `context`:

Example 5.5: Context data

```
from annunciator import context, get_logger

log = get_logger(__name__)

with context(run="R-17", sample="CeO2-04"):
    log.info("The acquisition started.")
    acquire()
    log.info("The acquisition stopped.")
```

2. To add data until the end of the thread or the task, use `bind`:

```
from annunciator import bind, new_correlation_id

bind(operator="T. Scott")
new_correlation_id()
```

3. To give permanent fields to one logger, use the method `bind` of the logger:

```
detector_log = log.bind(detector="HPGe-2")
detector_log.info("The bias is stable.")
```

Result

All events in the block of step 1 have the fields `run` and `sample`. Events from functions that the block calls also have these fields.

5.3.3 Keep secret data out of the log

Caution

Do not put secret data in the message text. The redaction filter replaces the values of fields that have secret names. It examines the message text only for the patterns that you supply.

Procedure

1. Give secret values in fields that have a secret name. Examples are `password` and `api_token`:

```
log.info("The connection is open.", user="tom", api_token=token)
```

2. If your secret fields have other names, add the names to the option `redact_keys`.
3. If secret text can occur in messages, add a regular expression to the option `redact_patterns`:

```
configure(redact_patterns=(r"sk-[A-Za-z0-9]{20,}",))
```

Result

The log shows `[REDACTED]` as a replacement for each secret value. Table 6.6 gives the default key names.

5.4 Define a message catalogue

Purpose

To give a code, a probable cause and a remedy to each message of your program.

Before you start

- Read *the description of message codes* in *Concept of operations*.
- Select a prefix for your catalogue. The prefix has two to eight capital letters or digits. The prefix `ANN` is not available, because the library uses it.

5.4.1 Define the messages in a module

Procedure

1. Make a module for the messages, for example `myapp/messages.py`.
2. Make the catalogue and define each message:

Example 5.6: A message catalogue

```
from annunciator import MessageCatalogue

APP = MessageCatalogue("APP", "Messages of the spectrum tools")

APP.define(
    1, "E", "The detector '{name}' does not answer.",
    cause="The detector has no power or the cable is not connected.",
    remedy="Make sure that the detector has power. Examine the cable.",
    category="Hardware",
)
APP.define(
    2, "W", "The dead time is {dead_time} %.",
    cause="The count rate is too high for the detector.",
    remedy="Increase the distance between the source and the detector.",
    category="Measurement",
)
APP.define(
    3, "I", "The run '{run}' completed. Live time: {live_time} s.",
)
```

3. Import the module before the program uses a code. Usually, import it in the file `__init__.py` of your package.

Result

The command that follows shows your three messages:

```
annunciator --import myapp.messages catalogue --prefix APP
```

Note

Write the message text as a full sentence. Write the cause as a statement. Write the remedy as an instruction.

5.4.2 Record a message from the catalogue

Procedure

1. Call the method `event` of a logger. Give the code and the values for the replacement fields:

```
from annunciator import get_logger

log = get_logger(__name__)
log.event("APP-W-0002", dead_time=31)
log.event("APP-I-0003", run="R-17", live_time=600)
```

Result

The logger gets the severity and the text from the definition. For the severity `WARNING` and higher, the plain text shows the cause and the remedy:

```
2026-11-12T10:15:02.454Z WARNING  myapp.daq [APP-W-0002] The dead time is 31 %. |␣
↪dead_time=31
   Cause:  The count rate is too high for the detector.
   Remedy: Increase the distance between the source and the detector.
```

Note

If the code has no definition, the logger records an `ERROR` event that contains the code and the fields. It does not raise an exception. To get an exception in your tests, set the parameter `strict` to `True`.

5.4.3 Keep the catalogue in a file

Use this procedure when persons who do not write Python must change the message texts.

Procedure

1. Write the catalogue to a JSON file one time:

```
APP.save("messages.json")
```

2. Change the texts in the file with a text editor.
3. In the program, read the file as a replacement for the calls of `define`:

```
from annunciator import MessageCatalogue

APP = MessageCatalogue.load("messages.json")
```

Result

The program uses the texts of the file. *Record formats and file formats* gives the structure of the file.

5.4.4 Show the definition of a code

Procedure

1. In a terminal, give the code to the command `explain`:

```
annunciator --import myapp.messages explain APP-E-0001
```

Result

The terminal shows the severity, the message text, the cause and the remedy of the code.

5.5 Raise and catch coded errors

Purpose

To raise exceptions that have a message code, and to give the user the cause and the remedy when the program catches them.

Before you start

Define a message catalogue. Refer to *Define a message catalogue*. The examples use the catalogue of Example 5.6.

5.5.1 Raise a coded error

Procedure

1. Select the class in Figure 3.3 that agrees with the condition. If no class agrees, use `CodedError`.
2. Raise the class with the code and the values for the replacement fields:

```
from annunciator import ResourceError

raise ResourceError(code="APP-E-0001", name="HPGe-2")
```

3. If the error has no definition in a catalogue, give the message text. You can also give the cause and the remedy:

```
from annunciator import ValidationError

raise ValidationError(
    "The live time must be more than 0 s.",
    remedy="Set the live time to a positive value.",
    context={"live_time": live_time},
)
```

Result

The exception has the attributes `code`, `message`, `severity`, `cause`, `remedy`, `fields` and `context`. The text of the exception starts with the code in square brackets.

5.5.2 Make your own error class

Procedure

1. Make a subclass of `CodedError` or of one of its subclasses.
2. Set the class attribute `default_code` to the code of the usual message.
3. If the program is a command-line program, set `exit_code`.

Example 5.7: An error class with a default code

```
from annunciator import ResourceError

class DetectorError(ResourceError):
    """The detector does not operate."""
```

(continues on next page)

(continued from previous page)

```
    default_code = "APP-E-0001"
    exit_code = 74

raise DetectorError(name="HPGe-2")
```

Result

The class makes the message text from the definition of `APP-E-0001`. You can catch the error as `DetectorError`, `ResourceError` or `CodedError`.

5.5.3 Catch a coded error

Procedure

1. Catch `CodedError` at the top level of the program.
2. Record the error with the method `report`.
3. Show the result of the method `describe` to the user.
4. Stop the program with the attribute `exit_code`.

Example 5.8: Top level of a command-line program

```
import sys

from annunciator import CodedError, configure, get_logger

def main() -> int:
    configure(level="INFO", file="logs/run.jsonl", file_format="json")
    log = get_logger("myapp")
    try:
        run_measurement()
    except CodedError as error:
        log.report(error)
        print(error.describe(), file=sys.stderr)
        return error.exit_code
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

Result

When the program raises a coded error, the log contains one event with the code and the traceback. The user sees the message, the cause and the remedy:

```
[APP-E-0001] The detector 'HPGe-2' does not answer.
Cause: The detector has no power or the cable is not connected.
Remedy: Make sure that the detector has power. Examine the cable.
```

Table 6.9 gives the exit status of each class.

5.5.4 Add context to an error that goes up

Procedure

1. Catch the error at the level that knows the context.
2. Call the method `with_context`.

3. Raise the error again:

```
try:
    analyse(sample)
except CodedError as error:
    raise error.with_context(sample=sample.name)
```

Result

The record of the error contains the field `sample`.

5.6 Issue coded warnings

Purpose

To tell the user about a condition that does not stop the operation.

Before you start

Select the correct tool. Table 5.2 helps you.

Table 5.2: Selection of the tool for a warning

Condition	Tool
The user of your function must be able to ignore the warning or to change it into an exception.	`warn` (this procedure)
The condition is an event of the program, and filters are not necessary.	`log.warning` or `log.event`. Refer to *Record events*.

5.6.1 Issue a warning

Procedure

1. Select a warning class: `CodedWarning`, `ConfigurationWarning`, `DataQualityWarning` or `PerformanceWarning`.
2. Call the function `warn` with the code and the values for the replacement fields:

Example 5.9: A coded warning

```
from annunciator import DataQualityWarning, warn

def dead_time_correction(dead_time):
    if dead_time > 30:
        warn(code="APP-W-0002", category=DataQualityWarning,
             stacklevel=2, dead_time=dead_time)
    return 1.0 / (1.0 - dead_time / 100.0)
```

3. Set `stacklevel` to 2. Then the warning identifies the line that called your function.
4. If the warning has no definition in a catalogue, give the message text as the first argument.

Result

When the option `capture_warnings` is set, the log contains one `WARNING` event with the code. The logger name is `py.warnings`. The fields `filename` and `lineno` identify the line that called your function.

Note

Python shows the same warning from the same line only one time. This is the default behaviour of the standard `warnings` module.

5.6.2 Change a warning into an error in tests

Procedure

1. Use the standard `warnings` module to set a filter:

```
import warnings

from annunciator import DataQualityWarning

warnings.simplefilter("error", DataQualityWarning)
```

Result

Each `DataQualityWarning` is now an exception. A test fails if the code issues the warning.

5.6.3 Mark a function as deprecated

Procedure

1. Add the decorator `deprecated` to the old function. Give the version and the name of the replacement:

```
from annunciator import deprecated

@deprecated("2.0", alternative="load_spectrum", removal="3.0")
def read_spectrum(path):
    return load_spectrum(path)
```

Result

Each call of `read_spectrum` issues a `DeprecatedFeatureWarning` with the code `ANN-W-0309`. The warning identifies the line that called the old function.

5.7 Apply an error policy

Purpose

To handle exceptions in the same way in all parts of a program.

Before you start

Read *the description of the error policy* in *Concept of operations*. Then select the tool in Table 5.3.

Table 5.3: Selection of the handling tool

Task	Tool
Record all exceptions of a function	`handle_errors`
Record all exceptions of a block of code	`guard`
Do an operation again when it fails	`retry`
Continue after an error and show all errors at the end	`ErrorCollector`
Record the duration of a block of code	`timed`
Record each call of a function	`log_calls`

5.7.1 Record the exceptions of a function

Procedure

1. Add the decorator `handle_errors` to the function.
2. Give the exception classes that the decorator must handle.
3. Select the policy with the parameters `reraise`, `default` and `wrap`. Table 5.4 gives the effect.

Example 5.10: Three policies for `handle_errors`

```
from annunciator import DataError, handle_errors

# Record the exception, then raise it again (default).
@handle_errors(OSError)
def load(path):
    return open(path, "rb").read()

# Record the exception, then return a default value.
@handle_errors(ZeroDivisionError, reraise=False, default=float("nan"))
def ratio(a, b):
    return a / b

# Record the exception, then raise a coded error from it.
@handle_errors(KeyError, IndexError, wrap=DataError)
def channel(spectrum, index):
    return spectrum[index]
```

Table 5.4: Policy parameters

Parameters	Effect after the record
No parameter	The decorator raises the initial exception again.
`wrap=CLASS`	The decorator raises `CLASS` from the initial exception.
`reraise=False`	The decorator returns `default`. The exception stops.

Result

Each handled exception makes one event with the traceback. A coded error keeps its code. All other exceptions get the code `ANN-E-0303`.

Caution

Use `reraise=False` only when the default value is a correct result for the code that called the function. If not, the program continues with incorrect data.

5.7.2 Record the exceptions of a block

Procedure

1. Put the block in a `with guard(...)` statement. Give a name for the operation:

```
from annunciator import guard

with guard(OSError, operation="write summary", reraise=False) as result:
    write_summary(path)
if result.failed:
    print("The summary is not available.")
```

Result

If the block raises an `OSError`, the log contains one event. The attribute `failed` is `True` and the program continues after the block.

5.7.3 Do an operation again

Procedure

1. Add the decorator `retry` to the function.
2. Give only the exception classes of temporary conditions.
3. Set the number of attempts and the delay:

Example 5.11: Retry with an increasing delay

```
from annunciator import retry

@retry(ConnectionError, TimeoutError, attempts=4, delay=0.5, backoff=2.0)
def read_register(address):
    return instrument.query(address)
```

Result

The delays before attempts 2, 3 and 4 are 0.5 s, 1 s and 2 s. Each failed attempt makes one `WARNING` event with the code `ANN-W-0301`. If attempt 4 fails, the decorator records `ANN-E-0302`. Then it raises `RetryExhaustedError` from the last exception.

Caution

Do not use `retry` for an operation that is not safe to do two times. An example is an operation that moves a motor by a relative distance.

5.7.4 Collect many errors

Procedure

1. Make an `ErrorCollector` with the name of the operation.
2. Put each item of work in a `with collector.catch(...)` block.
3. After the loop, call `raise_if_errors`:

Example 5.12: Collection of errors

```
from annunciator import ErrorCollector

collector = ErrorCollector("import spectra")
for path in paths:
    with collector.catch(OSError, ValueError, path=str(path)):
        spectra.append(load(path))
print(collector.summary())
collector.raise_if_errors()
```

Result

The loop examines all paths. Each error makes one event. If one or more errors occurred, `raise_if_errors` raises `AggregateError` with the code `ANN-E-0304`. The attribute `errors` contains the exceptions.

5.7.5 Record durations and calls

Procedure

1. To record the duration of a block, use `timed`. Set `warn_after` to get a warning for a slow block:

```
from annunciator import timed

with timed("peak fit", warn_after=5.0, peaks=12):
    fit_peaks(spectrum)
```

2. To record each call of a function, add the decorator `log_calls`:

```
from annunciator import log_calls

@log_calls(arguments=True)
def calibrate(energy, channel):
    ...
```

Caution

The option `arguments` records all arguments of the function. Do not use this option for a function that gets secret data.

Result

Step 1 records `ANN-I-0308` with the duration. If the duration is more than 5 s, it records `ANN-W-0305`. Step 2 records `ANN-D-0306` and `ANN-D-0307` with the severity `DEBUG`.

5.8 Record unhandled exceptions

Purpose

To make sure that the log contains each exception that stops the program or a thread.

Before you start

The function `configure` installs the hooks by default. Do this procedure only in these conditions:

- you set the option `install_hooks` to `False`, or
- you do not use `configure`, or
- your program uses `asyncio`.

5.8.1 Install the hooks

Procedure

1. Call `install_hooks` one time, near the start of the program:

```
from annunciator import install_hooks

install_hooks()
```

2. To remove the hooks, call `uninstall_hooks`.

Result

Table 5.5 gives the events that the hooks record.

Table 5.5: Events of the hooks

Condition	Code	Logger
An exception stops the program	`ANN-C-0201`	`annunciator.hooks`
An exception stops a thread	`ANN-C-0202`	`annunciator.hooks`
Python cannot raise an exception	`ANN-E-0204`	`annunciator.hooks`
A module issues a Python warning	`ANN-W-0205`	`py.warnings`

A coded error keeps its own code. The hook records it with the severity `CRITICAL` and the field `uncaught`.

Note

The hooks do not record `KeyboardInterrupt` and `SystemExit`. These exceptions are a normal stop of the program.

5.8.2 Record the errors of an `asyncio` event loop

Procedure

1. In the first coroutine of the program, call `install_asyncio_handler`:

```
import asyncio

from annunciator import configure, install_asyncio_handler

async def main():
    install_asyncio_handler()
    await run_tasks()

configure()
asyncio.run(main())
```

Result

When a task raises an exception and no code awaits the task, the log contains an event with the code `ANN-E-0203`.

5.9 Examine and summarise a log

Purpose

To find events in a log and to make a summary of a run.

Before you start

For the procedures that use a log file, configure a file with the option `file_format` set to `"json"`. Refer to *Configure the library*.

5.9.1 Make a summary at the end of a program

Procedure

1. Call `summarise` without arguments before the program stops.

2. Show the text of the report.
3. Use the method `exit_code` for the exit status of the program:

Example 5.13: Summary at the end of a program

```python
import sys

from annunciator import summarise

report = summarise()
print(report.to_text())
sys.exit(report.exit_code())
```

Result

The terminal shows the number of events for each severity and for each message code:

```
Events: 14
Worst severity: ERROR
First event: 2026-11-12T13:45:26+00:00
Last event:  2026-11-12T13:45:26+00:00
By severity:
  ERROR    6
  WARNING  5
  INFO     1
  DEBUG    2
By message code:
  ANN-E-0303       3
  APP-E-0001       2
```

Note

The summary uses the event store. The store keeps the last 1000 events by default. To keep more events, increase the option `store_capacity`.

5.9.2 Examine a log file in the terminal

Procedure

1. Show a summary of the file:

```
annunciator report logs/run.jsonl
```

2. Show only the events of the severity `WARNING` and higher:

```
annunciator view logs/run.jsonl --min-severity WARNING
```

3. Show only the events of your catalogue that are errors:

```
annunciator view logs/run.jsonl --code "APP-E-*"
```

4. Write the selected events to a CSV file for a spreadsheet:

```
annunciator view logs/run.jsonl --min-severity WARNING --csv problems.csv
```

Result

Steps 2 and 3 show the events in the plain text format. The exit status of step 1 is 0, 1 or 2. *Command-line interface* gives the meaning.

5.9.3 Examine events in a program

Procedure

1. Read the events with `read_events`, or get them from the event store.
2. Select the events with `select_events`:

```
from annunciator import get_store, read_events, select_events

# From a log file.
slow = select_events(read_events("logs/run.jsonl"), code="ANN-W-0305")
for event in slow:
    print(event.timestamp, event.fields["operation"], event.fields["elapsed"])

# From the event store of the program.
problems = get_store().select(minimum="WARNING")
```

Result

`select_events` gives the events one at a time. `select` returns a list. Each item is an `Event` object. *Record formats and file formats* gives its attributes.

5.9.4 Send events to a different system

Procedure

1. Write a function that accepts one `Event`.
2. Add a `CallbackHandler` with that function to the root logger:

```
import logging

from annunciator import Severity
from annunciator.handlers import CallbackHandler

def show_in_status_bar(event):
    status_bar.set_text(f"{event.severity.name}: {event.message}")

logging.getLogger().addHandler(
    CallbackHandler(show_in_status_bar, level=Severity.WARNING)
)
```

Result

The library calls the function for each event of the severity `WARNING` and higher.

5.10 Document your messages with Sphinx

Purpose

To write the message reference of your own manual automatically from your message catalogue. This manual uses the same procedure for *Messages and problem resolution*.

Before you start

- Define a message catalogue in a module. Refer to *Define a message catalogue*.
- Make a Sphinx project for your manual.
- Make sure that Sphinx can import your package.

Procedure

1. Open the file `conf.py` of your Sphinx project.
2. Add the extension to the list `extensions`:

```
extensions = [
    "annunciator.sphinxext",
]
```

3. In a page of your manual, add the directive. Give the prefix of the catalogue and the module that defines it:

Example 5.14: Directive for a message reference

```
Messages
========

.. annunciator-catalogue:: APP
   :module: myapp.messages
   :min-severity: WARNING
```

4. To refer to one message in your text, use the role `msg`:

```
If the detector has no power, the program shows :msg:`APP-E-0001`.
```

5. Build the manual.

Result

The page contains one section for each message. Each section shows the severity, the message text, the cause and the remedy. Each code is in the index. The role `msg` makes a link to the section of the code.

Note

The options `min-severity` and `category` are optional. Without them, the directive writes all messages of the catalogue.

5.11 Monitor a long run

Purpose

To get the necessary information from a run that operates for hours, without a log that is too large to read.

Before you start

Configure the library. Refer to *Configure the library*. Then select the tool in Table 5.6.

Table 5.6: Selection of the tool for a long run

Task	Tool
Get the detailed records only when a failure occurs	Flight recorder
Record a message in a loop without thousands of equal lines	`once`, `every`, `throttle`
Show that a long operation continues, with the remaining time	`Progress`, `track`
Know when an operation stopped without an error	`Watchdog`
Count how many different failures occurred	Fingerprints

5.11.1 Keep detailed records for failures only

The flight recorder keeps the last detailed records in memory. Figure 5.1 shows its operation. The log file gets the detailed records only when an event of the trigger severity occurs.

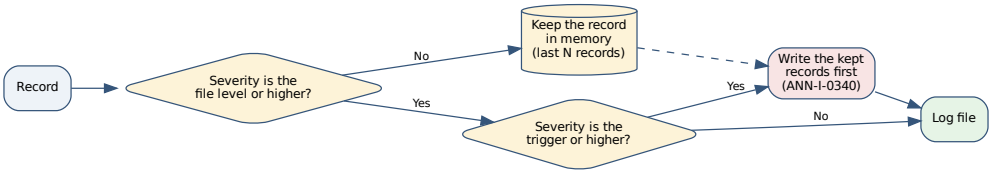


Figure 5.1: Operation of the flight recorder

Procedure

1. Set the option `level` or `file_level` to the severity for a normal run, for example `"INFO"`.
2. Set the option `flight_recorder` to the number of records to keep:

Example 5.15: Configuration of the flight recorder

```
from annunciator import configure

configure(
    level="INFO",
    file="logs/run.jsonl",
    file_format="json",
    flight_recorder=500,
    flight_level="DEBUG",
    flight_trigger="ERROR",
)
```

3. Record detailed data with `log.debug` in your code, as usual.

Result

In a run without errors, the log file contains `INFO` and higher. When an `ERROR` occurs, the file first gets the code `ANN-I-0340`. Then it gets a maximum of 500 `DEBUG` records that came before the error. Each of these records has the field `flight_recorder`.

Note

The kept records have their initial times. Thus the times in the file are not always in sequence after the flight recorder wrote its records.

Caution

If the program stops abnormally without an event of the trigger severity, the kept records are lost. An example is a loss of power.

5.11.2 Record repeated messages

Procedure

1. For a message that the user must see only one time, use `once`:

```
log.once("WARNING", "No calibration file. The default applies.")
```

2. For a message in a loop, use `every` with the number of calls between two records:

```
for index, particle in enumerate(particles):
    log.every(1000, "INFO", "The classification continues.", index=index)
```

3. For a message that can occur at a high rate, use `throttle` with a time in seconds:

```
log.throttle(30.0, "WARNING", "The buffer is almost full.", fill=fill)
```

Result

`once` records the first call. `every` records calls 1, 1001, 2001 and so on, with the field `occurrence`. `throttle` records a maximum of one message in 30 s, with the field `suppressed`.

At the end of the run, the library records `ANN-I-0320` for each message that has rejected records. The event gives the number of rejected records.

Note

The library identifies a message by the logger name and the message text. If the text changes with each call, give the parameter `key`.

5.11.3 Record the progress of an operation

Procedure

1. If the operation is a loop, put `track` around the items:

Example 5.16: Progress of a loop

```
from annunciator import track

for image in track(images, "classify images", interval=30.0):
    classify(image)
```

2. If the operation is not a simple loop, use `Progress` and call `update`:

```
from annunciator import Progress

with Progress("simulate histories", total=1_000_000, interval=60.0) as progress:
```

(continues on next page)

(continued from previous page)

```
    while not simulation.done:
        progress.update(simulation.step())
```

Result

The log contains `ANN-I-0330` a maximum of one time in each interval:

```
INFO  [ANN-I-0330] Progress of 'classify images': 1200 of 4800 (25.0 %). Rate: 9.87␣
↪per second. Time to completion: 365 s.
```

At the end, the log contains `ANN-I-0331` with the number of items and the duration.

5.11.4 Find an operation that stopped

Use this procedure for an operation that can stop without an exception. An example is a device that does not answer.

Procedure

1. If you use `track` or `Progress`, give the parameter `stall_after` with a time in seconds:

   ```
   for image in track(images, "acquire images", stall_after=120.0):
       acquire(image)
   ```

2. If you do not use these tools, make a `Watchdog` and call `beat` each time the operation makes progress:

Example 5.17: Stall detection

```
from annunciator import Watchdog

with Watchdog(120.0, "acquisition") as watchdog:
    while detector.is_acquiring():
        read_block(detector)
        watchdog.beat()
```

Result

If the operation makes no progress for 120 s, the log contains `ANN-W-0332`. The warning occurs again after each subsequent 120 s. When the operation continues, the log contains `ANN-I-0333`.

Note

The watchdog only records the stall. It does not stop the operation. To do an action, give a function in the parameter `on_stall`.

5.11.5 Count the different failures

The library gives a fingerprint to each recorded exception. Two exceptions have the same fingerprint if they have the same class and the same sequence of functions in the traceback.

Procedure

1. Record exceptions with `report`, `exception` or a handling tool. The library then adds the fingerprint automatically.
2. Make a summary of the log. Refer to *Examine and summarise a log*.

Result

The summary contains the section `Distinct failures`:

```
Distinct failures: 2
  46c1ff9f2e2a   3120 x  IndexError: list index out of range
  7e04fdba7321      1 x  ZeroDivisionError: division by zero
```

Thus you know that 3121 error records come from only two defects. To see the events of one failure, select the fingerprint:

```
annunciator view logs/run.jsonl --fingerprint 46c1ff9f2e2a
```

5.12 Record and compare runs

Purpose

To record the data that are necessary to do a run again. Also to show that a log file did not change and to compare two runs.

Before you start

Configure a log file with the option `file_format` set to `"json"`. Refer to *Configure the library*.

5.12.1 Record the manifest of a run

Procedure

1. Put the code of the run in a `with start_run(...)` block.
2. Give the names of the packages that have an effect on the result.
3. Give the seed of the random number generator and the parameters of the run:

Example 5.18: A run with a manifest

```
import random
import sys

from annunciator import configure, start_run

configure(file="logs/run.jsonl", file_format="json")

seed = 20261112
random.seed(seed)
with start_run("depletion", packages=("numpy", "scipy"), seed=seed,
               cooling_time=3.0, burnup=45.0) as run:
    calculate()
    run.save_manifest("logs/run_manifest.json")
sys.exit(run.exit_code)
```

Result

The log contains `ANN-N-0401` at the start of the run. The field `manifest` of that event contains the items in Table 6.17. At the end, the log contains `ANN-N-0402` with the duration and the number of errors and warnings. If an exception stops the run, the log contains `ANN-E-0403`.

All events of the run have the context field `run_id`.

Note

The function records the seed. It does not set the seed. Step 3 shows how to set it.

Note

The manifest does not contain the name of the user by default. To include it, set the parameter `include_user` to `True`.

5.12.2 Make a tamper-evident log file

Use this procedure when a log file is a record that other persons must trust.

Procedure

1. Set the option `file_chain` to true:

```
configure(file="logs/run.jsonl", file_format="json", file_chain=True)
```

2. For protection against a person who can write the file, set a key. Put the key in the environment variable `ANNUNCIATOR_CHAIN_KEY` before you start the program.
3. Run the program.
4. After the run, record the last hash in a different location. The command in the subsequent procedure shows the sequence numbers.

Result

Each record of the log file contains the object `chain`. The object contains a sequence number, the hash of the record before it and its own hash.

Caution

Without a key, a person who can write the file can also calculate a new chain. Use a key and keep it in a location that the person cannot read.

Caution

The chain cannot show that a person removed the last records of a file. To find that, compare the last sequence number with your record from step 4.

5.12.3 Examine the integrity of a log file

Procedure

1. If the program used a key, set the environment variable `ANNUNCIATOR_CHAIN_KEY` to the same key.
2. Do this command:

```
annunciator verify logs/run.jsonl
```

3. Read the result.

Result

If the file is complete and did not change, the command shows this text and the exit status is 0:

```
The integrity check passed: logs/run.jsonl
Records: 47 (sequence 1 to 47)
```

If a record changed or is absent, the command shows the line number of each problem. The exit status is 1:

```
The integrity check FAILED: logs/run.jsonl
  line 6: the contents were changed
  line 9: sequence number 10, expected 9
```

Note

After a rotation, the new file continues the chain of the old file. Then the first sequence number is more than 1. Examine the old file also.

5.12.4 Compare two runs

Procedure

1. Keep the log file of a run that had a correct result. This file is the reference.
2. Compare the log file of the new run with the reference:

   ```
   annunciator diff logs/reference.jsonl logs/run.jsonl
   ```

Result

The command shows the message codes and the failures that are different:

```
Severity counts (first run -> second run):
  ERROR    1 -> 3
Message codes with different counts:
  ANN-E-0303       1 -> 3
New failures in the second run:
  46c1ff9f2e2a  IndexError: list index out of range
Result: the second run has new problems.
```

The exit status is 1 if the second run has a new failure or a new code of the severity `WARNING` or higher. If not, the exit status is 0.

In a program, the function `compare_runs` gives the same data.

5.13 Make the records robust

Purpose

To get correct log files from many processes and to keep the data of a failure. Also to examine data without much code.

5.13.1 Write one log from many processes

Caution

Do not let many processes write to the same log file directly. The lines can mix and the rotation can remove records.

Procedure

1. In the primary process, call `configure`.
2. Call `start_listener` to get the queue.
3. Start the worker processes with `init_worker` as the initializer and the queue as its argument.
4. After the worker processes stop, call `stop_listener`:

Example 5.19: One log file for a pool of processes

```
import multiprocessing

from annunciator import configure, get_logger
from annunciator.multiprocess import init_worker, start_listener, stop_listener

def simulate(case):
    log = get_logger("worker")
    log.info("The case started.", case=case)
    return run_case(case)

if __name__ == "__main__":
    configure(file="logs/run.jsonl", file_format="json")
    queue = start_listener()
    with multiprocessing.Pool(8, init_worker, (queue,)) as pool:
        results = pool.map(simulate, cases)
    stop_listener()
```

Result

Only the primary process writes the log file. Each event contains the identifier of the process that recorded it. The filters, the event store and the flight recorder of the primary process apply to all events.

Note

On Windows, put the code of the primary process below the line `if __name__ == "__main__":`. This is a rule of the standard `multiprocessing` module.

5.13.2 Keep the local variables of a failure

Use this procedure for runs that are too long to do again with a debugger.

Procedure

1. Set the option `capture_locals` to true:

```
configure(file="logs/run.jsonl", file_format="json", capture_locals=True)
```

2. Record exceptions with `report`, `exception` or a handling tool.

Result

Each event with a traceback contains the local variables of the last two functions of the traceback. In the plain text format, the variables are below the traceback:

```
IndexError: list index out of range
Fingerprint: 46c1ff9f2e2a
Local variables of channel_counts (spectrum.py:88):
    password = [REDACTED]
    index = 8192
    counts = [0, 3, 1, 0, 2, ...]
```

The library keeps a maximum of 200 characters for each value.

Caution

Local variables can contain secret data. The redaction filter replaces the values of variables that have secret names. It does not examine the values of other variables. Read a log file before you send it.

5.13.3 Examine data with soft checks

A soft check examines a condition on data and gives a coded message when the condition is not true. You select if the check raises an error or only records a warning.

Procedure

1. Select the check in Table 5.7.
2. Call the check with the value and a name for the value.
3. Select the action with the parameter `on_fail`. Table 5.8 gives the effect.

Example 5.20: Soft checks

```
from annunciator import check, check_finite, check_not_empty, check_range

check_not_empty(spectrum, name="spectrum")
check_range(live_time, 0, None, name="live_time", inclusive=False)
check_finite(efficiency, name="efficiency", on_fail="warn")
if not check_range(dead_time, 0, 30, name="dead_time", on_fail="log"):
    apply_dead_time_correction()
check(len(energy) == len(counts), name="axis length",
      energy=len(energy), counts=len(counts))
```

Table 5.7: Soft checks

Function	Condition that the check examines	Codes
`check_range`	A number is between a minimum and a maximum.	`ANN-E-0601`, `ANN-W-0611`
`check_finite`	The data contain no NaN and no infinite values.	`ANN-E-0602`, `ANN-W-0612`
`check_type`	A value has a permitted type.	`ANN-E-0603`, `ANN-W-0613`
`check_not_empty`	A sequence, a text or a data set contains items.	`ANN-E-0604`, `ANN-W-0614`
`check_one_of`	A value is in a list of permitted values.	`ANN-E-0605`, `ANN-W-0615`
`check`	A condition that you supply is true.	`ANN-E-0606`, `ANN-W-0616`

Table 5.8: Values of the parameter `on_fail`

Value	Effect when the condition is not true
`"raise"`	The check raises `ValidationError` with the `E` code. This is the default.
`"warn"`	The check issues `DataQualityWarning` with the `W` code. The program continues. Warning filters apply.
`"log"`	The check records a `WARNING` event with the `W` code. The program continues.

Result

Each check returns `True` if the condition is true. If the condition is not true, the check does the selected action and returns `False`.

Note

To use a message of your own catalogue, give the parameter `code`.

5.14 Examine your code and share results

Purpose

To find incorrect message codes before a run, to examine events in tests, and to give a readable report of a run to other persons.

5.14.1 Examine the message codes in your source code

Before you start

Define a message catalogue in a module. Refer to *Define a message catalogue*.

Procedure

1. Do the command `lint`. Give the module that defines the catalogue and the directory of your source code:

   ```
   annunciator --import myapp.messages lint src/myapp
   ```

2. Correct each finding of the types `undefined` and `missing-field`.
3. Examine each finding of the type `unused`. Remove the definition if the message is no longer necessary.
4. Do step 1 again.

Result

The command shows one line for each finding. Table 5.9 gives the types. The exit status is 1 if the command found an `undefined` or a `missing-field` problem.

```
src/myapp/daq.py:41: undefined: APP-E-0012
src/myapp/daq.py:57: missing-field: APP-E-0001 (no value for: name)
unused: APP-W-0007 (The buffer is {fill} % full.)
3 findings, 2 errors
```

Table 5.9: Types of findings of the command `lint`

Type	Meaning
`undefined`	The source code uses a code that has no definition. At run time, the message has no text, no cause and no remedy.
`missing-field`	A call gives a code but gives no value for a replacement field. At run time, the message shows the field name in braces.
`unused`	A catalogue defines a code that the source code does not use.

Note

The command reads the source code. It does not run it. It cannot find a code that the program makes at run time.

Note

Add this command to the automatic checks of your project. Then an incorrect code cannot go into a release.

5.14.2 Examine events in a test

Procedure

1. If you use pytest, add the fixture `annunciator_events` to the arguments of the test function. No import is necessary.
2. Call the code that you must examine.
3. Call `assert_logged` or `assert_not_logged` with the conditions:

Example 5.21: A test with the pytest fixture

```
def test_high_dead_time_gives_warning(annunciator_events):
    measure(dead_time=45)

    event = annunciator_events.assert_logged("APP-W-0002", count=1)
    assert event.fields["dead_time"] == 45
    annunciator_events.assert_not_logged(minimum="ERROR")
```

4. If you do not use pytest, use the context manager `capture_events`:

```
from annunciator.testing import capture_events

with capture_events() as captured:
    measure(dead_time=45)
captured.assert_logged("APP-W-0002")
```

Result

The test fails if the necessary event is absent or if a forbidden event occurred. The failure text shows the last 20 recorded events.

Table 5.10: Conditions of `assert_logged` and `assert_not_logged`

Parameter	Condition
`code`	The event has this message code.
`severity`	The event has this severity.
`minimum`	The event has this severity or higher.
`text`	The message contains this text.
`logger`	The event comes from the logger with this name.
`count`	For `assert_logged` only: the number of events that must agree.
Other keywords	The event has a structured field with this name and this value.

5.14.3 Make an HTML report of a run

Procedure

1. Do the command `report` with the option `--html`:

```
annunciator report logs/run.jsonl --html run_report.html --title "Run R-17"
```

2. Open the file `run_report.html` in a browser.

Result

The page has the style of a printed industrial report. It has a title block, four numbered sections and ruled tables in a fixed-width font. It does not use colour. An asterisk identifies each error.

Table 5.11: Sections of the HTML report

Section	Contents
Title block	The title, the date of issue, the number of events, the worst severity and the period of the record.
1. Summary by severity	The number of events for each severity.
2. Summary by message code	The number of events for each message code.
3. Distinct failures	One line for each fingerprint.
4. Event log	One line for each event, with its number in the log.

On the page, you can do these actions:

- Click a severity to show or hide its events.
- Click a message code or a failure to show only its events.
- Type text in the field `FIND` to find events.
- Click an event to show its fields, cause, remedy and traceback.
- Click `EXPAND ALL` to show these data for all listed events.
- Click `PRINT` to print the report. The printed report does not show the controls.

The file has no external parts. You can send it to other persons and they can open it without a network connection.

Caution

The HTML file contains the full data of the events. Make sure that it contains no secret data before you send it.

In a program, the function `write_html_report` writes the same file.

CHAPTER

SIX

REFERENCE INFORMATION

This chapter gives the reference data of the library. Use it to find the permitted values of options, commands and parameters.

6.1 Severities

Table 6.1 gives the seven severities. The numeric values agree with the levels of the standard `logging` module. `TRACE` and `NOTICE` are additions of this library.

Table 6.1: Severities

Name	Value	Letter	Log method	When to use
`TRACE`	5	T	`trace`	Very detailed data, for example each step of a loop.
`DEBUG`	10	D	`debug`	Data that helps a developer to find the cause of a fault.
`INFO`	20	I	`info`	Normal progress of the program.
`NOTICE`	25	N	`notice`	A normal event that is important to the user, for example the end of a run.
`WARNING`	30	W	`warning`	The program continues, but a result is possibly incorrect.
`ERROR`	40	E	`error`	An operation did not complete.
`CRITICAL`	50	C	`critical`	The program cannot continue.

You can give a severity in four forms. All forms have the same effect:

- a member of the class `Severity`, for example `Severity.WARNING`,
- a name, for example `"WARNING"` or `"warning"`,
- a letter, for example `"W"`,
- a number, for example `30`.

The names `WARN`, `ERR` and `FATAL` are also permitted. They select `WARNING`, `ERROR` and `CRITICAL`.

With the default configuration, the console shows the severities `TRACE` to `NOTICE` on `stdout`. It shows `WARNING` to `CRITICAL` on `stderr`.

6.2 Configuration options

This section gives all options of the class `LogConfig`. You can give each option in three ways:

- as a keyword argument of `configure` or `LogConfig`,
- as a key in a TOML file or a JSON file,
- as an environment variable with the name `ANNUNCIATOR_` and the option name in capital letters.

If an option name is not known, the library raises *ANN-E-0001*. If a value is not permitted, the library raises *ANN-E-0002*.

6.2.1 General options

Table 6.2: General options

Option	Type	Default	Function
`level`	severity	`"INFO"`	The minimum severity that the library records.
`logger_name`	text	`""`	The logger that gets the handlers. The empty text selects the root logger.
`logger_levels`	table	empty	The minimum severity for named loggers. The keys are logger names.
`utc`	boolean	`true`	If true, the time is in UTC. If false, the time is the local time with its UTC offset.
`asynchronous`	boolean	`false`	If true, a different thread writes the events. Use this option when a slow destination must not delay the program.
`store_capacity`	integer	`1000`	The number of events in the event store. The value 0 selects no event store.
`install_hooks`	boolean	`true`	If true, the library records exceptions that no code caught.
`capture_warnings`	boolean	`true`	If true, the library records Python warnings.

6.2.2 Console options

Table 6.3: Console options

Option	Type	Default	Function
`console`	boolean	`true`	If true, the library writes events to the console.
`console_level`	severity	none	The minimum severity for the console. Without a value, the library uses `level`.
`console_format`	text	`"auto"`	`"auto"`, `"plain"`, `"colour"` or `"json"`. `"auto"` selects `"colour"` on a terminal and `"plain"` in all other conditions.
`split_streams`	boolean	`true`	If true, `WARNING` and higher go to `stderr` and all lower severities go to `stdout`. If false, all events go to `stderr`.

Note

The environment variable `NO_COLOR` stops the colour in the mode `"auto"`. The environment variable `FORCE_COLOR` selects the colour in the mode `"auto"`.

6.2.3 File options

Table 6.4: File options

Option	Type	Default	Function
`file`	path	none	The path of the log file. Without a value, the library writes no log file.
`file_level`	severity	none	The minimum severity for the log file. Without a value, the library uses `level`.
`file_format`	text	`"plain"`	`"plain"` or `"json"`. Use `"json"` for a file that the report functions must read.
`rotate`	text	`"size"`	`"none"`, `"size"` or `"time"`.
`max_bytes`	integer	`10000000`	For the rotation by size: the maximum size of the file in bytes.
`when`	text	`"midnight"`	For the rotation by time: the interval. The permitted values are those of the standard class `TimedRotatingFileHandler`.
`backup_count`	integer	`5`	The number of old log files that the library keeps.
`encoding`	text	`"utf-8"`	The text encoding of the log file.

Caution

When the rotation removes an old log file, the events in that file are lost. Set `backup_count` to a value that keeps the files for the necessary time.

6.2.4 Text options

These options apply to the formats `"plain"` and `"colour"`.

Table 6.5: Text options

Option	Type	Default	Function
`show_fields`	boolean	`true`	If true, the text shows the structured fields after a vertical bar.
`show_help`	boolean	`true`	If true, the text shows the cause and the remedy for `WARNING` and higher.
`show_source`	boolean	`false`	If true, the text shows the source file and the line number.

6.2.5 Filter options

The text about the default value of `redact_keys` is after Table 6.7.

Table 6.6: Filter options

Option	Type	Default	Function
`redact_keys`	list of text	See below	Parts of field names that identify secret data. Letter case is not important.
`redact_patterns`	list of text	empty	Regular expressions for secret text in messages and in text values.
`duplicate_window`	number	`0`	The time window in seconds. In this window, the library rejects equal records. The value 0 selects no duplicate filter.
`rate_limit`	number	`0`	The maximum mean number of records each second. The value 0 selects no limit. The limit does not apply to `ERROR` and higher.
`rate_burst`	integer	`100`	The maximum number of records at one time for the rate limit.

6.2.6 Diagnostic options

Table 6.7: Diagnostic options

Option	Type	Default	Function
`flight_recorder`	integer	`0`	The number of detailed records that the flight recorder keeps in memory. The value 0 selects no flight recorder.
`flight_level`	severity	`"DEBUG"`	The minimum severity of the records that the flight recorder keeps.
`flight_trigger`	severity	`"ERROR"`	The severity that causes the flight recorder to write the kept records.
`file_chain`	boolean	`false`	If true, each record of the log file contains integrity data. The option `file_format` must be `"json"`.
`capture_locals`	boolean	`false`	If true, each event with a traceback contains the local variables of the last two functions.

The flight recorder is on the log file. If the configuration has no log file, the flight recorder is on the console.

The environment variable `ANNUNCIATOR_CHAIN_KEY` supplies the key for the integrity data. It is not an option of `LogConfig`. Thus a configuration file does not contain the key.

The default value of `redact_keys` is: `password`, `passwd`, `secret`, `token`, `api_key`, `apikey`, `authorization`, `credential`, `private_key`.

6.2.7 Format of environment variables

Table 6.8: Values of environment variables

Option type	Permitted text	Example
boolean	`1`, `true`, `yes`, `on`, `0`, `false`, `no`, `off`	`ANNUNCIATOR_CONSOLE=off`
integer, number	Digits	`ANNUNCIATOR_BACKUP_COUNT=14`
list of text	Items with commas between them	`ANNUNCIATOR_REDACT_KEYS=password,pin`
table	`name=LEVEL` items with commas between them	`ANNUNCIATOR_LOGGER_LEVELS=numba=WARNING`

6.3 Error classes and warning classes

6.3.1 Coded errors

Table 6.9 gives the classes of coded errors. Figure 3.3 shows the relations between the classes. The exit status values agree with the usual values of the file `sysexits.h`.

Table 6.9: Classes of coded errors

Class	Exit status	Default code	Use
`CodedError`	1	None	Base class. Use it when no other class agrees.
`ConfigurationError`	78	None	A configuration value or file is not correct.
`ValidationError`	65	None	An input value is not correct. Also a `ValueError`.
`DataError`	65	None	The contents of a data set are not correct.
`ResourceError`	74	None	A file, a device or a network resource caused an error.
`ResourceNotFoundError`	66	None	A necessary resource does not exist.
`ResourceAccessError`	77	None	The program does not have access to a resource.
`OperationError`	1	None	An operation did not complete.
`OperationTimeoutError`	75	`ANN-E-0310`	An operation did not complete in the permitted time.
`RetryExhaustedError`	75	`ANN-E-0302`	All attempts of an operation failed.
`DependencyError`	69	None	A necessary package, program or service is not available.
`InternalError`	70	None	The program found a defect in its own logic. The severity is `CRITICAL`.
`CatalogueError`	70	None	A message code or a catalogue is not correct.
`AggregateError`	1	`ANN-E-0304`	A group of errors from an `ErrorCollector`.

6.3.2 Attributes of a coded error

Table 6.10: Attributes of coded errors and coded warnings

Attribute	Type	Contents
`code`	`str` or `None`	The message code.
`message`	`str`	The message text with the field values.
`severity`	`Severity`	The severity of the definition, or the default of the class.
`cause`	`str`	The probable cause.
`remedy`	`str`	The corrective action.
`fields`	`dict`	The values of the replacement fields.
`context`	`dict`	More data about the conditions.

Note

The attribute `cause` is a text for the user. The attribute `__cause__` is the initial exception of a `raise ... from` statement.

6.3.3 Coded warnings

Table 6.11: Classes of coded warnings

Class	Default code	Use
`CodedWarning`	None	Base class. A subclass of `UserWarning`.
`ConfigurationWarning`	None	A configuration value is permitted but is possibly not correct.
`DataQualityWarning`	None	The quality of the input data can cause an incorrect result.
`PerformanceWarning`	None	An operation is slow or uses much memory.
`DeprecatedFeatureWarning`	`ANN-W-0309`	The program uses a feature that a future version will remove. Also a `DeprecationWarning`.

6.4 Record formats and file formats

6.4.1 Plain text format

The formats `"plain"` and `"colour"` write one primary line for each event:

```
TIME SEVERITY LOGGER [CODE] MESSAGE | KEY=VALUE ... (FILE:LINE)
```

Table 6.12: Parts of the primary line

Part	Contents
`TIME`	The time in the ISO 8601 format with milliseconds. The suffix `Z` shows that the time is in UTC.
`SEVERITY`	The severity name in a column of eight characters.
`LOGGER`	The name of the logger.
`[CODE]`	The message code. This part is absent for an event without a code.
`MESSAGE`	The message text.
`KEY=VALUE`	The structured fields. The option `show_fields` controls this part.
`(FILE:LINE)`	The source position. The option `show_source` controls this part.

Indented lines can follow the primary line, in this sequence:

1. `Cause:` and the probable cause.
2. `Remedy:` and the corrective action.
3. The traceback of a related exception.

The lines `Cause:` and `Remedy:` occur only for the severity `WARNING` and higher.

6.4.2 JSON Lines format

The format `"json"` writes one JSON object on each line. Table 6.13 gives the keys of the object. The object does not contain keys that have no value.

Table 6.13: Keys of an event object

Key	Type	Contents
`timestamp`	text	The time in UTC, in the ISO 8601 format with the suffix `Z`.
`severity`	text	The severity name.
`level`	number	The numeric value of the severity.
`code`	text	The message code.
`logger`	text	The name of the logger.
`message`	text	The message text.
`fields`	object	The structured fields and the context data.
`cause`	text	The probable cause.
`remedy`	text	The corrective action.
`exception`	object	Data about a related exception. Refer to Table 6.14.
`source`	object	The keys `file`, `line`, `function` and `module`.
`process`	number	The identifier of the process.
`thread`	text	The name of the thread.
`event_id`	text	A unique identifier of the event.
`chain`	object	The integrity data. Refer to Table 6.15. The option `file_chain` controls this key.

Table 6.14: Keys of the object `exception`

Key	Type	Contents
`type`	text	The name of the exception class.
`message`	text	The text of the exception.
`fingerprint`	text	Twelve hexadecimal characters that identify the type of failure.
`traceback`	text	The traceback.
`locals`	object	The local variables. Each key identifies a function, a file and a line. The option `capture_locals` controls this key.

Table 6.15: Keys of the object `chain`

Key	Type	Contents
`seq`	number	The sequence number of the record. The first record has the number 1.
`prev`	text	The hash of the record before this record. For the first record, the value is 64 zeros.
`hash`	text	The hash of `prev`, `seq` and the other keys of the record.

The hash function is SHA-256. With a key, the function is HMAC-SHA-256. The library calculates the hash from the JSON text of the record with sorted keys and without spaces.

Example 6.1: One event in the JSON Lines format (shown on many lines)

```
{
  "timestamp": "2026-11-12T10:15:02.454Z",
  "severity": "WARNING",
  "level": 30,
  "code": "APP-W-0002",
  "logger": "myapp.daq",
  "message": "The dead time is 31 %.",
  "fields": {"run": "R-17", "dead_time": 31},
  "cause": "The count rate is too high for the detector.",
  "remedy": "Increase the distance between the source and the detector.",
```

(continues on next page)

(continued from previous page)

```
  "source": {"file": "daq.py", "line": 88, "function": "acquire", "module": "daq"},
  "process": 4112,
  "thread": "MainThread",
  "event_id": "bd4649eafecf4ec1"
}
```

If a field value has a type that JSON cannot contain, the library converts the value to text.

6.4.3 Catalogue file format

A catalogue file is a JSON file or a TOML file. The method `MessageCatalogue.save` writes the JSON form. The method `MessageCatalogue.load` reads the two forms.

Table 6.16: Keys of a catalogue file

Key	Necessary	Contents
`prefix`	Yes	The catalogue prefix.
`title`	No	A short title for the documentation.
`messages`	No	A list of message objects.
`messages.code`	Yes	The message code.
`messages.text`	Yes	The message text, with optional replacement fields.
`messages.cause`	No	The probable cause.
`messages.remedy`	No	The corrective action.
`messages.category`	No	A group name.
`messages.since`	No	The version of your software that added the message.

Example 6.2: A catalogue file in the JSON form

```
{
  "prefix": "APP",
  "title": "Messages of the spectrum tools",
  "messages": [
    {
      "code": "APP-E-0001",
      "text": "The detector '{name}' does not answer.",
      "cause": "The detector has no power or the cable is not connected.",
      "remedy": "Make sure that the detector has power. Examine the cable."
    }
  ]
}
```

6.4.4 CSV format

The function `export_csv` and the command `view --csv` write a CSV file with one row for each event. The columns are, in this sequence: `timestamp`, `severity`, `code`, `logger`, `message`, `cause`, `remedy`, `exception_type`, `exception_message`, `fields`. The column `fields` contains the structured fields as JSON text.

6.4.5 Run manifest

The function `start_run` records the manifest in the field `manifest` of the event `ANN-N-0401`. The method `Run.save_manifest` writes the same data to a JSON file, with the result of the run.

Table 6.17: Keys of a run manifest

Key	Contents
`annunciator`	The version of this library.
`python`, `implementation`	The version and the type of the Python interpreter.
`platform`, `host`	The operating system and the name of the computer.
`executable`, `argv`, `cwd`	The Python program file, the command line and the current directory.
`pid`	The identifier of the process.
`started`	The start time in UTC.
`packages`	The versions of the packages that you named.
`seed`	The seed that you gave.
`git`	The keys `commit` and `dirty`. `dirty` is true if the source directory has changes that are not committed.
`config_hash`	A short hash of the configuration in use.
`extra`	The other keyword arguments of `start_run`.
`user`	The login name. Only with `include_user`.

The file of `save_manifest` also contains the keys `run`, `run_id`, `status`, `elapsed`, `errors` and `warnings`.

6.4.6 HTML report

The function `write_html_report` and the command `report --html` write one HTML file. The file contains the page, its style and the events. It does not load data from a network. The file contains a maximum of 20000 events by default. The counts at the top of the page include all events.

6.5 Command-line interface

6.5.1 Syntax

```
annunciator [--version] [-i MODULE] [-c FILE] COMMAND [ARGUMENTS]
```

If the terminal cannot find the command `annunciator`, use `python -m annunciator` with the same arguments.

6.5.2 General options

Put the general options before the command.

Table 6.18: General options of the command-line interface

Option	Function
`--version`	Shows the version of the library.
`-h`, `--help`	Shows the help text. You can also put this option after a command.
`-i MODULE`, `--import MODULE`	Imports a module that defines message catalogues. You can give this option more than one time.
`-c FILE`, `--catalogue-file FILE`	Reads a message catalogue from a JSON file or a TOML file. You can give this option more than one time.

6.5.3 Commands

`catalogue`

Shows the messages of the registered catalogues.

```
annunciator catalogue [--prefix PREFIX] [--min-severity SEVERITY]
                      [--search TEXT] [--format {text,json,rst}]
```

Table 6.19: Options of the command `catalogue`

Option	Function
`--prefix PREFIX`	Shows only the catalogue that has this prefix.
`--min-severity SEVERITY`	Shows only the messages of this severity or higher.
`--search TEXT`	Shows only the messages that contain this text.
`--format FORMAT`	`text` (default), `json` or `rst`.

explain

Shows the full definition of one message.

```
annunciator explain CODE
```

report

Shows a summary of a JSON Lines log file.

```
annunciator report PATH [--json] [--html FILE] [--title TEXT] [SELECTION]
```

Table 6.20: Options of the command `report`

Option	Function
`--json`	Writes the summary as a JSON object.
`--html FILE`	Also writes an HTML report of the selected events to the file.
`--title TEXT`	The title of the HTML report.

view

Shows the events of a JSON Lines log file in the plain text format.

```
annunciator view PATH [--no-help] [--csv FILE] [SELECTION]
```

Table 6.21: Options of the command `view`

Option	Function
`--no-help`	Does not show the cause and the remedy.
`--csv FILE`	Writes the selected events to a CSV file and shows no events.

check-config

Examines a configuration file and shows the value of each option.

```
annunciator check-config PATH
```

verify

Examines the integrity data of a JSON Lines log file.

```
annunciator verify PATH [--key-env NAME] [--expected-start HASH]
```

Table 6.22: Options of the command `verify`

Option	Function
`--key-env NAME`	The environment variable that contains the key. The default is `ANNUNCIATOR_CHAIN_KEY`.
`--expected-start HASH`	The last hash of the older file. Use this option for a file after a rotation.

diff

Compares the events of two JSON Lines log files.

```
annunciator diff FIRST SECOND [--json]
```

`FIRST` is the reference run. `SECOND` is the run to examine. The option `--json` writes the comparison as a JSON object.

lint

Examines the message codes in Python source files. Use the general option `--import` to load your catalogue.

```
annunciator lint PATH [PATH ...] [--prefix PREFIX] [--no-unused] [--strict]
```

Table 6.23: Options of the command `lint`

Option	Function
`--prefix PREFIX`	Examines only the catalogue with this prefix. You can give this option more than one time.
`--no-unused`	Does not show the codes that the source code does not use.
`--strict`	Gives the exit status 1 for unused codes also.

demo

Shows one example event of each severity and one event with a code.

```
annunciator demo
```

6.5.4 Selection options

The commands `report` and `view` accept the selection options. An event must agree with all given options.

Table 6.24: Selection options

Option	Function
`--min-severity SEVERITY`	Selects the events of this severity or higher.
`--code PATTERN`	Selects the events with a code that agrees with the pattern, for example `"APP-E-*"`.
`--logger PATTERN`	Selects the events with a logger name that agrees with the pattern.
`--text TEXT`	Selects the events that contain this text in the message.
`--fingerprint TEXT`	Selects the events of the failure that has this fingerprint.

In a pattern, `*` agrees with all characters and `?` agrees with one character. Put a pattern in quotation marks.

6.5.5 Exit status

Table 6.25: Exit status of the command-line interface

Status	Meaning
0	The command completed. For `report`: the log has no errors. For `verify`, `diff` and `lint`: the command found no problem.
1	For `report`: the worst severity in the selected events is `ERROR`. For `verify`: the integrity check failed. For `diff`: the second run has new problems. For `lint`: the source code has incorrect codes.
2	For `report`: the worst severity in the selected events is `CRITICAL`.
64	The command line is not correct.
66	The interface cannot import a module or cannot read a file.
Other	The exit status of the coded error that stopped the command. Refer to Table 6.9.

Note

You can use the exit status of `report` in a script to stop a sequence of steps when a run had errors.

6.6 Programming interface

This section gives the reference data for each public module. The text comes from the documentation text in the source code.

You can import the primary names directly from the package `annunciator`. For example, `from annunciator import get_logger` and `from annunciator.logger import get_logger` give the same function.

6.6.1 Severities (`annunciator.levels`)

Severity levels.

This module defines the severity levels of the library. The levels agree with the numeric levels of the standard `logging` module. The library adds two levels: `TRACE` (5) and `NOTICE` (25).

Examples

```
>>> from annunciator import Severity
>>> Severity.coerce("warn")
<Severity.WARNING: 30>
>>> Severity.WARNING.letter
'W'
```

`class annunciator.levels.Severity(*values)`

Bases: `IntEnum`

Severity of an event.

Each member has the same numeric value as the related level of the `logging` module. Thus you can give a member to all functions that accept a `logging` level.

`TRACE`

Very detailed data for a developer. Value: 5.

`DEBUG`

Data for fault isolation. Value: 10.

`INFO`
: Normal progress of the program. Value: 20.

`NOTICE`
: A normal event that is important to the user. Value: 25.

`WARNING`
: A condition that can cause an incorrect result. Value: 30.

`ERROR`
: An operation did not complete. Value: 40.

`CRITICAL`
: The program cannot continue. Value: 50.

property letter: str
: The letter that identifies the severity in a message code.

  The letter is the first letter of the name, for example `W` for `WARNING`.

property is_problem: bool
: `True` if the severity is `WARNING` or higher.

property is_failure: bool
: `True` if the severity is `ERROR` or higher.

classmethod from_letter(*letter*)
: Return the severity that a code letter identifies.

  Parameters
  letter (*str*) – One of `T`, `D`, `I`, `N`, `W`, `E`, `C`.

  Raises
  ValueError – If the letter does not identify a severity.

  Return type
  Severity

classmethod coerce(*value*)
: Convert a value to a *Severity*.

  Parameters
  value (Severity | *int* | *str*) – A *Severity*, an integer, a name, or a code letter. The names `WARN`, `ERR` and `FATAL` are also correct. Letter case is not important. An integer between two levels gives the lower level.

  Returns
  Severity – The related severity.

  Raises
  - **ValueError** – If the text does not identify a severity.
  - **TypeError** – If the type of the value is not correct.

  Return type
  Severity

annunciator.levels.**SeverityLike**
: Types that *Severity.coerce()* accepts.

  alias of *Severity* | `int` | `str`

annunciator.levels.register_levels()

Give the names TRACE and NOTICE to the logging module.

The package calls this function when you import it. It is safe to call the function again.

Return type
None

6.6.2 Structured loggers (annunciator.logger)

Structured loggers.

A *StructuredLogger* is a thin layer on a logger of the standard logging module. It adds message codes, structured fields and context data to each record.

Use *get_logger()* to get a structured logger.

Examples

```
>>> from annunciator import get_logger
>>> log = get_logger("demo.module")
>>> log.name
'demo.module'
>>> child = log.bind(sample="S-1")
>>> child.bound
{'sample': 'S-1'}
```

class annunciator.logger.StructuredLogger(*logger*, *bound=None*)

Bases: object

Logger that records messages with codes and structured fields.

Do not make an instance directly. Use *get_logger()*.

Parameters

- **logger** (*logging.Logger*) – The logger of the logging module that receives the records.
- **bound** (*Optional[Mapping[str, Any]]*) – Fields that the logger adds to each record.

Notes

Each log method accepts keyword arguments. The logger records these arguments as structured fields. The keywords exc_info, stack_info and stacklevel have the same function as in the logging module.

property name: str

The name of the logger.

property stdlib: Logger

The related logger of the logging module.

property bound: Dict[str, Any]

A copy of the fields that the logger adds to each record.

bind(**fields*)

Return a new logger that adds the given fields to each record.

The initial logger does not change.

Parameters
fields (*Any*)

Return type
StructuredLogger

`set_level`(*severity*)

Set the minimum severity that the logger records.

Parameters
severity (Severity | *int* | *str*)

Return type
None

`is_enabled_for`(*severity*)

Return `True` if the logger records the given severity.

Parameters
severity (Severity | *int* | *str*)

Return type
bool

`log`(*severity*, *message*, **args*, *code=None*, *exc_info=None*, *stack_info=False*, *stacklevel=1*, ***fields*)

Record a message with the given severity.

Parameters

- **severity** (Severity | *int* | *str*) – The severity of the message.
- **message** (*str*) – The message text. The text can contain `%` format specifiers for `args`.
- ***args** (*Any*) – The values for the `%` format specifiers.
- **code** (*str* | *None*) – An optional message code. If the code has a definition, the logger adds the cause and the remedy of the definition.
- **exc_info** (*Any*) – `True`, an exception or an exception tuple. The logger then records the traceback.
- **stack_info** (*bool*) – If `True`, record the call stack.
- **stacklevel** (*int*) – The number of stack frames between this call and the line that the record must identify.
- ****fields** (*Any*) – Structured fields for the record.

Return type
None

`trace`(*message*, **args*, ***kwargs*)

Record a message with the severity `TRACE`.

Parameters

- **message** (*str*)
- **args** (*Any*)
- **kwargs** (*Any*)

Return type
None

`debug`(*message*, **args*, ***kwargs*)

Record a message with the severity `DEBUG`.

Parameters

- **message** (*str*)

- **args** (*Any*)
- **kwargs** (*Any*)

Return type
None

info(*message*, **args*, ***kwargs*)

Record a message with the severity INFO.

Parameters

- **message** (*str*)
- **args** (*Any*)
- **kwargs** (*Any*)

Return type
None

notice(*message*, **args*, ***kwargs*)

Record a message with the severity NOTICE.

Parameters

- **message** (*str*)
- **args** (*Any*)
- **kwargs** (*Any*)

Return type
None

warning(*message*, **args*, ***kwargs*)

Record a message with the severity WARNING.

Parameters

- **message** (*str*)
- **args** (*Any*)
- **kwargs** (*Any*)

Return type
None

error(*message*, **args*, ***kwargs*)

Record a message with the severity ERROR.

Parameters

- **message** (*str*)
- **args** (*Any*)
- **kwargs** (*Any*)

Return type
None

critical(*message*, **args*, ***kwargs*)

Record a message with the severity CRITICAL.

Parameters

- **message** (*str*)
- **args** (*Any*)

- **kwargs** (*Any*)

Return type
None

exception(*message*, **args*, ***kwargs*)

Record an ERROR message with the current traceback.

Call this method only in an except block.

Parameters

- **message** (*str*)
- **args** (*Any*)
- **kwargs** (*Any*)

Return type
None

once(*severity*, *message*, **args*, *key=None*, ***kwargs*)

Record a message only the first time.

Use this method for a condition that the user must know about one time, for example a default value that the program applied.

Parameters

- **severity** (Severity / *int* / *str*) – The severity of the message.
- **message** (*str*) – The message text.
- ***args** (*Any*) – The values for the % format specifiers.
- **key** (*str* / *None*) – The identifier of the message. The default is the logger name with the code or the message text.
- ****kwargs** (*Any*) – The other parameters of *log()*.

Returns
bool – True if the logger recorded the message.

Return type
bool

every(*interval*, *severity*, *message*, **args*, *key=None*, ***kwargs*)

Record a message on the first call and then on each n-th call.

The logger adds the field occurrence with the number of the call.

Parameters

- **interval** (*int*) – The number of calls between two records. The minimum is 1.
- **severity** (Severity / *int* / *str*) – The severity of the message.
- **message** (*str*) – The message text.
- ***args** (*Any*) – The values for the % format specifiers.
- **key** (*str* / *None*) – The identifier of the message.
- ****kwargs** (*Any*) – The other parameters of *log()*.

Returns
bool – True if the logger recorded the message.

Return type
bool

throttle(*seconds*, *severity*, *message*, **args*, *key=None*, ***kwargs*)

Record a message a maximum of one time in a time interval.

When the logger records the message again, it adds the field `suppressed` with the number of rejected records.

Parameters

- **seconds** (*float*) – The minimum time between two records.
- **severity** (Severity / *int* / *str*) – The severity of the message.
- **message** (*str*) – The message text.
- ***args** (*Any*) – The values for the `%` format specifiers.
- **key** (*str* / *None*) – The identifier of the message.
- ****kwargs** (*Any*) – The other parameters of *log()*.

Returns

bool – `True` if the logger recorded the message.

Return type

bool

event(*code*, ** (Keyword-only parameters separator (PEP 3102))*, *strict=False*, *exc_info=None*, *stacklevel=1*, ***fields*)

Record a message from a message catalogue.

The logger gets the severity, the text, the cause and the remedy from the definition of the code.

Parameters

- **code** (*str*) – The message code.
- **strict** (*bool*) – If `True`, raise an exception when the code has no definition. If `False`, record an `ERROR` message that contains the code and the fields.
- **exc_info** (*Any*) – `True`, an exception or an exception tuple.
- **stacklevel** (*int*) – The number of stack frames between this call and the line that the record must identify.
- ****fields** (*Any*) – The values for the replacement fields. The logger also records them as structured fields.

Raises

CatalogueError – If `strict` is `True` and the code has no definition (`ANN-E-0103`).

Return type

None

report(*exc*, ***, *severity=None*, *operation=''*, *traceback=True*, *force=False*, *stacklevel=1*, ***fields*)

Record an exception as an event.

For a coded exception, the logger uses the code, the text, the cause, the remedy, the fields and the context of the exception. For all other exceptions, the logger uses the code `ANN-E-0303`.

The logger records each exception object only one time. Thus an exception that goes through two handlers does not make two records.

Parameters

- **exc** (*BaseException*) – The exception.
- **severity** (Severity / *int* / *str* / *None*) – The severity of the record. The default is the severity of the exception, or `ERROR`.

- **operation** (*str*) – The name of the operation that failed.
- **traceback** (*bool*) – If `True`, record the traceback.
- **force** (*bool*) – If `True`, record the exception again when a logger already recorded it.
- **stacklevel** (*int*) – The number of stack frames between this call and the line that the record must identify.
- ****fields** (*Any*) – More structured fields for the record.

Returns
bool – `True` if the logger made a record.

Return type
bool

`annunciator.logger.get_logger(`*name=None*`)`

Return the structured logger that has the given name.

The function returns the same object each time for the same name.

Parameters
name (*str | None*) – The name of the logger. Use `__name__` in a module. If you give `None`, the function returns the root logger.

Returns
StructuredLogger – The logger.

Return type
StructuredLogger

`annunciator.logger.suppressed_counts()`

Return the number of rejected records for each message key.

The methods `once`, `every` and `throttle` of *StructuredLogger* reject records. The dictionary contains only the keys that have one or more rejected records.

Return type
Dict[str, int]

`annunciator.logger.record_suppressed_summary(`*logger=None*`)`

Record one event for each message that has rejected records.

The function records the code `ANN-I-0320`. Then it sets the counts to zero. The functions *shutdown()* and *finish()* call this function.

Parameters
logger (StructuredLogger | *None*) – The logger for the records. The default is the logger `annunciator`.

Returns
int – The number of events that the function recorded.

Return type
int

`annunciator.logger.reset_occurrences()`

Remove the state of the methods `once`, `every` and `throttle`.

After this call, `once` records its message again.

Return type
None

6.6.3 Context data (annunciator.context)

Context data for events.

Context data are key and value pairs that the library adds to each event. Typical examples are a run identifier, a sample name, or a user name.

The library keeps the context in a `contextvars.ContextVar`. Thus each thread and each `asyncio` task has its own context.

Examples

```
>>> from annunciator import bind, context, get_context, clear_context
>>> bind(run="R-17")
>>> with context(sample="CeO2-04"):
...     sorted(get_context().items())
[('run', 'R-17'), ('sample', 'CeO2-04')]
>>> get_context()
{'run': 'R-17'}
>>> clear_context()
```

annunciator.context.**bind**(***values*)

Add key and value pairs to the context.

The pairs stay in the context until you remove them with *unbind()* or *clear_context()*.

Parameters
****values** (*Any*) – The pairs to add. A new value replaces an old value that has the same key.

Return type
None

annunciator.context.**unbind**(**keys*)

Remove keys from the context.

Parameters
***keys** (*str*) – The keys to remove. The function ignores a key that is not in the context.

Return type
None

annunciator.context.**clear_context**()

Remove all pairs from the context.

Return type
None

annunciator.context.**get_context**()

Return a copy of the context of the current thread or task.

Return type
Dict[str, *Any*]

annunciator.context.**context**(***values*)

Add pairs to the context for the duration of a `with` block.

At the end of the block, the function sets the context back to its initial state.

Parameters
****values** (*Any*) – The pairs to add.

Yields
dict – A copy of the context in the block.

Return type
Iterator[*Dict*[str, *Any*]]

annunciator.context.**new_correlation_id**(*key='correlation_id'*)

Make a unique identifier and add it to the context.

Use a correlation identifier to find all events of one operation.

Parameters
key (*str*) – The context key for the identifier.

Returns
str – The identifier. It has 16 hexadecimal characters.

Return type
str

6.6.4 Message catalogues (annunciator.catalogue)

Message catalogues.

A message catalogue is a set of message definitions. Each definition has a unique message code, a text template, a probable cause and a remedy.

A message code has three parts:

```
APP-E-0042
|   |  |
|   |  +-- message number (four digits)
|   +----- severity letter (T, D, I, N, W, E or C)
+--------- catalogue prefix (two to eight characters)
```

When you use message codes, each message has one definition. The log records, the exceptions and the user documentation then agree.

Examples

```
>>> from annunciator import MessageCatalogue, Severity
>>> cat = MessageCatalogue("DEMO", "Demonstration messages", replace=True)
>>> definition = cat.define(
...     1, Severity.ERROR, "The file '{path}' is not available.",
...     cause="The file does not exist.",
...     remedy="Make sure that the path is correct.",
... )
>>> definition.code
'DEMO-E-0001'
>>> definition.format(path="a.csv")
"The file 'a.csv' is not available."
```

annunciator.catalogue.**CODE_PATTERN = re.compile('^(?P<prefix>[A-Z][A-Z0-9]{1,7})-(?P<letter>[TDINWEC])-(?P<number>\\d{4})$')**

Regular expression that a message code must agree with.

class annunciator.catalogue.**MessageDefinition**(*code*, *text*, *cause=''*, *remedy=''*, *category=''*, *since=''*)

Bases: object

Definition of one message.

Parameters

- **code** (*str*) – The unique message code.

- **text** (*str*) – The message text. The text can contain replacement fields in the `str.format()` syntax, for example `{path}`.
- **cause** (*str*) – The probable cause of the event.
- **remedy** (*str*) – The action that the user must do to correct the condition.
- **category** (*str*) – An optional group name, for example `Configuration`.
- **since** (*str*) – The version of your software that added the message.

property prefix: str

The catalogue prefix of the code.

property severity: *Severity*

The severity that the code letter identifies.

property number: int

The message number of the code.

property field_names: Tuple[str, ...]

The names of the replacement fields in the text.

format(***fields*)

Return the message text with the field values.

If a field value is not available, the placeholder stays in the text. The method does not raise an exception.

Parameters
****fields** (*Any*) – The values for the replacement fields.

Return type
str

to_dict()

Return the definition as a dictionary without empty items.

Return type
Dict[str, str]

classmethod from_dict(*data*)

Make a definition from a dictionary.

Raises
CatalogueError – If the dictionary does not contain a correct code and text.

Parameters
data (*Mapping[str, Any]*)

Return type
MessageDefinition

class annunciator.catalogue.MessageCatalogue(*prefix*, *title=''*, ***, *register=True*, *replace=False*)

Bases: `object`

A set of message definitions that have the same prefix.

Parameters

- **prefix** (*str*) – The catalogue prefix. The prefix has two to eight characters. It starts with a capital letter. The other characters are capital letters or digits.
- **title** (*str*) – A short title for the documentation.
- **register** (*bool*) – If `True`, add the catalogue to the registry of the library. Then *lookup()* can find its messages.

- **replace** (*bool*) – If `True`, replace a registered catalogue that has the same prefix. Use this option in an interactive session.

Raises
CatalogueError – If the prefix is not correct (`ANN-E-0101`) or is already registered (`ANN-E-0105`).

`add`(*definition*)

Add a definition to the catalogue.

Raises
CatalogueError – If the prefix of the code is not the catalogue prefix (`ANN-E-0101`). Also if the code or the message number is already in the catalogue (`ANN-E-0102`).

Parameters
definition (MessageDefinition)

Return type
MessageDefinition

`define`(*number*, *severity*, *text*, *, *cause=''*, *remedy=''*, *category=''*, *since=''*)

Make a definition and add it to the catalogue.

Parameters
- **number** (*int*) – The message number, from 0 to 9999. A number can occur only one time in a catalogue.
- **severity** (Severity / *int* / *str*) – The severity of the message.
- **text** (*str*) – The message text with optional replacement fields.
- **cause** (*str*) – The probable cause.
- **remedy** (*str*) – The corrective action.
- **category** (*str*) – An optional group name.
- **since** (*str*) – The version of your software that added the message.

Returns
MessageDefinition – The new definition.

Return type
MessageDefinition

`find`(*code*)

Return the definition for a code, or `None`.

Parameters
code (*str*)

Return type
MessageDefinition | None

`get`(*code*)

Return the definition for a code.

Raises
CatalogueError – If the code is not in the catalogue (`ANN-E-0103`).

Parameters
code (*str*)

Return type
MessageDefinition

`select`(*, *severity=None*, *minimum=None*, *category=None*, *text=None*)

Return the definitions that agree with all given conditions.

Parameters

- **severity** (Severity | *int* | *str* | *None*) – Keep only the messages that have this severity.
- **minimum** (Severity | *int* | *str* | *None*) – Keep only the messages that have this severity or higher.
- **category** (*str* | *None*) – Keep only the messages in this category.
- **text** (*str* | *None*) – Keep only the messages that contain this text in the code, the text, the cause or the remedy. Letter case is not important.

Return type

List[MessageDefinition]

`to_dict`()

Return the catalogue as a dictionary that JSON can contain.

Return type

Dict[str, *Any*]

`classmethod from_dict`(*data*, *, *register=True*, *replace=False*)

Make a catalogue from a dictionary.

The dictionary must have the structure that *to_dict()* returns.

Parameters

- **data** (*Mapping[str, Any]*)
- **register** (*bool*)
- **replace** (*bool*)

Return type

MessageCatalogue

`classmethod load`(*path*, *, *register=True*, *replace=False*)

Read a catalogue from a JSON file or a TOML file.

Parameters

- **path** (*str* | *Path*) – The path of the file. The suffix must be `.json` or `.toml`.
- **register** (*bool*) – If `True`, add the catalogue to the registry.
- **replace** (*bool*) – If `True`, replace a registered catalogue that has the same prefix.

Raises

CatalogueError – If the library cannot read the file (`ANN-E-0104`).

Return type

MessageCatalogue

`save`(*path*)

Write the catalogue to a JSON file.

Returns

pathlib.Path – The path of the file.

Parameters

path (*str* | *Path*)

Return type

Path

annunciator.catalogue.**parse_code**(*code*)

Divide a message code into its parts.

Parameters
code (*str*) – The message code, for example APP-E-0042.

Returns
tuple – The prefix, the severity and the message number.

Raises
CatalogueError – If the format of the code is not correct (ANN-E-0101).

Return type
Tuple[str, Severity, int]

annunciator.catalogue.**make_code**(*prefix*, *severity*, *number*)

Make a message code from its parts.

Parameters

- **prefix** (*str*) – The catalogue prefix.
- **severity** (Severity / *int* / *str*) – The severity of the message.
- **number** (*int*) – The message number, from 0 to 9999.

Returns
str – The message code.

Raises
CatalogueError – If a part is not correct (ANN-E-0101).

Return type
str

annunciator.catalogue.**register_catalogue**(*catalogue*, *, *replace=False*)

Add a catalogue to the registry of the library.

Parameters

- **catalogue** (MessageCatalogue) – The catalogue to add.
- **replace** (*bool*) – If True, replace a catalogue that has the same prefix.

Raises
CatalogueError – If the prefix is already registered and replace is False (ANN-E-0105).

Return type
MessageCatalogue

annunciator.catalogue.**unregister_catalogue**(*prefix*)

Remove a catalogue from the registry.

Returns
MessageCatalogue or None – The catalogue that the function removed.

Parameters
prefix (*str*)

Return type
MessageCatalogue | None

annunciator.catalogue.**get_catalogue**(*prefix*)

Return the registered catalogue that has the given prefix.

Raises
CatalogueError – If the prefix is not registered (ANN-E-0103).

Parameters
prefix (*str*)

Return type
MessageCatalogue

annunciator.catalogue.**catalogues**()

Return all registered catalogues in the sequence of their prefixes.

Return type
List[MessageCatalogue]

annunciator.catalogue.**lookup**(*code*)

Find a message definition in all registered catalogues.

Parameters
code (*str | None*) – The message code.

Returns
MessageDefinition or None – The definition, or `None` if no registered catalogue contains the code.

Return type
MessageDefinition | None

annunciator.catalogue.**explain**(*code*)

Return a text that gives the full definition of a message.

Parameters
code (*str*) – The message code.

Raises
CatalogueError – If the code is not in a registered catalogue (`ANN-E-0103`).

Return type
str

6.6.5 Coded errors (annunciator.errors)

Coded exceptions.

A coded exception is an exception that has a message code, a probable cause and a remedy. All coded exceptions are subclasses of *CodedError*.

You can use the classes of this module directly. You can also make your own subclasses. Set the class attribute `default_code` to give a standard message code to a subclass.

Examples

```
>>> from annunciator import CodedError, MessageCatalogue
>>> cat = MessageCatalogue("DOC", replace=True)
>>> _ = cat.define(7, "E", "The detector '{name}' does not answer.",
...                remedy="Make sure that the detector has power.")
>>> class DetectorError(CodedError):
...     default_code = "DOC-E-0007"
>>> error = DetectorError(name="HPGe-2")
>>> str(error)
"[DOC-E-0007] The detector 'HPGe-2' does not answer."
>>> error.remedy
'Make sure that the detector has power.'
```

exception annunciator.errors.**CodedError**(*message=None*, *, *code=None*, *context=None*, *cause=None*, *remedy=None*, ***fields*)

Bases: `CodedMixin`, `Exception`

Base class of all coded exceptions.

Parameters

- **message** (*str*) – The message text. If you give `None` and the code has a definition, the class makes the text from the definition.
- **code** (*str | None*) – The message code. If you give `None`, the class uses `default_code`.
- **context** (*Dict[str, Any]*) – More data about the conditions when the error occurred.
- **cause** (*str*) – The probable cause. This text replaces the cause of the definition.
- **remedy** (*str*) – The corrective action. This text replaces the remedy of the definition.
- ****fields** – The values for the replacement fields of the message text.

Return type
None

Notes

The attribute `cause` is a text for the user. It is different from the attribute `__cause__` that Python sets for `raise ... from`.

`default_severity: ClassVar[Severity] = 40`

Severity that the class uses when the code has no definition.

`exit_code: ClassVar[int] = 1`

The exit status that a command-line program can return for this error.

`exception annunciator.errors.ConfigurationError(`*message=None*, *, *code=None*, *context=None*, *cause=None*, *remedy=None*, ***fields*`)`

Bases: *CodedError*

A configuration value or a configuration file is not correct.

Parameters

- **message** (*str*)
- **code** (*str | None*)
- **context** (*Dict[str, Any]*)
- **cause** (*str*)
- **remedy** (*str*)
- **fields** (*Dict[str, Any]*)

Return type
None

`exit_code: ClassVar[int] = 78`

The exit status that a command-line program can return for this error.

`exception annunciator.errors.ValidationError(`*message=None*, *, *code=None*, *context=None*, *cause=None*, *remedy=None*, ***fields*`)`

Bases: *CodedError*, `ValueError`

An input value is not correct.

The class is also a subclass of `ValueError`.

Parameters

- **message** (*str*)
- **code** (*str | None*)
- **context** (*Dict[str, Any]*)
- **cause** (*str*)
- **remedy** (*str*)
- **fields** (*Dict[str, Any]*)

Return type
None

`exit_code: ClassVar[int] = 65`

The exit status that a command-line program can return for this error.

`exception annunciator.errors.DataError(`*message=None*, ***, *code=None*, *context=None*, *cause=None*, *remedy=None*, ***fields*`)`

Bases: *CodedError*

The contents of a data set are not correct or not complete.

Parameters

- **message** (*str*)
- **code** (*str | None*)
- **context** (*Dict[str, Any]*)
- **cause** (*str*)
- **remedy** (*str*)
- **fields** (*Dict[str, Any]*)

Return type
None

`exit_code: ClassVar[int] = 65`

The exit status that a command-line program can return for this error.

`exception annunciator.errors.ResourceError(`*message=None*, ***, *code=None*, *context=None*, *cause=None*, *remedy=None*, ***fields*`)`

Bases: *CodedError*

A file, a device or a network resource caused an error.

Parameters

- **message** (*str*)
- **code** (*str | None*)
- **context** (*Dict[str, Any]*)
- **cause** (*str*)
- **remedy** (*str*)
- **fields** (*Dict[str, Any]*)

Return type
None

exit_code: ClassVar[int] = 74

The exit status that a command-line program can return for this error.

exception annunciator.errors.ResourceNotFoundError(*message=None*, *, *code=None*, *context=None*, *cause=None*, *remedy=None*, ***fields*)

Bases: *ResourceError*

A necessary resource does not exist.

Parameters

- **message** (*str*)
- **code** (*str | None*)
- **context** (*Dict[str, Any]*)
- **cause** (*str*)
- **remedy** (*str*)
- **fields** (*Dict[str, Any]*)

Return type
None

exit_code: ClassVar[int] = 66

The exit status that a command-line program can return for this error.

exception annunciator.errors.ResourceAccessError(*message=None*, *, *code=None*, *context=None*, *cause=None*, *remedy=None*, ***fields*)

Bases: *ResourceError*

The program does not have access to a resource.

Parameters

- **message** (*str*)
- **code** (*str | None*)
- **context** (*Dict[str, Any]*)
- **cause** (*str*)
- **remedy** (*str*)
- **fields** (*Dict[str, Any]*)

Return type
None

exit_code: ClassVar[int] = 77

The exit status that a command-line program can return for this error.

exception annunciator.errors.OperationError(*message=None*, *, *code=None*, *context=None*, *cause=None*, *remedy=None*, ***fields*)

Bases: *CodedError*

An operation did not complete.

Parameters

- **message** (*str*)
- **code** (*str | None*)
- **context** (*Dict[str, Any]*)

- **cause** (*str*)
- **remedy** (*str*)
- **fields** (*Dict[str, Any]*)

Return type
None

exception annunciator.errors.**OperationTimeoutError**(*message=None*, ***, *code=None*, *context=None*, *cause=None*, *remedy=None*, ***fields*)

Bases: *OperationError*

An operation did not complete in the permitted time.

Parameters

- **message** (*str*)
- **code** (*str | None*)
- **context** (*Dict[str, Any]*)
- **cause** (*str*)
- **remedy** (*str*)
- **fields** (*Dict[str, Any]*)

Return type
None

default_code: ClassVar[str | None] = 'ANN-E-0310'

Message code that the class uses when you do not give a code.

exit_code: ClassVar[int] = 75

The exit status that a command-line program can return for this error.

exception annunciator.errors.**RetryExhaustedError**(*message=None*, ***, *code=None*, *context=None*, *cause=None*, *remedy=None*, ***fields*)

Bases: *OperationError*

All attempts of an operation failed.

The *retry()* decorator raises this exception. The attribute `__cause__` contains the last exception.

Parameters

- **message** (*str*)
- **code** (*str | None*)
- **context** (*Dict[str, Any]*)
- **cause** (*str*)
- **remedy** (*str*)
- **fields** (*Dict[str, Any]*)

Return type
None

default_code: ClassVar[str | None] = 'ANN-E-0302'

Message code that the class uses when you do not give a code.

exit_code: ClassVar[int] = 75

The exit status that a command-line program can return for this error.

exception annunciator.errors.DependencyError(*message=None*, *, *code=None*, *context=None*, *cause=None*, *remedy=None*, ***fields*)

Bases: *CodedError*

A necessary package, program or service is not available.

Parameters

- **message** (*str*)
- **code** (*str | None*)
- **context** (*Dict[str, Any]*)
- **cause** (*str*)
- **remedy** (*str*)
- **fields** (*Dict[str, Any]*)

Return type
None

exit_code: ClassVar[int] = 69

The exit status that a command-line program can return for this error.

exception annunciator.errors.InternalError(*message=None*, *, *code=None*, *context=None*, *cause=None*, *remedy=None*, ***fields*)

Bases: *CodedError*

The program found a defect in its own logic.

Parameters

- **message** (*str*)
- **code** (*str | None*)
- **context** (*Dict[str, Any]*)
- **cause** (*str*)
- **remedy** (*str*)
- **fields** (*Dict[str, Any]*)

Return type
None

default_severity: ClassVar[*Severity*] = 50

Severity that the class uses when the code has no definition.

exit_code: ClassVar[int] = 70

The exit status that a command-line program can return for this error.

exception annunciator.errors.CatalogueError(*message=None*, *, *code=None*, *context=None*, *cause=None*, *remedy=None*, ***fields*)

Bases: *CodedError*

A message code or a message catalogue is not correct.

Parameters

- **message** (*str*)
- **code** (*str | None*)
- **context** (*Dict[str, Any]*)
- **cause** (*str*)

- **remedy** (*str*)
- **fields** (*Dict[str, Any]*)

Return type
None

`exit_code: ClassVar[int] = 70`

The exit status that a command-line program can return for this error.

`exception annunciator.errors.AggregateError(message=None, errors=(), **kwargs)`

Bases: *CodedError*

A group of errors that occurred in one operation.

The *ErrorCollector* class raises this exception.

Parameters

- **message** (*str*) – The message text.
- **errors** (*Sequence[BaseException]*) – The exceptions of the group.
- ****kwargs** (*Any*) – The other parameters of *CodedError*.

Return type
None

`errors`

The exceptions of the group.

Type
list of BaseException

`default_code: ClassVar[str | None] = 'ANN-E-0304'`

Message code that the class uses when you do not give a code.

`describe()`

Return the message and one line for each error of the group.

Return type
str

6.6.6 Coded warnings (annunciator.warns)

Coded warnings.

A coded warning is a Python warning that has a message code, a probable cause and a remedy. Use *warn()* to issue a coded warning.

The standard `warnings` module controls the warnings. Thus the user can use warning filters to ignore a warning or to change it into an exception. When the exception hooks are installed, the library records each warning in the log.

Examples

```
>>> import warnings
>>> from annunciator import warn, DataQualityWarning
>>> with warnings.catch_warnings(record=True) as caught:
...     warnings.simplefilter("always")
...     warn("The spectrum has only 12 counts.", DataQualityWarning)
>>> str(caught[0].message)
'The spectrum has only 12 counts.'
```

exception annunciator.warns.CodedWarning(*message=None*, *, *code=None*, *context=None*, *cause=None*, *remedy=None*, ***fields*)

Bases: CodedMixin, UserWarning

Base class of all coded warnings.

The parameters are the same as the parameters of *CodedError*.

Parameters

- **message** (*str*)
- **code** (*str | None*)
- **context** (*Dict[str, Any]*)
- **cause** (*str*)
- **remedy** (*str*)
- **fields** (*Dict[str, Any]*)

Return type
None

default_severity: ClassVar[*Severity*] = 30

Severity that the class uses when the code has no definition.

exception annunciator.warns.ConfigurationWarning(*message=None*, *, *code=None*, *context=None*, *cause=None*, *remedy=None*, ***fields*)

Bases: *CodedWarning*

A configuration value is permitted but is possibly not correct.

Parameters

- **message** (*str*)
- **code** (*str | None*)
- **context** (*Dict[str, Any]*)
- **cause** (*str*)
- **remedy** (*str*)
- **fields** (*Dict[str, Any]*)

Return type
None

exception annunciator.warns.DataQualityWarning(*message=None*, *, *code=None*, *context=None*, *cause=None*, *remedy=None*, ***fields*)

Bases: *CodedWarning*

The quality of the input data can cause an incorrect result.

Parameters

- **message** (*str*)
- **code** (*str | None*)
- **context** (*Dict[str, Any]*)
- **cause** (*str*)
- **remedy** (*str*)
- **fields** (*Dict[str, Any]*)

Return type
None

exception annunciator.warns.PerformanceWarning(*message=None*, *, *code=None*, *context=None*, *cause=None*, *remedy=None*, ***fields*)

Bases: *CodedWarning*

An operation is slower or uses more memory than is usual.

Parameters

- **message** (*str*)
- **code** (*str | None*)
- **context** (*Dict[str, Any]*)
- **cause** (*str*)
- **remedy** (*str*)
- **fields** (*Dict[str, Any]*)

Return type
None

exception annunciator.warns.DeprecatedFeatureWarning(*message=None*, *, *code=None*, *context=None*, *cause=None*, *remedy=None*, ***fields*)

Bases: *CodedWarning*, DeprecationWarning

The program uses a feature that a future version will remove.

The class is also a subclass of DeprecationWarning.

Parameters

- **message** (*str*)
- **code** (*str | None*)
- **context** (*Dict[str, Any]*)
- **cause** (*str*)
- **remedy** (*str*)
- **fields** (*Dict[str, Any]*)

Return type
None

default_code: ClassVar[str | None] = 'ANN-W-0309'

Message code that the class uses when you do not give a code.

annunciator.warns.warn(*message=None*, *category=<class 'annunciator.warns.CodedWarning'>*, *, *code=None*, *stacklevel=1*, *context=None*, ***fields*)

Issue a coded warning.

Parameters

- **message** (*str | None*) – The message text. If you give None, the function makes the text from the definition of the code.
- **category** (*Type[CodedWarning]*) – The warning class. It must be a subclass of *CodedWarning*.
- **code** (*str | None*) – The message code.

- **stacklevel** (*int*) – The number of stack frames between this call and the line that the warning must identify. The value 1 identifies the line that calls this function.
- **context** (*Mapping[str, Any] | None*) – More data about the conditions.
- ****fields** (*Any*) – The values for the replacement fields of the message text.

Raises
TypeError – If the category is not a subclass of *CodedWarning*.

Return type
None

6.6.7 Events (annunciator.events)

Event records.

An *Event* is the structured form of one log record. The JSON formatter, the event store and the report functions use this class.

Examples

```
>>> from annunciator import Event, Severity
>>> event = Event(severity=Severity.WARNING, message="Low count rate",
...               code="APP-W-0003", fields={"rate": 1.5})
>>> data = event.to_dict()
>>> data["severity"], data["code"], data["fields"]
('WARNING', 'APP-W-0003', {'rate': 1.5})
>>> Event.from_dict(data).message
'Low count rate'
```

class annunciator.events.**Event**(*severity*, *message*, *code=None*, *logger=''*, *fields=<factory>*, *cause=''*, *remedy=''*, *exception=None*, *source=None*, *timestamp=<factory>*, *process=<factory>*, *thread=''*, *event_id=<factory>*)

Bases: `object`

One event that the library recorded.

Parameters

- **severity** (annunciator.levels.Severity) – The severity of the event.
- **message** (*str*) – The message text.
- **code** (*str | None*) – The message code, or `None`.
- **logger** (*str*) – The name of the logger that recorded the event.
- **fields** (*Dict[str, Any]*) – The structured data of the event. The context data are included.
- **cause** (*str*) – The probable cause.
- **remedy** (*str*) – The corrective action.
- **exception** (annunciator.events.ExceptionInfo | *None*) – Data about a related exception.
- **source** (annunciator.events.SourceLocation | *None*) – The position in the source code.
- **timestamp** (*datetime.datetime*) – The time of the event. The time zone is UTC.
- **process** (*int*) – The identifier of the process.

- **thread** (*str*) – The name of the thread.
- **event_id** (*str*) – A unique identifier of the event.

`to_dict()`

Return the event as a dictionary.

The dictionary does not contain empty items. The time is an ISO 8601 text in UTC with the suffix `Z`.

Return type
Dict[str, *Any*]

`to_json()`

Return the event as one line of JSON text.

Return type
str

`classmethod from_dict(`*data*`)`

Make an event from a dictionary.

The dictionary must have the structure that *to_dict()* returns.

Raises
ValueError – If the dictionary does not contain a severity and a message.

Parameters
data (*Mapping[str, Any]*)

Return type
Event

`classmethod from_json(`*text*`)`

Make an event from one line of JSON text.

Parameters
text (*str*)

Return type
Event

`classmethod from_record(`*record*`)`

Make an event from a record of the `logging` module.

The method reads the attributes that *StructuredLogger* adds to the record. It also accepts records from standard loggers. If the record has a coded exception and no message code, the method uses the code, the cause and the remedy of the exception.

Parameters
record (*LogRecord*)

Return type
Event

`class annunciator.events.ExceptionInfo(`*type*, *message*, *traceback=None*, *fingerprint=''*, *locals=None*`)`

Bases: `object`

Data about an exception that is related to an event.

Parameters

- **type** (*str*) – The name of the exception class.
- **message** (*str*) – The text of the exception.
- **traceback** (*str | None*) – The traceback text, or `None`.

- **fingerprint** (*str*) – The identifier of the type of failure. Refer to *fingerprint()*.
- **locals** (*Dict[str, Dict[str, str]] | None*) – The local variables of the last frames, or `None`. The keys identify the frames. The option `capture_locals` controls this item.

`to_dict()`

Return the data as a dictionary without empty items.

Return type
Dict[str, *Any*]

`classmethod from_dict(data)`

Make the data object from a dictionary.

Parameters
data (*Mapping[str, Any]*)

Return type
ExceptionInfo

`classmethod from_exception(exc, *, with_traceback=True)`

Make the data object from an exception.

Parameters

- **exc** (*BaseException*) – The exception.
- **with_traceback** (*bool*) – If `True`, include the traceback text.

Return type
ExceptionInfo

`class annunciator.events.SourceLocation(file='', line=0, function='', module='')`

Bases: `object`

The position in the source code that made an event.

Parameters

- **file** (*str*) – The path of the source file.
- **line** (*int*) – The line number.
- **function** (*str*) – The name of the function.
- **module** (*str*) – The name of the module.

`to_dict()`

Return the location as a dictionary.

Return type
Dict[str, *Any*]

`classmethod from_dict(data)`

Make a location from a dictionary.

Parameters
data (*Mapping[str, Any]*)

Return type
SourceLocation

`annunciator.events.json_default(value)`

Convert a value that JSON cannot contain to a value that it can.

The function converts dates to ISO 8601 text, sets to sorted lists, and bytes to hexadecimal text. It converts all other types to their `str()` form.

Parameters
value (*Any*)

Return type
Any

annunciator.events.**fingerprint**(*type_name*, *tb*)

Return a short identifier for a type of failure.

Two exceptions have the same fingerprint if they have the same class and the same sequence of functions in the traceback. The line numbers and the message text have no effect. Thus you can count how many times the same failure occurred.

Parameters

- **type_name** (*str*) – The name of the exception class.
- **tb** (*Any*) – The traceback object, or `None`.

Returns
str – Twelve hexadecimal characters.

Return type
str

6.6.8 Configuration (annunciator.config)

Configuration.

The *configure()* function installs the handlers, the formatters and the filters that a *LogConfig* object selects. You can make the configuration in the program, or read it from a TOML file, a JSON file or the environment variables.

Examples

```
>>> from annunciator import LogConfig
>>> config = LogConfig(level="debug", file="run.log", file_format="json")
>>> config.level
'DEBUG'
>>> LogConfig.from_dict({"level": "warning", "console": False}).level
'WARNING'
```

class annunciator.config.**LogConfig**(*level='INFO'*, *logger_name=''*, *console=True*, *console_level=None*, *console_format='auto'*, *split_streams=True*, *file=None*, *file_level=None*, *file_format='plain'*, *rotate='size'*, *max_bytes=10000000*, *backup_count=5*, *when='midnight'*, *encoding='utf-8'*, *utc=True*, *show_fields=True*, *show_help=True*, *show_source=False*, *redact_keys=('password', 'passwd', 'secret', 'token', 'api_key', 'apikey', 'authorization', 'credential', 'private_key')*, *redact_patterns=()*, *duplicate_window=0.0*, *rate_limit=0.0*, *rate_burst=100*, *store_capacity=1000*, *capture_warnings=True*, *install_hooks=True*, *asynchronous=False*, *logger_levels=<factory>*, *flight_recorder=0*, *flight_level='DEBUG'*, *flight_trigger='ERROR'*, *file_chain=False*, *capture_locals=False*)

Bases: `object`

The configuration of the library.

All parameters are optional. The configuration reference in the user manual gives the same data in a table.

Parameters

- **level** (*str*) – The minimum severity that the library records.
- **logger_name** (*str*) – The name of the logger that gets the handlers. The empty text selects the root logger.
- **console** (*bool*) – If `True`, write events to the console.
- **console_level** (*str | None*) – The minimum severity for the console. `None` selects `level`.
- **console_format** (*str*) – `"auto"`, `"plain"`, `"colour"` or `"json"`. `"auto"` selects `"colour"` on a terminal and `"plain"` in all other conditions.
- **split_streams** (*bool*) – If `True`, write `WARNING` and higher to `stderr` and all lower severities to `stdout`. If `False`, write all events to `stderr`.
- **file** (*str | None*) – The path of a log file. `None` selects no log file.
- **file_level** (*str | None*) – The minimum severity for the log file. `None` selects `level`.
- **file_format** (*str*) – `"plain"` or `"json"`.
- **rotate** (*str*) – `"none"`, `"size"` or `"time"`.
- **max_bytes** (*int*) – The maximum size of the log file for the rotation by size.
- **backup_count** (*int*) – The number of old log files to keep.
- **when** (*str*) – The interval for the rotation by time, for example `"midnight"`.
- **encoding** (*str*) – The text encoding of the log file.
- **utc** (*bool*) – If `True`, write the time in UTC.
- **show_fields** (*bool*) – If `True`, write the structured fields in plain text.
- **show_help** (*bool*) – If `True`, write the cause and the remedy in plain text.
- **show_source** (*bool*) – If `True`, write the source file and the line number in plain text.
- **redact_keys** (*Tuple[str, ...]*) – Parts of field names that identify secret data.
- **redact_patterns** (*Tuple[str, ...]*) – Regular expressions for secret text.
- **duplicate_window** (*float*) – The time window in seconds for the duplicate filter. The value 0 selects no duplicate filter.
- **rate_limit** (*float*) – The maximum mean number of records each second. The value 0 selects no rate limit.
- **rate_burst** (*int*) – The maximum number of records at one time for the rate limit.
- **store_capacity** (*int*) – The number of events that the event store keeps. The value 0 selects no event store.
- **capture_warnings** (*bool*) – If `True`, record Python warnings in the log.
- **install_hooks** (*bool*) – If `True`, record exceptions that no code caught.
- **asynchronous** (*bool*) – If `True`, write the events in a different thread.
- **logger_levels** (*Dict[str, str]*) – The minimum severity for named loggers. The keys are logger names.
- **flight_recorder** (*int*) – The number of detailed records that the flight recorder keeps in memory. The value 0 selects no flight recorder.

- **flight_level** (*str*) – The minimum severity of the records that the flight recorder keeps.
- **flight_trigger** (*str*) – The severity that causes the flight recorder to write the kept records.
- **file_chain** (*bool*) – If `True`, add integrity data to each record of the log file. The option `file_format` must be `"json"`.
- **capture_locals** (*bool*) – If `True`, record the local variables of the last frames of each traceback.

Raises
ConfigurationError – If a value is not permitted (ANN-E-0002).

classmethod option_names()

Return the names of all configuration options.

Return type
Tuple[str, ...]

classmethod from_dict(*data*)

Make a configuration from a dictionary.

Raises
ConfigurationError – If the dictionary contains an option that is not known (ANN-E-0001) or a value that is not permitted (ANN-E-0002).

Parameters
data (*Mapping[str, Any]*)

Return type
LogConfig

classmethod from_file(*path*)

Read a configuration from a TOML file or a JSON file.

In a TOML file, the options can be in the table `[tool.annunciator]`, in the table `[annunciator]` or at the top level. In a JSON file, the options can be in the object `"annunciator"` or at the top level.

Raises
ConfigurationError – If the library cannot read the file (ANN-E-0003) or does not support the file type (ANN-E-0004).

Parameters
path (*str | Path*)

Return type
LogConfig

classmethod from_env(*environ=None*, *, *prefix='ANNUNCIATOR_'*, *base=None*)

Make a configuration from the environment variables.

The name of a variable is the prefix and the option name in capital letters, for example `ANNUNCIATOR_LEVEL`. For a sequence, divide the items with commas. For `logger_levels`, use the form `name=LEVEL,name=LEVEL`.

Parameters

- **environ** (*Mapping[str, str] | None*) – The mapping to read. The default is `os.environ`.
- **prefix** (*str*) – The prefix of the variable names.
- **base** (LogConfig | *None*) – A configuration that gives the values for the options that are not in the environment.

Raises
ConfigurationError – If a value is not permitted (ANN-E-0002).

Return type
LogConfig

`to_dict()`

Return the configuration as a dictionary.

Return type
Dict[str, *Any*]

`replace(**changes)`

Return a copy of the configuration with changed options.

Raises
ConfigurationError – If an option is not known (ANN-E-0001).

Parameters
changes (*Any*)

Return type
LogConfig

`annunciator.config.configure(config=None, **options)`

Configure the library.

The function removes the handlers of a previous call. Then it installs the handlers, the formatters, the filters and the hooks that the configuration selects. It does not remove handlers that other code installed.

Parameters

- **config** (LogConfig | *Mapping[str, Any]* | *str* | *Path* | *None*) – A *LogConfig*, a dictionary of options, or the path of a TOML file or a JSON file. None selects the default values.
- ****options** (*Any*) – Options that replace the values of config.

Returns
LogConfig – The configuration in use.

Raises
ConfigurationError – If the configuration is not correct.

Return type
LogConfig

Examples

```
>>> from annunciator import configure, shutdown
>>> config = configure(level="DEBUG", console=False, install_hooks=False)
>>> config.level
'DEBUG'
>>> shutdown()
```

`annunciator.config.shutdown()`

Remove the handlers and the hooks that *configure()* installed.

The function writes all records that are in the queue. Then it closes the log file. It is safe to call the function when the library is not configured.

Return type
None

annunciator.config.is_configured()

Return True if *configure()* installed the handlers.

Return type
bool

annunciator.config.current_config()

Return the configuration in use, or None.

Return type
LogConfig | None

annunciator.config.get_store()

Return the event store of the configuration, or None.

The event store is available when the option store_capacity is more than 0.

Return type
EventStoreHandler | None

annunciator.config.ENV_PREFIX = 'ANNUNCIATOR_'

Prefix of the environment variables that *LogConfig.from_env()* reads.

6.6.9 Handlers (annunciator.handlers)

Handlers.

A handler sends log records to a destination. This module adds four handlers to the handlers of the standard logging module:

- *ConsoleHandler* writes to the current sys.stdout or sys.stderr.
- *EventStoreHandler* keeps the last events in memory.
- *CallbackHandler* gives each event to a function.
- *AsyncQueueHandler* puts records in a queue for a different thread.

Examples

```
>>> import logging
>>> from annunciator.handlers import EventStoreHandler
>>> store = EventStoreHandler(capacity=10)
>>> logger = logging.getLogger("doctest.handlers")
>>> logger.addHandler(store)
>>> logger.propagate = False
>>> logger.warning("Low pressure")
>>> [event.message for event in store.events]
['Low pressure']
>>> store.counts()
{'WARNING': 1}
```

class annunciator.handlers.ConsoleHandler(*stream_name='stderr'*)

Bases: StreamHandler

Handler that writes to the current standard stream.

The handler finds sys.stdout or sys.stderr each time it writes. Thus it operates correctly when a test tool or a notebook replaces the standard streams.

Parameters
stream_name (*str*) – "stdout" or "stderr".

property stream

The current standard stream.

class annunciator.handlers.**EventStoreHandler**(*capacity=1000*, *level=Severity.TRACE*)

Bases: Handler

Handler that keeps the last events in memory.

Use the store to make a summary at the end of a program. You can also show the events in a graphical user interface or examine them in a test.

Parameters

- **capacity** (*int*) – The maximum number of events. When the store is full, the handler removes the oldest event.
- **level** (SeverityLike) – The minimum severity that the handler keeps.

emit(*record*)

Add the event of a record to the store.

Parameters
record (*LogRecord*)

Return type
None

property events: List[*Event*]

A list of the events in the store, oldest first.

counts()

Return the number of events for each severity name.

The numbers include the events that the handler removed because the store was full.

Return type
Dict[str, int]

select(*, *minimum=None*, *code=None*, *logger=None*)

Return the events that agree with all given conditions.

Parameters

- **minimum** (Severity | *int* | *str* | *None*) – Keep only the events of this severity or higher.
- **code** (*str* | *None*) – Keep only the events that have this message code.
- **logger** (*str* | *None*) – Keep only the events of this logger and of its children.

Return type
List[Event]

clear()

Remove all events and set all counts to zero.

Return type
None

class annunciator.handlers.**CallbackHandler**(*callback*, *level=Severity.TRACE*)

Bases: Handler

Handler that gives each event to a function.

Use this handler to send events to a status bar, a message queue or an alarm system.

Parameters

- **callback** (*Callable*[[Event], *None*]) – A function that accepts one *Event*.
- **level** (SeverityLike) – The minimum severity that the handler sends.

Notes

If the function raises an exception, the handler uses the standard error procedure of the `logging` module. The exception does not stop the program.

`emit`(*record*)

Give the event of a record to the function.

Parameters
`record` (*LogRecord*)

Return type
None

`class annunciator.handlers.AsyncQueueHandler`(*queue*)

Bases: `QueueHandler`

Queue handler that keeps the structured data of each record.

The standard queue handler changes the exception data into text. This handler keeps the exception data and the context data. Thus the handlers on the other side of the queue can write complete events.

`prepare`(*record*)

Make a record that is safe to give to a different thread.

Parameters
`record` (*LogRecord*)

Return type
LogRecord

`annunciator.handlers.make_file_handler`(*path*, ***, *rotate='size'*, *max_bytes=10000000*, *backup_count=5*, *when='midnight'*, *encoding='utf-8'*, *utc=True*)

Make a file handler with the selected rotation.

The function makes the parent directory if it does not exist.

Parameters

- `path` (*str | Path*) – The path of the log file.
- `rotate` (*str*) – `"none"`, `"size"` or `"time"`.
- `max_bytes` (*int*) – For `"size"`: the maximum size of the file in bytes.
- `backup_count` (*int*) – The number of old files to keep.
- `when` (*str*) – For `"time"`: the rotation interval of `logging.handlers.TimedRotatingFileHandler`.
- `encoding` (*str*) – The text encoding of the file.
- `utc` (*bool*) – For `"time"`: if `True`, use UTC for the rotation time.

Returns
logging.Handler – The file handler.

Raises
`ValueError` – If `rotate` is not a permitted value.

Return type
Handler

6.6.10 Formatters (annunciator.formatters)

Formatters.

A formatter changes a log record into text. The module has three formatters:

- *PlainFormatter* writes one line of text for each event. It can add the cause, the remedy and the traceback on subsequent lines.
- *ColourFormatter* writes the same text with ANSI colour codes.
- *JsonFormatter* writes one JSON object on one line (JSON Lines).

Examples

```
>>> import logging
>>> from annunciator.formatters import PlainFormatter
>>> from annunciator import Event, Severity
>>> event = Event(severity=Severity.ERROR, message="No beam",
...               code="APP-E-0001", logger="daq", fields={"run": 4})
>>> line = PlainFormatter().format_event(event)
>>> line.split(" ", 1)[1]
'ERROR    daq [APP-E-0001] No beam | run=4'
```

class annunciator.formatters.**PlainFormatter**(*, *utc=True*, *show_fields=True*, *show_help=True*, *show_source=False*, *show_traceback=True*)

Bases: Formatter

Formatter that writes readable text.

The first line has this structure:

```
TIME SEVERITY LOGGER [CODE] MESSAGE | KEY=VALUE ...
```

Parameters

- **utc** (*bool*) – If True, write the time in UTC with the suffix Z. If False, write the local time with its UTC offset.
- **show_fields** (*bool*) – If True, add the structured fields to the first line.
- **show_help** (*bool*) – If True, add the cause and the remedy for events that have the severity WARNING or higher.
- **show_source** (*bool*) – If True, add the source file and the line number.
- **show_traceback** (*bool*) – If True, add the traceback of a related exception.

format(*record*)

Return the text for a record of the logging module.

Parameters
record (*LogRecord*)

Return type
str

format_event(*event*)

Return the text for an event.

Parameters
event (Event) – The event to format.

Return type
str

class annunciator.formatters.ColourFormatter(*, *utc=True*, *show_fields=True*, *show_help=True*, *show_source=False*, *show_traceback=True*)

Bases: *PlainFormatter*

Formatter that writes readable text with ANSI colour codes.

The formatter adds colour only to the severity name. The parameters are the same as the parameters of *PlainFormatter*.

Parameters

- **utc** (*bool*)
- **show_fields** (*bool*)
- **show_help** (*bool*)
- **show_source** (*bool*)
- **show_traceback** (*bool*)

class annunciator.formatters.JsonFormatter(*, *include_source=True*)

Bases: Formatter

Formatter that writes one JSON object on one line.

The object has the structure that *annunciator.events.Event.to_dict()* returns. A file of these lines is a JSON Lines file. The report functions and the command-line interface can read such a file.

Parameters

include_source (*bool*) – If True, include the source position in each object.

format(*record*)

Return the JSON text for a record.

Parameters

record (*LogRecord*)

Return type

str

annunciator.formatters.supports_colour(*stream=None*)

Return True if a stream can show ANSI colour codes.

The function returns False if the environment variable NO_COLOR is set or if the stream is not a terminal. On Windows, the function also sets the console to the virtual terminal mode.

Parameters

stream (*Any*) – The stream to examine. The default is sys.stderr.

Return type

bool

6.6.11 Filters (annunciator.filters)

Filters.

A filter accepts, rejects or changes a log record before a handler writes it. Add a filter to a handler with handler.addFilter. The *configure()* function adds the filters that the configuration selects.

Examples

```
>>> import logging
>>> from annunciator.filters import RedactionFilter
>>> record = logging.LogRecord("demo", 20, __file__, 1, "login", (), None)
>>> record.ann_fields = {"user": "tom", "api_token": "abc123"}
>>> RedactionFilter().filter(record)
True
>>> record.ann_fields
{'user': 'tom', 'api_token': '[REDACTED]'}
```

annunciator.filters.**DEFAULT_REDACT_KEYS**: Tuple[str, ...] = ('password', 'passwd', 'secret', 'token', 'api_key', 'apikey', 'authorization', 'credential', 'private_key')

Parts of field names that identify secret data.

class annunciator.filters.**ContextFilter**(*name=''*)

Bases: `Filter`

Filter that adds the context data to each record.

The filter makes sure that records from standard loggers also have the context data. It accepts all records.

filter(*record*)

Add the context data and return `True`.

Parameters
record (*LogRecord*)

Return type
bool

class annunciator.filters.**SeverityRangeFilter**(*minimum=Severity.TRACE*, *maximum=Severity.CRITICAL*)

Bases: `Filter`

Filter that accepts records in a range of severities.

Parameters

- **minimum** (SeverityLike) – The lowest severity that the filter accepts.
- **maximum** (SeverityLike) – The highest severity that the filter accepts.

filter(*record*)

Return `True` if the severity is in the range.

Parameters
record (*LogRecord*)

Return type
bool

class annunciator.filters.**CodeFilter**(*include=()*, *exclude=()*, ***, *pass_uncoded=True*)

Bases: `Filter`

Filter that accepts or rejects records by their message code.

Parameters

- **include** (*Sequence[str]*) – Patterns of the codes to accept, for example `APP-E-*`. If the sequence is empty, the filter accepts all codes.
- **exclude** (*Sequence[str]*) – Patterns of the codes to reject.
- **pass_uncoded** (*bool*) – If `True`, accept records that have no message code.

Notes

The patterns use the syntax of the `fnmatch` module.

`filter`(*record*)

Return `True` if the filter accepts the code of the record.

Parameters
`record` (*LogRecord*)

Return type
bool

`class annunciator.filters.RedactionFilter`(*keys=('password', 'passwd', 'secret', 'token', 'api_key', 'apikey', 'authorization', 'credential', 'private_key')*, *patterns=()*, *mask='[REDACTED]'*)

Bases: `Filter`

Filter that removes secret data from records.

The filter replaces the value of each field that has a secret name. It also replaces each part of the message text that agrees with a pattern. The filter accepts all records.

Parameters

- `keys` (*Iterable[str]*) – Parts of field names. If a field name contains one of these parts, the filter replaces the value. Letter case is not important.
- `patterns` (*Iterable[Union[str, Pattern[str]]]*) – Regular expressions for secret text in messages and in text values.
- `mask` (*str*) – The text that replaces the secret data.

`filter`(*record*)

Remove the secret data and return `True`.

Parameters
`record` (*LogRecord*)

Return type
bool

`class annunciator.filters.DuplicateFilter`(*window=5.0*, ***, *max_entries=1024*, *exempt_from=None*, *clock=<built-in function monotonic>*)

Bases: `Filter`

Filter that rejects equal records in a time window.

Two records are equal if they have the same logger, severity, code and message text. When the filter accepts a record again after the window, it adds the field `repeats` with the number of rejected records.

Parameters

- `window` (*float*) – The length of the time window in seconds.
- `max_entries` (*int*) – The maximum number of different records that the filter keeps in memory.
- `exempt_from` (*Optional[SeverityLike]*) – The filter always accepts records of this severity or higher. Give `None` to apply the filter to all records.
- `clock` (*Callable[[], float]*) – A function that returns the time in seconds. The default is `time.monotonic()`.

`filter`(*record*)

Return `False` for an equal record in the time window.

Parameters
`record` (*LogRecord*)

Return type
bool

`class annunciator.filters.RateLimitFilter`(*rate=50.0*, *burst=100*, ***, *exempt_from=Severity.ERROR*, *clock=<built-in function monotonic>*)

Bases: `Filter`

Filter that limits the number of records in a unit of time.

The filter uses a token bucket. Each accepted record uses one token. The bucket gets `rate` new tokens each second, to a maximum of `burst` tokens.

Parameters

- `rate` (*float*) – The mean number of records that the filter accepts each second.
- `burst` (*int*) – The maximum number of records that the filter accepts at one time.
- `exempt_from` (*Optional[SeverityLike]*) – The filter always accepts records of this severity or higher.
- `clock` (*Callable[[], float]*) – A function that returns the time in seconds.

`dropped`

The number of records that the filter rejected.

Type
int

`filter`(*record*)

Return `False` if no token is available.

Parameters
`record` (*LogRecord*)

Return type
bool

`class annunciator.filters.LocalsFilter`(*depth=2*, ***, *max_length=200*, *max_items=40*)

Bases: `Filter`

Filter that adds the local variables of a failure to a record.

When a record has an exception, the filter reads the local variables of the last frames of the traceback. It keeps a short text for each value. The redaction filter then removes secret data. The filter accepts all records.

Parameters

- `depth` (*int*) – The number of frames to read. The last frame is the function that raised the exception.
- `max_length` (*int*) – The maximum number of characters for each value.
- `max_items` (*int*) – The maximum number of variables for each frame.

`filter`(*record*)

Add the local variables and return `True`.

Parameters
`record` (*LogRecord*)

Return type
bool

6.6.12 Handling tools (annunciator.handling)

Tools for error handling.

This module contains decorators and context managers that apply a standard policy to exceptions:

- *handle_errors()* and *guard* record an exception. Then they raise it again, convert it, or stop it.
- *retry()* does an operation again when it fails.
- *ErrorCollector* collects many errors and raises them as one group.
- *log_calls()* and *timed()* record calls and durations.
- *deprecated()* issues a warning for an old function.

All decorators accept normal functions and coroutine functions.

Examples

```
>>> from annunciator import handle_errors
>>> @handle_errors(ZeroDivisionError, reraise=False, default=0.0)
... def ratio(a, b):
...     return a / b
>>> ratio(1, 0)
0.0
```

annunciator.handling.**handle_errors**(**exceptions*, *logger=None*, *severity=None*, *operation=None*, *reraise=True*, *default=None*, *wrap=None*, *on_error=None*)

Decorator that applies an error policy to a function.

When the function raises one of the given exceptions, the decorator does these steps:

1. It records the exception with *report()*.
2. It calls `on_error`, if you gave it.
3. If you gave `wrap`, it raises the new exception from the initial exception.
4. If `reraise` is `True`, it raises the initial exception again.
5. If not, it returns `default`.

Parameters

- ***exceptions** (*Type[BaseException]*) – The exception classes to handle. The default is `Exception`.
- **logger** (StructuredLogger | *Logger* | *str* | *None*) – A structured logger, a standard logger or a logger name. The default is the logger of the module of the function.
- **severity** (Severity | *int* | *str* | *None*) – The severity of the record. The default is the severity of the exception, or `ERROR`.
- **operation** (*str* | *None*) – The name of the operation for the record. The default is the qualified name of the function.
- **reraise** (*bool*) – If `True`, raise the exception again after the record.
- **default** (*Any*) – The value to return when `reraise` is `False`.
- **wrap** (*Type[BaseException]* | *Callable[[BaseException], BaseException]* | *None*) – An exception class, or a function that makes an

exception from the initial exception. A *CodedError* subclass gets the fields `operation` and `reason`.

- **on_error** (*Callable[[BaseException], Any] | None*) – A function that the decorator calls with the exception.

Returns
callable – The decorator.

Return type
Callable[[*F*], *F*]

Examples

```
>>> from annunciator import handle_errors, DataError
>>> @handle_errors(KeyError, wrap=DataError)
... def channel(spectrum, index):
...     return spectrum[index]
>>> try:
...     channel({}, 7)
... except DataError as error:
...     type(error.__cause__).__name__
'KeyError'
```

class annunciator.handling.**guard**(**exceptions*, *operation='block'*, *logger=None*, *severity=None*, *reraise=True*, *wrap=None*, *on_error=None*)

Bases: `object`

Context manager that applies an error policy to a block.

The policy is the same as the policy of *handle_errors()*. A block cannot return a default value. Thus, when `reraise` is `False`, the program continues after the block.

Parameters

- ***exceptions** (*Type[BaseException]*) – The exception classes to handle. The default is `Exception`.
- **operation** (*str*) – The name of the operation for the record.
- **logger** (*LoggerLike*) – A structured logger, a standard logger or a logger name.
- **severity** (*Optional[SeverityLike]*) – The severity of the record.
- **reraise** (*bool*) – If `True`, raise the exception again after the record.
- **wrap** (*WrapLike*) – An exception class, or a function that makes an exception.
- **on_error** (*Optional[Callable[[BaseException], Any]]*) – A function that the context manager calls with the exception.

exception

The exception that the block raised.

Type
BaseException or None

Examples

```
>>> from annunciator import guard
>>> with guard(ValueError, operation="parse", reraise=False) as g:
...     value = int("12a")
>>> g.failed
True
```

(continues on next page)

(continued from previous page)

```
>>> type(g.exception).__name__
'ValueError'
```

property failed: bool

`True` if the block raised a handled exception.

annunciator.handling.**retry**(**exceptions*, *attempts=3*, *delay=1.0*, *backoff=2.0*, *max_delay=60.0*, *jitter=0.0*, *logger=None*, *operation=None*, *on_retry=None*, *reraise=False*, *sleep=None*)

Decorator that does a function again when it fails.

The delay before attempt `n + 1` is `min(delay * backoff ** (n - 1), max_delay)` seconds. The decorator records each failed attempt with the code `ANN-W-0301`. When all attempts fail, it records the code `ANN-E-0302`.

Parameters

- ***exceptions** (*Type[BaseException]*) – The exception classes that start a new attempt. The default is `Exception`.
- **attempts** (*int*) – The maximum number of attempts. The minimum is 1.
- **delay** (*float*) – The delay in seconds before the second attempt.
- **backoff** (*float*) – The factor that increases the delay after each attempt.
- **max_delay** (*float*) – The maximum delay in seconds.
- **jitter** (*float*) – A fraction from 0 to 1. The decorator adds a random time of up to this fraction of the delay.
- **logger** (StructuredLogger | *Logger* | *str* | *None*) – A structured logger, a standard logger or a logger name.
- **operation** (*str* | *None*) – The name of the operation for the records.
- **on_retry** (*Callable[[BaseException, int], Any]* | *None*) – A function that the decorator calls before each delay. It gets the exception and the attempt number.
- **reraise** (*bool*) – If `True`, raise the last exception when all attempts fail. If `False`, raise *RetryExhaustedError* from the last exception.
- **sleep** (*Callable[[float], Any]* | *None*) – A function that waits for the given number of seconds. Use this parameter in tests. For a coroutine function, the default is `asyncio.sleep()`.

Raises

ValueError – If a parameter is out of its range.

Return type

Callable[[*F*], *F*]

Examples

```
>>> from annunciator import retry
>>> calls = []
>>> @retry(OSError, attempts=3, delay=0.0)
... def read():
...     calls.append(1)
...     if len(calls) < 3:
...         raise OSError("busy")
...     return "data"
```

(continues on next page)

(continued from previous page)

```
>>> read()
'data'
>>> len(calls)
3
```

annunciator.handling.**log_calls**(*logger=None*, ***, *severity=Severity.DEBUG*, *arguments=False*, *result=False*)

Decorator that records the start and the end of each call.

The decorator records the start with the code `ANN-D-0306` and the end with the code `ANN-D-0307`. The end record contains the duration.

Parameters

- **logger** (StructuredLogger | *Logger* | *str* | *None*) – A structured logger, a standard logger or a logger name.
- **severity** (Severity | *int* | *str*) – The severity of the records.
- **arguments** (*bool*) – If `True`, record the arguments as the fields `args` and `kwargs`. Do not use this option for secret data.
- **result** (*bool*) – If `True`, record the return value as the field `result`.

Return type

Callable[[*F*], *F*]

class annunciator.handling.**timed**(*operation*, ***, *logger=None*, *severity=Severity.INFO*, *warn_after=None*, ***fields*)

Bases: `object`

Context manager that records the duration of a block.

At the end of the block, the context manager records the code `ANN-I-0308`. If the duration is more than `warn_after`, it records the code `ANN-W-0305`.

Parameters

- **operation** (*str*) – The name of the operation for the record.
- **logger** (*LoggerLike*) – A structured logger, a standard logger or a logger name.
- **severity** (SeverityLike) – The severity of the normal record.
- **warn_after** (*Optional[float]*) – A limit in seconds, or `None` for no limit.
- ****fields** (*Any*) – More structured fields for the record.

elapsed

The duration of the block in seconds.

Type

float

Examples

```
>>> from annunciator import timed
>>> with timed("fit") as timer:
...     total = sum(range(1000))
>>> timer.elapsed >= 0.0
True
```

class annunciator.handling.ErrorCollector(*operation='operation'*, *, *logger=None*, *record=True*)

Bases: object

Object that collects errors and warnings of one operation.

Use a collector when an operation must continue after an error, for example when a program examines many files. At the end, the collector can raise all errors as one *AggregateError*.

Parameters

- **operation** (*str*) – The name of the operation.
- **logger** (*LoggerLike*) – A structured logger, a standard logger or a logger name.
- **record** (*bool*) – If True, record each error and each warning in the log when the collector gets it.

errors

The collected errors.

Type
list of BaseException

warnings

The collected warning texts.

Type
list of str

Examples

```
>>> from annunciator import ErrorCollector, AggregateError
>>> collector = ErrorCollector("import", record=False)
>>> for text in ("1", "x", "3", "y"):
...     with collector.catch(ValueError):
...         number = int(text)
>>> len(collector.errors)
2
>>> try:
...     collector.raise_if_errors()
... except AggregateError as error:
...     error.message
"2 errors occurred in 'import'."
```

property ok: bool

True if the collector has no errors.

add(*error*, ***context*)

Add an error to the collector.

Parameters

- **error** (*BaseException | str*) – An exception or a message text. The collector changes a text into a *CodedError*.
- ****context** (*Any*) – Context data for the record.

Return type
None

add_warning(*message*, ***fields*)

Add a warning text to the collector.

Parameters

- **message** (*str*)
- **fields** (*Any*)

Return type
None

catch(**exceptions*, ***context*)

Return a context manager that collects exceptions of a block.

Parameters

- ***exceptions** (*Type[BaseException]*) – The exception classes to collect. The default is `Exception`.
- ****context** (*Any*) – Context data for the record.

Return type
_Catch

raise_if_errors()

Raise an *AggregateError* if errors exist.

Raises
AggregateError – If the collector has one or more errors (`ANN-E-0304`).

Return type
None

summary()

Return a text with the number of errors and warnings.

Return type
str

annunciator.handling.**deprecated**(*since*, ***, *alternative=None*, *removal=None*)

Decorator that issues a warning when an old function is called.

The warning has the class *DeprecatedFeatureWarning* and the code `ANN-W-0309`.

Parameters

- **since** (*str*) – The version that deprecated the function.
- **alternative** (*str | None*) – The name of the function to use as a replacement.
- **removal** (*str | None*) – The version that will remove the function.

Return type
Callable[[*F*], *F*]

Examples

```
>>> import warnings
>>> from annunciator import deprecated
>>> @deprecated("2.0", alternative="load_spectrum")
... def read_spectrum():
...     return []
>>> with warnings.catch_warnings(record=True) as caught:
...     warnings.simplefilter("always")
...     _ = read_spectrum()
>>> caught[0].message.message
"'read_spectrum' is deprecated since version 2.0. Use 'load_spectrum'."
```

6.6.13 Hooks (annunciator.hooks)

Exception hooks and warning capture.

The hooks record the problems that no code in the program handles:

- exceptions that stop the program,
- exceptions that stop a thread,
- exceptions that Python cannot raise,
- errors that an `asyncio` event loop reports,
- Python warnings.

The *configure()* function installs the hooks when the options `install_hooks` and `capture_warnings` are set.

Examples

```
>>> from annunciator import hooks
>>> hooks.install_hooks()
>>> hooks.hooks_installed()
True
>>> hooks.uninstall_hooks()
>>> hooks.hooks_installed()
False
```

annunciator.hooks.**install_hooks**(*logger=None*, *, *exceptions=True*, *threads=True*, *unraisable=True*, *warnings=True*)

Install the hooks that record unhandled problems.

Parameters

- **logger** (StructuredLogger / *None*) – The logger for the records. The default is the logger `annunciator.hooks`. Warnings always go to the logger `py.warnings`.
- **exceptions** (*bool*) – If `True`, replace `sys.excepthook`. The hook records each exception that stops the program (`ANN-C-0201`). For a coded exception, the hook records the code of the exception with the severity `CRITICAL`.
- **threads** (*bool*) – If `True`, replace `threading.excepthook` (`ANN-C-0202`).
- **unraisable** (*bool*) – If `True`, replace `sys.unraisablehook` (`ANN-E-0204`).
- **warnings** (*bool*) – If `True`, replace `warnings.showwarning`. The hook records a coded warning with its code. It records all other warnings with the code `ANN-W-0205`.

Return type
None

Notes

The hooks do not record `KeyboardInterrupt` and `SystemExit`. The initial hooks handle these exceptions. If you call the function again, the function first removes the hooks of the previous call.

annunciator.hooks.**uninstall_hooks**()

Remove the hooks and install the initial hooks again.

Return type
None

annunciator.hooks.**hooks_installed**()

Return `True` if one or more hooks of the library are installed.

Return type
bool

annunciator.hooks.**install_asyncio_handler**(*loop=None*, *logger=None*)

Record the errors that an `asyncio` event loop reports.

The function sets the exception handler of the loop. The handler records each error with the code `ANN-E-0203`.

Parameters

- **loop** (*AbstractEventLoop | None*) – The event loop. The default is the loop that is in operation. Call the function in a coroutine if you do not give a loop.
- **logger** (StructuredLogger | *None*) – The logger for the records.

Return type
None

6.6.14 Reports (annunciator.report)

Reports.

The functions of this module read, select, write and summarise events. The source of the events can be the event store or a JSON Lines log file.

Examples

```
>>> from annunciator import Event, Severity
>>> from annunciator.report import EventReport
>>> events = [
...     Event(severity=Severity.INFO, message="Start"),
...     Event(severity=Severity.ERROR, message="No beam", code="APP-E-0001"),
... ]
>>> report = EventReport.from_events(events)
>>> report.total, report.worst.name, report.exit_code()
(2, 'ERROR', 1)
```

class annunciator.report.**EventReport**(*total=0*, *by_severity=<factory>*, *by_code=<factory>*, *by_logger=<factory>*, *first=None*, *last=None*, *worst=None*, *by_fingerprint=<factory>*)

Bases: `object`

Summary of a set of events.

Parameters

- **total** (*int*)
- **by_severity** (*Dict[str, int]*)
- **by_code** (*Dict[str, int]*)
- **by_logger** (*Dict[str, int]*)
- **first** (*datetime | None*)
- **last** (*datetime | None*)
- **worst** (Severity | *None*)
- **by_fingerprint** (*Dict[str, Dict[str, Any]]*)

`total`
: The number of events.

  Type
  int

`by_severity`
: The number of events for each severity name.

  Type
  dict

`by_code`
: The number of events for each message code.

  Type
  dict

`by_logger`
: The number of events for each logger name.

  Type
  dict

`first`
: The time of the first event.

  Type
  datetime or None

`last`
: The time of the last event.

  Type
  datetime or None

`worst`
: The highest severity in the set.

  Type
  Severity or None

`by_fingerprint`
: The distinct failures. The keys are fingerprints. Each value is a dictionary with the keys `count`, `type`, `message`, `first` and `last`.

  Type
  dict

`classmethod from_events(`*events*`)`
: Make a report from events.

  Parameters
  events (*Iterable[*Event*]*)

  Return type
  EventReport

`count(`*minimum*`)`
: Return the number of events of a severity or higher.

  Parameters
  minimum (Severity | *int* | *str*)

  Return type
  int

property has_warnings: bool

`True` if the set contains `WARNING` or higher.

property has_errors: bool

`True` if the set contains `ERROR` or higher.

exit_code()

Return an exit status for a command-line program.

Returns
int – 0 if the set has no errors, 1 if the worst severity is `ERROR`, 2 if the worst severity is `CRITICAL`.

Return type
int

to_dict()

Return the report as a dictionary that JSON can contain.

Return type
Dict[str, *Any*]

to_text()

Return the report as readable text.

Return type
str

annunciator.report.**read_events**(*path*, *, *strict=True*)

Read the events of a JSON Lines log file.

Parameters

- **path** (*str | Path*) – The path of the file.
- **strict** (*bool*) – If `True`, raise an exception for an incorrect line. If `False`, ignore incorrect lines.

Yields
Event – The events of the file, in the sequence of the lines.

Raises
DataError – If `strict` is `True` and a line is not a correct event record (`ANN-E-0501`).

Return type
Iterator[Event]

annunciator.report.**write_events**(*events*, *path*)

Write events to a JSON Lines file.

Returns
int – The number of events that the function wrote.

Parameters

- **events** (*Iterable[Event]*)
- **path** (*str | Path*)

Return type
int

annunciator.report.**export_csv**(*events*, *path*)

Write events to a CSV file.

The file has one row for each event. The column `fields` contains the structured fields as JSON text.

Returns
int – The number of events that the function wrote.

Parameters

- **events** (*Iterable[Event]*)
- **path** (*str | Path*)

Return type
int

annunciator.report.**select_events**(*events*, ***, *minimum=None*, *maximum=None*, *code=None*, *logger=None*, *text=None*, *fingerprint=None*, *since=None*, *until=None*)

Select the events that agree with all given conditions.

Parameters

- **events** (*Iterable[Event]*) – The events to examine.
- **minimum** (Severity | *int* | *str* | *None*) – Keep only the events of this severity or higher.
- **maximum** (Severity | *int* | *str* | *None*) – Keep only the events of this severity or lower.
- **code** (*str | None*) – A pattern for the message code, for example `APP-E-*`.
- **logger** (*str | None*) – A pattern for the logger name.
- **text** (*str | None*) – Keep only the events that contain this text in the message. Letter case is not important.
- **fingerprint** (*str | None*) – Keep only the events with an exception that has this fingerprint.
- **since** (*datetime | None*) – Keep only the events at or after this time.
- **until** (*datetime | None*) – Keep only the events at or before this time.

Return type
Iterator[Event]

Notes

The patterns use the syntax of the `fnmatch` module.

annunciator.report.**summarise**(*source=None*)

Make a report from a log file, from events or from the event store.

Parameters
source (*str | Path | Iterable[Event] | None*) – The path of a JSON Lines log file, or events. `None` selects the event store of the configuration.

Returns
EventReport – The report. If the event store is not available, the report is empty.

Return type
EventReport

class annunciator.report.**RunComparison**(*new_codes=<factory>*, *gone_codes=<factory>*, *changed_codes=<factory>*, *severity_changes=<factory>*, *new_failures=<factory>*, *gone_failures=<factory>*)

Bases: `object`

The differences between the events of two runs.

Parameters

- **new_codes** (*Dict[str, int]*)
- **gone_codes** (*Dict[str, int]*)
- **changed_codes** (*Dict[str, tuple]*)
- **severity_changes** (*Dict[str, tuple]*)
- **new_failures** (*Dict[str, str]*)
- **gone_failures** (*Dict[str, str]*)

`new_codes`

The message codes that occur only in the second run, with their counts.

Type
dict

`gone_codes`

The message codes that occur only in the first run, with their counts.

Type
dict

`changed_codes`

The message codes that have different counts. Each value is a tuple of the first count and the second count.

Type
dict

`severity_changes`

The severities that have different counts. Each value is a tuple of the first count and the second count.

Type
dict

`new_failures`

The failures that occur only in the second run. The keys are fingerprints. The values are texts.

Type
dict

`gone_failures`

The failures that occur only in the first run.

Type
dict

`property regression: bool`

`True` if the second run has new problems.

A new problem is a new failure or a new message code of the severity `WARNING` or higher.

`property identical: bool`

`True` if the comparison found no differences.

`to_dict()`

Return the comparison as a dictionary that JSON can contain.

Return type
Dict[str, *Any*]

`to_text()`

Return the comparison as readable text.

Return type
str

`annunciator.report.compare_runs(`*first*, *second*`)`

Compare the events of two runs.

Use this function to find what changed between a good run and a bad run, or between two versions of a program.

Parameters

- **first** (*str | Path | Iterable[Event] | EventReport*) – The reference run: the path of a JSON Lines log file, events or a report.
- **second** (*str | Path | Iterable[Event] | EventReport*) – The run to examine, in one of the same forms.

Returns
RunComparison – The differences.

Return type
RunComparison

6.6.15 Flight recorder (`annunciator.recorder`)

Flight recorder.

A flight recorder keeps the last detailed records in memory. It writes them only when an event of a trigger severity occurs. Thus the log of a normal run stays short. The log of a failed run contains the detailed records that came before the failure.

The option `flight_recorder` of the configuration installs a flight recorder on the log file. If the configuration has no log file, the flight recorder is on the console.

Examples

```
>>> import logging
>>> from annunciator.handlers import EventStoreHandler
>>> from annunciator.recorder import FlightRecorderHandler
>>> target = EventStoreHandler()
>>> target.setLevel(logging.WARNING)
>>> recorder = FlightRecorderHandler(target, capacity=50)
>>> logger = logging.getLogger("doctest.recorder")
>>> logger.propagate = False
>>> logger.setLevel(logging.DEBUG)
>>> logger.addHandler(recorder)
>>> logger.debug("step 1")
>>> logger.debug("step 2")
>>> len(target.events)
0
>>> logger.error("failure")
>>> [event.message for event in target.events][1:]
['step 1', 'step 2', 'failure']
```

`class annunciator.recorder.FlightRecorderHandler(`*target*, *capacity=200*, *, *trigger=Severity.ERROR*, *capture_level=Severity.DEBUG*`)`

Bases: `Handler`

Handler that keeps detailed records until a failure occurs.

The handler sends each record of the target severity directly to the target handler. It keeps all lower records in memory. When a record of the trigger severity or higher comes, the handler first sends the kept records to the target. Each of these records has the field `flight_recorder` with the value `True`.

Parameters

- **target** (*logging.Handler*) – The handler that writes the records. Its level is the target severity.
- **capacity** (*int*) – The maximum number of records in memory. When the memory is full, the handler removes the oldest record.
- **trigger** (SeverityLike) – The severity that causes the handler to write the kept records.
- **capture_level** (SeverityLike) – The minimum severity of the records that the handler keeps.

flushes

The number of times that the handler wrote the kept records.

Type
int

emit(*record*)

Keep the record or send it to the target handler.

Parameters
record (*LogRecord*)

Return type
None

discard()

Remove the kept records without writing them.

Returns
int – The number of records that the method removed.

Return type
int

flush()

Flush the target handler. The kept records stay in memory.

Return type
None

close()

Close the target handler and remove the kept records.

Return type
None

6.6.16 Progress and watchdog (annunciator.watch)

Progress records and stall detection.

Long operations must show that they are in operation. This module has two tools:

- *Progress* records the state of an operation at a controlled interval, with the rate and the time to completion.
- *Watchdog* records a warning when an operation does not report progress in a permitted time.

Examples

```
>>> from annunciator.watch import track
>>> total = 0
>>> for value in track(range(5), "sum", interval=3600):
...     total += value
>>> total
10
```

class annunciator.watch.**Progress**(*operation*, *total=None*, ***, *interval=10.0*, *logger=None*, *severity=Severity.INFO*, *stall_after=None*, ***fields*)

Bases: `object`

Object that records the progress of a long operation.

The object records the code `ANN-I-0330` a maximum of one time in each interval. At the end, it records the code `ANN-I-0331`.

Parameters

- **operation** (*str*) – The name of the operation for the records.
- **total** (*Optional[int]*) – The number of items, or `None` if the number is not known.
- **interval** (*float*) – The minimum time in seconds between two progress records.
- **logger** (*LoggerLike*) – A structured logger, a standard logger or a logger name.
- **severity** (SeverityLike) – The severity of the progress records.
- **stall_after** (*Optional[float]*) – A time in seconds. If you give a value, the object starts a *Watchdog* with this limit.
- ****fields** (*Any*) – More structured fields for the records.

done

The number of completed items.

Type
int

Examples

```
>>> from annunciator.watch import Progress
>>> with Progress("fit spectra", total=3, interval=3600) as progress:
...     for _ in range(3):
...         progress.update()
>>> progress.done
3
```

property elapsed: float

The time in seconds from the start of the operation.

update(*count=1*)

Add completed items and record the progress if the interval is over.

Parameters
count (*int*) – The number of items that the operation completed.

Return type
None

close()

Record the end of the operation and stop the watchdog.

Return type
None

class annunciator.watch.Watchdog(*timeout*, *operation='operation'*, ***, *logger=None*, *on_stall=None*, *repeat=True*)

Bases: object

Object that finds an operation that does not make progress.

A thread in the background examines the time of the last call of *beat()*. If the time is more than the limit, the watchdog records the code ANN-W-0332. When the operation continues, the watchdog records the code ANN-I-0333.

Parameters

- **timeout** (*float*) – The permitted time in seconds between two calls of *beat()*.
- **operation** (*str*) – The name of the operation for the records.
- **logger** (*LoggerLike*) – A structured logger, a standard logger or a logger name.
- **on_stall** (*Optional[Callable[[float], Any]]*) – A function that the watchdog calls when it finds a stall. The function gets the idle time in seconds.
- **repeat** (*bool*) – If True, record the warning again after each subsequent timeout interval of the same stall.

stalls

The number of stalls that the watchdog found.

Type
int

Examples

```
>>> from annunciator.watch import Watchdog
>>> with Watchdog(60.0, "acquisition") as watchdog:
...     watchdog.beat()
>>> watchdog.stalls
0
```

start()

Start the thread of the watchdog.

Return type
Watchdog

stop()

Stop the thread of the watchdog.

Return type
None

beat()

Tell the watchdog that the operation made progress.

Return type
None

annunciator.watch.track(*iterable*, *operation*, ***, *total=None*, *interval=10.0*, *logger=None*, *stall_after=None*, ***fields*)

Give the items of an iterable and record the progress.

Parameters

- **iterable** (*Iterable[T]*) – The items of the operation.
- **operation** (*str*) – The name of the operation for the records.
- **total** (*int | None*) – The number of items. The default is `len(iterable)`, if the length is available.
- **interval** (*float*) – The minimum time in seconds between two progress records.
- **logger** (StructuredLogger | *Logger | str | None*) – A structured logger, a standard logger or a logger name.
- **stall_after** (*float | None*) – A time in seconds for the stall detection, or `None`.
- ****fields** (*Any*) – More structured fields for the records.

Yields

object – The items of the iterable.

Return type

Iterator[*T*]

6.6.17 Runs (`annunciator.provenance`)

Run records and run manifests.

A run is one complete operation of a program, for example one measurement or one simulation. This module records the data that are necessary to do the same run again:

- the versions of Python and of the important packages,
- the revision of the source code,
- the computer, the command line and the configuration,
- the seed of the random number generator,
- the start time, the duration and the result.

Examples

```
>>> from annunciator.provenance import collect_manifest
>>> manifest = collect_manifest(seed=42, git=False)
>>> manifest["seed"], sorted(manifest)[:3]
(42, ['annunciator', 'argv', 'cwd'])
```

annunciator.provenance.**collect_manifest**(*, *packages=()*, *seed=None*, *git=True*, *include_user=False*, *extra=None*)

Collect the data that identify the environment of a run.

Parameters

- **packages** (*Iterable[str]*) – The names of the installed packages whose versions you must record, for example `("numpy", "scipy")`.
- **seed** (*Any*) – The seed of the random number generator. The function only records the value. It does not set the seed.
- **git** (*bool*) – If `True`, record the git revision of the current directory.
- **include_user** (*bool*) – If `True`, record the login name of the user. The default is `False` because the name is personal data.
- **extra** (*Dict[str, Any] | None*) – More items for the manifest, for example the parameters of the run.

Returns
dict – The manifest. JSON can contain all its values.

Return type
Dict[str, *Any*]

`annunciator.provenance.git_revision(path=None)`

Return the revision of the git repository that contains a path.

Parameters
path (*str | Path | None*) – A directory in the repository. The default is the current directory.

Returns
dict or None – The keys `commit` and `dirty`. `dirty` is `True` if the repository has changes that are not committed. The function returns `None` if git is not available or the path is not in a repository.

Return type
Dict[str, *Any*] | None

`class annunciator.provenance.Run(name, manifest, logger=None)`

Bases: `object`

One run of a program.

Do not make an instance directly. Use *start_run()*. A run is also a context manager.

Parameters
- **name** (*str*)
- **manifest** (*Dict[str, Any]*)
- **logger** (*LoggerLike*)

name
The name of the run.

Type
str

run_id
A unique identifier. All events of the run have the context field `run_id`.

Type
str

manifest
The manifest of the run.

Type
dict

status
`"running"`, `"completed"` or `"failed"`.

Type
str

property errors: int
The number of events of the severity `ERROR` or higher.

property warnings: int
The number of events of the severity `WARNING`.

property exit_code: int

0, 1 or 2.

The value is 0 if the run has no errors, 1 if the worst severity is ERROR and 2 if it is CRITICAL.

Type
An exit status for the program

save_manifest(*path*)

Write the manifest and the result of the run to a JSON file.

Returns
pathlib.Path – The path of the file.

Parameters
path (*str | Path*)

Return type
Path

finish(*exc=None*)

Record the end of the run.

The method records the code ANN-N-0402 for a normal end, or the code ANN-E-0403 for an end because of an exception. It also records the summary of rejected repeated messages.

Parameters
exc (*BaseException | None*) – The exception that stopped the run, or None.

Return type
None

annunciator.provenance.start_run(*name=None*, ***, *packages=()*, *seed=None*, *git=True*, *include_user=False*, *logger=None*, ***extra*)

Start a run and record its manifest.

The function records the code ANN-N-0401. The field manifest of the event contains the manifest. The function also sets the state of the log methods once, every and throttle to its initial condition.

Parameters

- **name** (*str | None*) – The name of the run. The default is the name of the program file.
- **packages** (*Iterable[str]*) – The names of the packages whose versions you must record.
- **seed** (*Any*) – The seed of the random number generator.
- **git** (*bool*) – If True, record the git revision.
- **include_user** (*bool*) – If True, record the login name of the user.
- **logger** (StructuredLogger | *Logger | str | None*) – A structured logger, a standard logger or a logger name.
- ****extra** (*Any*) – More items for the manifest, for example the parameters of the run.

Returns
Run – The run. Use it as a context manager, or call *Run.finish()* at the end.

Return type
Run

Examples

```
>>> from annunciator.provenance import start_run
>>> with start_run("demo", git=False, live_time=600) as run:
...     pass
>>> run.status, run.exit_code
('completed', 0)
```

6.6.18 Integrity (annunciator.integrity)

Tamper-evident log files.

This module makes a JSON Lines log file with a chain. Each record contains a hash of the record before it. The records thus make a chain. If a person changes, removes or adds a record, the chain is broken and *verify_chain()* finds the position.

Each record has the object `chain` with three keys:

seq
: The sequence number of the record. The first record has the number 1.

prev
: The hash of the record before this record.

hash
: The hash of `prev` and of the contents of this record.

Without a key, the hash function is SHA-256. With a key, the function is HMAC-SHA-256. A person who does not have the key cannot make a correct chain again after a change.

Examples

```
>>> import logging, os, tempfile
>>> from annunciator.integrity import ChainFormatter, verify_chain
>>> path = os.path.join(tempfile.mkdtemp(), "run.jsonl")
>>> handler = logging.FileHandler(path, encoding="utf-8")
>>> handler.setFormatter(ChainFormatter())
>>> logger = logging.getLogger("doctest.integrity")
>>> logger.propagate = False
>>> logger.addHandler(handler)
>>> logger.warning("first")
>>> logger.warning("second")
>>> handler.close()
>>> result = verify_chain(path)
>>> result.ok, result.records
(True, 2)
```

class annunciator.integrity.**ChainFormatter**(*resume_from=None*, ***, *key=None*, *include_source=True*)

Bases: *JsonFormatter*

JSON formatter that adds the chain data to each record.

Parameters

- **resume_from** (*Union[str, Path, None]*) – The path of the log file. If the file exists, the chain continues from its last record.
- **key** (*Optional[bytes]*) – The key for HMAC-SHA-256, or `None` for SHA-256.
- **include_source** (*bool*) – If `True`, include the source position in each object.

Notes

Use one formatter for one log file. When the handler rotates the file, the chain continues in the new file. The first record of the new file then refers to the last record of the old file.

`format`(*record*)

Return the JSON text for a record, with the chain data.

Parameters
record (*LogRecord*)

Return type
str

`class annunciator.integrity.ChainResult`(*path*, *records=0*, *first_seq=0*, *last_seq=0*, *problems=<factory>*)

Bases: `object`

The result of an integrity check.

Parameters

- **path** (*str*)
- **records** (*int*)
- **first_seq** (*int*)
- **last_seq** (*int*)
- **problems** (*List[Tuple[int, str]]*)

`path`

The path of the file.

Type
str

`records`

The number of records that have correct chain data.

Type
int

`first_seq`

The sequence number of the first record in the file. A value of more than 1 shows that the chain started in an older file.

Type
int

`last_seq`

The sequence number of the last correct record.

Type
int

`problems`

The line number and the reason for each problem.

Type
list of tuple

`property ok: bool`

`True` if the file has chain data and no problems.

`to_text()`

Return the result as readable text.

Return type
str

`annunciator.integrity.verify_chain(`*path*, *, *key=None*, *expected_start=None*`)`

Examine the chain of a JSON Lines log file.

Parameters

- **path** (*str | Path*) – The path of the log file.
- **key** (*bytes | None*) – The key that the program used when it wrote the file, or `None`.
- **expected_start** (*str | None*) – The hash that the first record must refer to. Use the last hash of the older file to examine a file after a rotation. `None` accepts all values for a file that does not start at the sequence number 1.

Returns
ChainResult – The result of the check.

Return type
ChainResult

Notes

The check finds a changed record, a removed record, an added record and a changed sequence. It cannot find the removal of the last records of a file. To find that, keep the last hash in a different location and compare it.

`annunciator.integrity.chain_key_from_env(`*variable='ANNUNCIATOR_CHAIN_KEY'*`)`

Return the key of the chain from an environment variable.

Returns
bytes or None – The key, or `None` if the variable is not set or is empty.

Parameters
variable (*str*)

Return type
bytes | None

`annunciator.integrity.GENESIS = '0000000000000000000000000000000000000000000000000000000000000000'`

The value of `prev` for the first record of a chain.

`annunciator.integrity.KEY_VARIABLE = 'ANNUNCIATOR_CHAIN_KEY'`

The environment variable that contains the key of the chain.

6.6.19 Multiprocess tools (`annunciator.multiprocess`)

Logs from many processes.

When many processes write to the same log file, the file can become damaged. The rotation of the file is not safe in this condition. This module sends the records of all worker processes through a queue to the primary process. Only the primary process writes the file.

The procedure has three steps:

1. In the primary process, call *configure()*. Then call *start_listener()* to get the queue.
2. Start each worker process with *init_worker()* as its initializer and the queue as its argument.
3. At the end, call *stop_listener()*.

Examples

```
import multiprocessing

from annunciator import configure, get_logger
from annunciator.multiprocess import init_worker, start_listener, stop_listener

def work(number):
    get_logger("worker").info("Item complete.", item=number)
    return number * number

if __name__ == "__main__":
    configure(file="logs/run.jsonl", file_format="json")
    queue = start_listener()
    with multiprocessing.Pool(4, init_worker, (queue,)) as pool:
        results = pool.map(work, range(100))
    stop_listener()
```

class annunciator.multiprocess.**ProcessQueueHandler**(*queue*)

Bases: *AsyncQueueHandler*

Queue handler for a worker process.

The handler changes the structured data of each record into simple types. Thus the queue can send the record to a different process.

prepare(*record*)

Make a record that a different process can read.

Parameters
record (*LogRecord*)

Return type
LogRecord

annunciator.multiprocess.**start_listener**(*queue=None*)

Start the thread that writes the records of the worker processes.

Call this function in the primary process, after *configure()*.

Parameters
queue (*Any*) – A queue of the `multiprocessing` module. If you give `None`, the function makes a queue.

Returns
multiprocessing.Queue – The queue. Give it to *init_worker()* in each worker process.

Return type
Any

annunciator.multiprocess.**stop_listener**()

Write the records that are in the queue and stop the thread.

Call this function after all worker processes stopped. It is safe to call the function when no listener is in operation.

Return type
None

annunciator.multiprocess.**init_worker**(*queue*, *level=Severity.TRACE*)

Configure a worker process to send its records to the queue.

Use this function as the initializer of a process pool or call it at the start of the function that the process runs. The function removes all handlers of the root logger in the worker. Then it installs one *ProcessQueueHandler*.

Parameters

- **queue** (*Any*) – The queue from *start_listener()*.
- **level** (*Severity* / *int* / *str*) – The minimum severity that the worker sends. The handlers of the primary process apply their own levels.

Return type
None

6.6.20 Soft checks (annunciator.checks)

Soft checks.

A check examines a condition on data. If the condition is not true, the check does one of three actions. The parameter `on_fail` selects the action:

`"raise"`
Raise a *ValidationError* (default).

`"warn"`
Issue a *DataQualityWarning*. The program continues.

`"log"`
Record a `WARNING` event. The program continues.

Each check returns `True` if the condition is true. Thus you can use a check in an `if` statement.

Examples

```
>>> from annunciator.checks import check_range, check_finite
>>> check_range(0.31, 0.0, 1.0, name="dead_time")
True
>>> check_finite([1.0, float("nan")], name="counts", on_fail="log")
False
>>> from annunciator import ValidationError
>>> try:
...     check_range(1.4, 0.0, 1.0, name="dead_time")
... except ValidationError as error:
...     error.code
'ANN-E-0601'
```

annunciator.checks.**check**(*condition*, *message=None*, ***, *name='condition'*, *code=None*, *on_fail='raise'*, *error=<class 'annunciator.errors.ValidationError'>*, *category=<class 'annunciator.warns.DataQualityWarning'>*, *logger=None*, ***fields*)

Examine a condition.

Parameters

- **condition** (*Any*) – The value to examine. The check passes if the value is true.
- **message** (*str* / *None*) – The message text for a failure. The default is the text of the code.
- **name** (*str*) – A name for the check, for the default message.
- **code** (*str* / *None*) – Your own message code for a failure. The default is `ANN-E-0606` or `ANN-W-0616`.
- **on_fail** (*str*) – `"raise"`, `"warn"` or `"log"`.

- **error** (*Type [CodedError]*) – The exception class for "raise".
- **category** (*Type [CodedWarning]*) – The warning class for "warn".
- **logger** (StructuredLogger / *Logger* / *str* / *None*) – The logger for "log".
- ****fields** (*Any*) – Structured fields for the failure, for example the values that the condition used.

Returns
bool – True if the check passed.

Raises
ValidationError – If the check failed and on_fail is "raise".

Return type
bool

annunciator.checks.**check_range**(*value*, *minimum=None*, *maximum=None*, ***, *name='value'*, *inclusive=True*, *on_fail='raise'*, *code=None*, *logger=None*)

Examine if a number is in a range.

A value that is not a number (NaN) is always out of the range.

Parameters

- **value** (*Any*) – The number to examine.
- **minimum** (*Any*) – The lower limit, or None for no lower limit.
- **maximum** (*Any*) – The upper limit, or None for no upper limit.
- **name** (*str*) – The name of the value for the message.
- **inclusive** (*bool*) – If True, the limits are permitted values.
- **on_fail** (*str*) – "raise", "warn" or "log".
- **code** (*str* / *None*) – Your own message code for a failure. The default is ANN-E-0601 or ANN-W-0611.
- **logger** (StructuredLogger / *Logger* / *str* / *None*) – The logger for "log".

Returns
bool – True if the value is in the range.

Return type
bool

annunciator.checks.**check_finite**(*value*, ***, *name='value'*, *on_fail='raise'*, *code=None*, *logger=None*)

Examine if all numbers are finite.

The check fails if the data contain NaN or infinite values. It accepts one number, a sequence, a sequence of sequences or a NumPy array.

Parameters

- **value** (*Any*) – The data to examine.
- **name** (*str*) – The name of the data for the message.
- **on_fail** (*str*) – "raise", "warn" or "log".
- **code** (*str* / *None*) – Your own message code for a failure. The default is ANN-E-0602 or ANN-W-0612.
- **logger** (StructuredLogger / *Logger* / *str* / *None*) – The logger for "log".

Returns
bool – `True` if all numbers are finite.

Return type
bool

`annunciator.checks.check_type`(*value*, *expected*, *, *name='value'*, *on_fail='raise'*, *code=None*, *logger=None*)

Examine if a value has a permitted type.

Parameters

- **value** (*Any*) – The value to examine.
- **expected** (*type | Tuple[type, ...]*) – A type or a tuple of types.
- **name** (*str*) – The name of the value for the message.
- **on_fail** (*str*) – `"raise"`, `"warn"` or `"log"`.
- **code** (*str | None*) – Your own message code for a failure. The default is `ANN-E-0603` or `ANN-W-0613`.
- **logger** (StructuredLogger | *Logger | str | None*) – The logger for `"log"`.

Returns
bool – `True` if the value has a permitted type.

Return type
bool

`annunciator.checks.check_not_empty`(*value*, *, *name='value'*, *on_fail='raise'*, *code=None*, *logger=None*)

Examine if a sequence, a text or a data set contains items.

Parameters

- **value** (*Any*) – The object to examine. `None` is empty.
- **name** (*str*) – The name of the object for the message.
- **on_fail** (*str*) – `"raise"`, `"warn"` or `"log"`.
- **code** (*str | None*) – Your own message code for a failure. The default is `ANN-E-0604` or `ANN-W-0614`.
- **logger** (StructuredLogger | *Logger | str | None*) – The logger for `"log"`.

Returns
bool – `True` if the object contains one or more items.

Return type
bool

`annunciator.checks.check_one_of`(*value*, *choices*, *, *name='value'*, *on_fail='raise'*, *code=None*, *logger=None*)

Examine if a value is one of the permitted values.

Parameters

- **value** (*Any*) – The value to examine.
- **choices** (*Collection[Any]*) – The permitted values.
- **name** (*str*) – The name of the value for the message.
- **on_fail** (*str*) – `"raise"`, `"warn"` or `"log"`.

- **code** (*str* / *None*) – Your own message code for a failure. The default is `ANN-E-0605` or `ANN-W-0615`.
- **logger** (StructuredLogger / *Logger* / *str* / *None*) – The logger for `"log"`.

Returns
bool – `True` if the value is one of the permitted values.

Return type
bool

6.6.21 Lint (annunciator.lint)

Static check of message codes in source code.

The check reads Python source files without running them. It compares the message codes in the files with the registered catalogues. It finds three types of problem:

undefined
The source code uses a code that has no definition. At run time, the message has no text, no cause and no remedy.

missing-field
A call gives a code but does not give a value for a replacement field of the message text.

unused
A catalogue defines a code that the source code does not use.

Import the modules that define your catalogues before you call *lint_paths()*. The command `annunciator lint` does this with the option `--import`.

Examples

```
>>> import os, tempfile
>>> from annunciator.lint import lint_paths
>>> folder = tempfile.mkdtemp()
>>> with open(os.path.join(folder, "demo.py"), "w") as stream:
...     _ = stream.write('log.event("ANN-E-9999")\nlog.event("ANN-E-0001")\n')
>>> [(f.kind, f.code, f.line) for f in lint_paths([folder], unused=False)]
[('undefined', 'ANN-E-9999', 1), ('missing-field', 'ANN-E-0001', 2)]
```

class annunciator.lint.**LintFinding**(*path*, *line*, *kind*, *code*, *detail=''*)

Bases: `object`

One finding of the static check.

Parameters

- **path** (*str*)
- **line** (*int*)
- **kind** (*str*)
- **code** (*str*)
- **detail** (*str*)

path
The path of the source file, or the empty text for `unused`.

Type
str

`line`

The line number, or 0 for `unused`.

Type
int

`kind`

`"undefined"`, `"missing-field"`, `"unused"` or `"syntax"`.

Type
str

`code`

The message code.

Type
str

`detail`

More data about the finding.

Type
str

`property is_error: bool`

`True` for all kinds but `unused`.

`to_text()`

Return the finding as one line of text.

Return type
str

`annunciator.lint.lint_paths`(*paths*, *, *prefixes=None*, *unused=True*)

Examine the message codes in Python source files.

Parameters

- **paths** (*Iterable[str | Path]*) – Python files or directories. The function reads all `.py` files in a directory and in its subdirectories.
- **prefixes** (*Sequence[str] | None*) – The catalogue prefixes to examine. The default is all registered catalogues. The check for `unused` codes does not include the catalogue `ANN` unless you give it here.
- **unused** (*bool*) – If `True`, also find the codes that the source does not use.

Returns
list of LintFinding – The findings, in the sequence of the files and the lines.

Return type
List[LintFinding]

Notes

The check finds only codes that are complete text in the source code. It cannot find a code that the program makes at run time.

6.6.22 Test tools (`annunciator.testing`)

Tools for tests.

Use this module to examine the events that your code records.

- *capture_events()* is a context manager for all test tools.

- `annunciator_events` is a fixture for pytest. The library registers it automatically when pytest is installed.

Examples

```
>>> from annunciator import get_logger
>>> from annunciator.testing import capture_events
>>> with capture_events() as captured:
...     get_logger("demo").warning("Low count rate", rate=1.5)
>>> event = captured.assert_logged(severity="WARNING", text="count rate")
>>> event.fields["rate"]
1.5
>>> captured.assert_not_logged(minimum="ERROR")
```

class annunciator.testing.**EventCapture**(*level=Severity.TRACE*)

Bases: `Handler`

Handler that collects events for a test.

Parameters
level (SeverityLike)

events

The collected events, oldest first.

Type
list of *Event*

emit(*record*)

Add the event of a record.

Parameters
record (*LogRecord*)

Return type
None

property codes: List[str]

The message codes of the events that have a code.

property messages: List[str]

The message texts of the events.

clear()

Remove all collected events.

Return type
None

select(*code=None*, ***, *severity=None*, *minimum=None*, *text=None*, *logger=None*, ***fields*)

Return the events that agree with all given conditions.

Parameters

- **code** (*str | None*) – The message code.
- **severity** (Severity | *int* | *str* | *None*) – The severity.
- **minimum** (Severity | *int* | *str* | *None*) – The lowest severity.
- **text** (*str | None*) – A text that the message must contain.
- **logger** (*str | None*) – The name of the logger.
- ****fields** (*Any*) – Structured fields and their values.

Return type
List[Event]

`assert_logged`(*code=None*, *, *count=None*, ***conditions*)

Make sure that an event was recorded.

Parameters

- **code** (*str | None*) – The message code.
- **count** (*int | None*) – The necessary number of events. `None` accepts one or more events.
- ****conditions** (*Any*) – The other conditions of *select()*.

Returns
Event – The first event that agrees with the conditions.

Raises
AssertionError – If the number of events is not correct.

Return type
Event

`assert_not_logged`(*code=None*, ***conditions*)

Make sure that no event agrees with the conditions.

Raises
AssertionError – If one or more events agree with the conditions.

Parameters

- **code** (*str | None*)
- **conditions** (*Any*)

Return type
None

`assert_no_problems`(*minimum=Severity.WARNING*)

Make sure that no event has the given severity or higher.

Raises
AssertionError – If one or more events have the severity or higher.

Parameters
minimum (Severity | *int* | *str*)

Return type
None

`annunciator.testing.capture_events`(*logger=''*, *level=Severity.TRACE*)

Collect the events of a block of code.

Parameters

- **logger** (*str*) – The name of the logger to examine. The empty text selects the root logger and thus all loggers.
- **level** (Severity | *int* | *str*) – The minimum severity to collect. The function decreases the level of the logger for the duration of the block, if this is necessary.

Yields
EventCapture – The object that contains the events.

Return type
Iterator[EventCapture]

6.6.23 HTML report (annunciator.htmlreport)

HTML report of a run.

The function *write_html_report()* writes one HTML file that contains the events of a run. The file has no external parts. You can open it in a browser without a network connection and send it to other persons.

The page has the style of a printed industrial report: a title block, numbered sections and ruled tables in a fixed-width font, without colour. It shows the number of events for each severity and for each message code, the distinct failures and a table of the events. The reader can select events by severity, by message code and by text. A click on an event shows its fields, cause, remedy and traceback.

Examples

```
>>> import os, tempfile
>>> from annunciator import Event, Severity
>>> from annunciator.htmlreport import write_html_report
>>> events = [Event(severity=Severity.ERROR, message="No beam", code="APP-E-0001")]
>>> path = write_html_report(events, os.path.join(tempfile.mkdtemp(), "run.html"))
>>> path.read_text(encoding="utf-8").startswith("<!DOCTYPE html>")
True
```

annunciator.htmlreport.**write_html_report**(*events*, *path*, ***, *title='Run report'*, *max_events=20000*)

Write the events of a run to one HTML file.

Parameters

- **events** (*Iterable[Event]*) – The events of the run.
- **path** (*str | Path*) – The path of the HTML file.
- **title** (*str*) – The title of the page.
- **max_events** (*int*) – The maximum number of events in the file. If the run has more events, the file contains the last events. The counts at the top of the page always include all events.

Returns

pathlib.Path – The path of the file.

Return type

Path

6.6.24 Command-line interface (annunciator.cli)

Command-line interface.

Start the interface with the command `annunciator` or with `python -m annunciator`. The user manual gives the full data for each command.

The exit status is 0 when a command completes. The user manual gives the other values.

annunciator.cli.**main**(*argv=None*)

Do one command of the command-line interface.

Parameters

argv (*Sequence[str] | None*) – The arguments. The default is `sys.argv[1:]`.

Returns

int – The exit status.

Return type

int

annunciator.cli.build_parser()

Make the argument parser of the command-line interface.

Return type
ArgumentParser

6.6.25 Sphinx extension (annunciator.sphinxext)

Sphinx extension for message catalogues.

The extension adds the directive `annunciator-catalogue` and the role `msg`. The directive writes the reference text for all messages of a catalogue. Thus the documentation and the software always agree.

To use the extension, add `"annunciator.sphinxext"` to the list `extensions` in the file `conf.py`. Then write, for example:

```
.. annunciator-catalogue:: APP
   :module: myapp.messages
   :min-severity: WARNING

Refer to :msg:`APP-E-0001`.
```

The directive has these options:

module
A module to import before the directive reads the catalogue. Use this option for the module that defines your catalogue.

min-severity
Write only the messages of this severity or higher.

category
Write only the messages in this category.

heading
The character for the section underline. The default is `~`.

class annunciator.sphinxext.CatalogueDirective(*name*, *arguments*, *options*, *content*, *lineno*, *content_offset*, *block_text*, *state*, *state_machine*)

Bases: `Directive`

Directive that writes the messages of one catalogue.

required_arguments = 1
Number of required directive arguments.

optional_arguments = 0
Number of optional arguments after the required arguments.

has_content = False
May the directive have content?

option_spec = {'category': <function unchanged>, 'heading': <function single_char_or_unicode>, 'min-severity': <function unchanged>, 'module': <function unchanged>}
Mapping of option names to validator functions.

annunciator.sphinxext.msg_role(*name*, *rawtext*, *text*, *lineno*, *inliner*, *options=None*, *content=None*)

Role that makes a reference to the section of a message code.

annunciator.sphinxext.**setup**(*app*)

Register the directive and the role with Sphinx.

Parameters
app (*Any*)

Return type
Dict[str, *Any*]

annunciator.sphinxext.**target_name**(*code*)

Return the reference label for a message code.

Parameters
code (*str*)

Return type
str

CHAPTER

SEVEN

MESSAGES AND PROBLEM RESOLUTION

This chapter helps you to correct problems. It has three parts:

- *how to find the remedy for a message*,
- *the reference of all messages of the library*,
- *the problems that do not show a message code*.

7.1 Find the remedy for a message

Purpose

To find the probable cause and the remedy of a message that a program shows.

Procedure

1. Find the message code in the message. The code is in square brackets, for example `[ANN-E-0003]`.
2. Read the lines `Cause:` and `Remedy:` below the message, if the program shows them.
3. If the program does not show these lines, do this command:

   ```
   annunciator explain ANN-E-0003
   ```

4. If the prefix is `ANN`, you can also find the code in *the message reference* or in the index.
5. If the prefix is different, the message comes from the program that uses the library. Refer to the manual of that program.
6. Do the remedy.
7. If the problem continues, do the procedure *Report a problem*.

Result

You know the probable cause of the message and the corrective action.

7.1.1 How to read a message code

The letter in the middle of the code gives the severity. Table 7.1 tells you what to do for each letter.

Table 7.1: Severity letters and necessary action

Letter	Severity	Action
T, D, I, N	`TRACE` to `NOTICE`	No action is necessary. The message is information.
W	`WARNING`	Read the cause. Make sure that the result is correct.
E	`ERROR`	Do the remedy. Then do the operation again.
C	`CRITICAL`	Do the remedy. Then start the program again.

7.2 Message reference

This section gives all messages of the library, in the sequence of their message numbers. Table 7.2 gives the number ranges.

Table 7.2: Number ranges of the library messages

Numbers	Subject
0001 to 0099	Configuration
0101 to 0199	Message catalogues
0201 to 0299	Hooks: unhandled exceptions and Python warnings
0301 to 0399	Handling tools, progress, stalls and the flight recorder
0401 to 0499	Runs
0501 to 0599	Reports and integrity checks
0601 to 0699	Soft checks

7.2.1 ANN-E-0001

Severity
ERROR

Message
`The configuration option '{option}' is not known.`

Category
Configuration

Cause
The configuration contains an option that the library does not define.

Remedy
Remove the option or correct its name. Refer to the configuration reference.

Since
1.0.0

7.2.2 ANN-E-0002

Severity
ERROR

Message
`The value {value!r} is not permitted for the configuration option '{option}'.`

Category
Configuration

Cause
The value has an incorrect type or is out of the permitted range.

Remedy
Set the option to a permitted value. Refer to the configuration reference.

Since
1.0.0

7.2.3 ANN-E-0003

Severity
ERROR

Message
`The configuration file '{path}' cannot be read: {reason}`

Category
Configuration

Cause
The file does not exist, you do not have access, or the contents are not correct.

Remedy
Make sure that the path is correct and that the file contains correct TOML or JSON data.

Since
1.0.0

7.2.4 ANN-E-0004

Severity
ERROR

Message
`The configuration file type '{suffix}' is not supported.`

Category
Configuration

Cause
The suffix of the file name is not .toml or .json.

Remedy
Use a TOML file or a JSON file.

Since
1.0.0

7.2.5 ANN-D-0010

Severity
DEBUG

Message
`Logging is configured. Level: {level}. Handlers: {handlers}.`

Category
Configuration

Cause
The program called the configure function.

Remedy
No action is necessary.

Since
1.0.0

7.2.6 ANN-E-0101

Severity
ERROR

Message
`The message code or prefix '{item}' is not correct.`

Category
Message catalogue

Cause
The code does not have the format PREFIX-L-NNNN, the prefix is not correct, or the message text is empty.

Remedy
Use a prefix of two to eight capital letters or digits, one severity letter (T, D, I, N, W, E or C) and four digits.

Since
1.0.0

7.2.7 ANN-E-0102

Severity
ERROR

Message
`The message code '{item}' is already in the catalogue.`

Category
Message catalogue

Cause
Two definitions in one catalogue have the same code or the same message number.

Remedy
Give a different message number to the new definition.

Since
1.0.0

7.2.8 ANN-E-0103

Severity
ERROR

Message
`The message code or prefix '{item}' is not in a registered catalogue.`

Category
Message catalogue

Cause
The code contains a typing error, or the program did not import the module that defines the catalogue.

Remedy
Correct the code, or import the module that defines the catalogue before you use the code.

Since
1.0.0

7.2.9 ANN-E-0104

Severity
ERROR

Message
`The catalogue file '{path}' cannot be read:  {reason}`

Category
Message catalogue

Cause
The file does not exist, you do not have access, or the contents are not correct.

Remedy
Make sure that the path is correct and that the file has the catalogue file structure.

Since
1.0.0

7.2.10 ANN-E-0105

Severity
ERROR

Message
`The catalogue prefix '{prefix}' is already registered.`

Category
Message catalogue

Cause
Two catalogues have the same prefix, or the program made the same catalogue two times.

Remedy
Use a different prefix. In an interactive session, set the parameter replace to True.

Since
1.0.0

7.2.11 ANN-C-0201

Severity
CRITICAL

Message
`An exception was not caught: {exc_type}: {exc_message}`

Category
Exception hooks

Cause
The program raised an exception and no code caught it. The program stops.

Remedy
Read the traceback in the log. Correct the cause, or add error handling to the program.

Since
1.0.0

7.2.12 ANN-C-0202

Severity
CRITICAL

Message
`An exception was not caught in the thread '{thread}': {exc_type}: {exc_message}`

Category
Exception hooks

Cause
A thread raised an exception and no code caught it. The thread stops.

Remedy
Read the traceback in the log. Add error handling to the function that the thread runs.

Since
1.0.0

7.2.13 ANN-E-0203

Severity
ERROR

Message
`The asyncio event loop reported an error: {detail}`

Category
Exception hooks

Cause
A task or a callback raised an exception and no code caught it.

Remedy
Read the traceback in the log. Await the task and catch its exceptions.

Since
1.0.0

7.2.14 ANN-E-0204

Severity
ERROR

Message
`Python ignored an exception: {exc_type}: {exc_message} ({detail})`

Category
Exception hooks

Cause
An exception occurred where Python cannot raise it, for example in a ___del___ method.

Remedy
Read the traceback in the log. Correct the code of the object that the message identifies.

Since
1.0.0

7.2.15 ANN-W-0205

Severity
WARNING

Message
`{category}: {text}`

Category
Exception hooks

Cause
A module issued a Python warning that does not have a message code.

Remedy
Read the warning text. Refer to the documentation of the module that issued the warning.

Since
1.0.0

7.2.16 ANN-W-0301

Severity
WARNING

Message
`Attempt {attempt} of {attempts} of '{operation}' failed: {reason}. The next attempt starts in {delay} s.`

Category
Error handling

Cause
The operation raised an exception that the retry policy accepts.

Remedy
No action is necessary if a subsequent attempt is successful.

Since
1.0.0

7.2.17 ANN-E-0302

Severity
ERROR

Message
`The operation '{operation}' failed after {attempts} attempts: {reason}.`

Category
Error handling

Cause
All attempts of the operation raised an exception.

Remedy
Read the last exception in the log. Correct its cause and start the operation again.

Since
1.0.0

7.2.18 ANN-E-0303

Severity
ERROR

Message
`The operation '{operation}' failed: {exc_type}: {exc_message}`

Category
Error handling

Cause
The operation raised an exception that does not have a message code.

Remedy
Read the traceback in the log and correct the cause.

Since
1.0.0

7.2.19 ANN-E-0304

Severity
ERROR

Message
`{count} errors occurred in '{operation}'.`

Category
Error handling

Cause
An error collector recorded one or more errors.

Remedy
Read each error in the log and correct its cause.

Since
1.0.0

7.2.20 ANN-W-0305

Severity
WARNING

Message
`The operation '{operation}' was slow. Time: {elapsed} s. Limit: {limit} s.`

Category
Error handling

Cause
The operation used more time than the limit that the program set.

Remedy
Examine the load on the computer and the size of the input data.

Since
1.0.0

7.2.21 ANN-D-0306

Severity
DEBUG

Message
`The call of '{function}' started.`

Category
Error handling

Cause
A function that has the log_calls decorator started.

Remedy
No action is necessary.

Since
1.0.0

7.2.22 ANN-D-0307

Severity
DEBUG

Message
`The call of '{function}' completed in {elapsed} s.`

Category
Error handling

Cause
A function that has the log_calls decorator returned.

Remedy
No action is necessary.

Since
1.0.0

7.2.23 ANN-I-0308

Severity
INFO

Message
`The operation '{operation}' completed in {elapsed} s.`

Category
Error handling

Cause
A timed block completed.

Remedy
No action is necessary.

Since
1.0.0

7.2.24 ANN-W-0309

Severity
WARNING

Message
`'{name}' is deprecated since version {since}. {advice}`

Category
Error handling

Cause
The program uses a function or a class that a future version will remove.

Remedy
Change the program as the message tells you.

Since
1.0.0

7.2.25 ANN-E-0310

Severity
ERROR

Message
`The operation '{operation}' did not complete in {limit} s.`

Category
Error handling

Cause
The operation used more time than the permitted limit.

Remedy
Increase the limit, or examine why the operation is slow.

Since
1.0.0

7.2.26 ANN-I-0320

Severity
INFO

Message
`{count} records of the message '{key}' were not recorded.`

Category
Error handling

Cause
A once, every or throttle method rejected records of the same message.

Remedy
No action is necessary.

Since
1.1.0

7.2.27 ANN-I-0330

Severity
INFO

Message
`Progress of '{operation}': {done} of {total} ({percent} %). Rate: {rate} per second. Time to completion: {eta} s.`

Category
Error handling

Cause
A progress object recorded the state of a long operation.

Remedy
No action is necessary.

Since
1.1.0

7.2.28 ANN-I-0331

Severity
INFO

Message
`The operation '{operation}' completed {done} items in {elapsed} s.`

Category
Error handling

Cause
A progress object came to its end.

Remedy
No action is necessary.

Since
1.1.0

7.2.29 ANN-W-0332

Severity
WARNING

Message
`No progress of '{operation}' for {idle} s.`

Category
Error handling

Cause
The operation did not report progress in the permitted time. It is possibly blocked or very slow.

Remedy
Examine the last records of the operation. Examine the load on the computer and the connected devices.

Since
1.1.0

7.2.30 ANN-I-0333

Severity
INFO

Message
`The operation '{operation}' continued after {idle} s without progress.`

Category
Error handling

Cause
The operation reported progress again after a stall.

Remedy
No action is necessary.

Since
1.1.0

7.2.31 ANN-I-0340

Severity
INFO

Message
`The flight recorder wrote {count} earlier records because of an event of the severity {trigger}.`

Category
Error handling

Cause
An event with the trigger severity or higher occurred. The flight recorder then wrote the records that it kept in memory.

Remedy
Read the earlier records to find what occurred before the event.

Since
1.1.0

7.2.32 ANN-N-0401

Severity
NOTICE

Message
`The run '{run}' started.`

Category
Runs

Cause
The program called the start_run function.

Remedy
No action is necessary.

Since
1.1.0

7.2.33 ANN-N-0402

Severity
NOTICE

Message
`The run '{run}' completed in {elapsed} s. Errors: {errors}. Warnings: {warnings}.`

Category
Runs

Cause
The run came to its end.

Remedy
If the number of errors is not 0, read the error records of the run.

Since
1.1.0

7.2.34 ANN-E-0403

Severity
ERROR

Message
`The run '{run}' stopped after {elapsed} s because of an exception: {exc_type}: {exc_message}`

Category
Runs

Cause
The code of the run raised an exception and no code caught it.

Remedy
Read the traceback in the log and correct the cause. Then start the run again.

Since
1.1.0

7.2.35 ANN-E-0501

Severity
ERROR

Message
`Line {line} of the file '{path}' is not a correct event record.`

Category
Reports

Cause
The line does not contain a JSON object that the library wrote.

Remedy
Make sure that the file is a JSON Lines log file of this library. Remove the incorrect line.

Since
1.0.0

7.2.36 ANN-E-0510

Severity
ERROR

Message
`The log file '{path}' did not pass the integrity check. First problem: line {line}:  {reason}`

Category
Reports

Cause
A person or a program changed, removed or added a record after the library wrote the file. An incomplete last line after an abnormal stop also causes this message.

Remedy
Do not use the file as evidence. Get a copy of the file from a backup and do the check again.

Since
1.1.0

7.2.37 ANN-E-0511

Severity
ERROR

Message
`The log file '{path}' has no integrity data.`

Category
Reports

Cause
The option file_chain was not set when the library wrote the file.

Remedy
Set the option file_chain to true for future runs.

Since
1.1.0

7.2.38 ANN-E-0601

Severity
ERROR

Message
`The value of '{name}' ({value}) is out of the range {minimum} to {maximum}.`

Category
Checks

Cause
The value is less than the minimum or more than the maximum.

Remedy
Supply a value in the permitted range.

Since
1.1.0

7.2.39 ANN-E-0602

Severity
ERROR

Message
`'{name}' contains {count} values that are not finite numbers.`

Category
Checks

Cause
The data contain NaN or infinite values. A division by zero or an empty input is a usual cause.

Remedy
Examine the input data and the calculation that made the values.

Since
1.1.0

7.2.40 ANN-E-0603

Severity
ERROR

Message
`'{name}' has the type {actual}. The permitted type is {expected}.`

Category
Checks

Cause
The program gave a value of an incorrect type.

Remedy
Supply a value of the permitted type.

Since
1.1.0

7.2.41 ANN-E-0604

Severity
ERROR

Message
`'{name}' is empty.`

Category
Checks

Cause
The program gave an empty sequence, text or data set.

Remedy
Supply data that contain one or more items.

Since
1.1.0

7.2.42 ANN-E-0605

Severity
ERROR

Message
`The value of '{name}' ({value}) is not one of the permitted values: {choices}.`

Category
Checks

Cause
The value is not in the list of permitted values.

Remedy
Supply one of the permitted values.

Since
1.1.0

7.2.43 ANN-E-0606

Severity
ERROR

Message
`The check '{name}' failed.`

Category
Checks

Cause
A condition that the program examined was not true.

Remedy
Read the fields of the record to find the values that failed.

Since
1.1.0

7.2.44 ANN-W-0611

Severity
WARNING

Message
`The value of '{name}' ({value}) is out of the range {minimum} to {maximum}.`

Category
Checks

Cause
The value is less than the minimum or more than the maximum.

Remedy
Supply a value in the permitted range. If the value is intended, no action is necessary.

Since
1.1.0

7.2.45 ANN-W-0612

Severity
WARNING

Message
`'{name}' contains {count} values that are not finite numbers.`

Category
Checks

Cause
The data contain NaN or infinite values. A division by zero or an empty input is a usual cause.

Remedy
Examine the input data and the calculation that made the values. If the value is intended, no action is necessary.

Since
1.1.0

7.2.46 ANN-W-0613

Severity
WARNING

Message
`'{name}' has the type {actual}. The permitted type is {expected}.`

Category
Checks

Cause
The program gave a value of an incorrect type.

Remedy
Supply a value of the permitted type. If the value is intended, no action is necessary.

Since
1.1.0

7.2.47 ANN-W-0614

Severity
WARNING

Message
`'{name}' is empty.`

Category
Checks

Cause
The program gave an empty sequence, text or data set.

Remedy
Supply data that contain one or more items. If the value is intended, no action is necessary.

Since
1.1.0

7.2.48 ANN-W-0615

Severity
WARNING

Message
`The value of '{name}' ({value}) is not one of the permitted values: {choices}.`

Category
Checks

Cause
The value is not in the list of permitted values.

Remedy
Supply one of the permitted values. If the value is intended, no action is necessary.

Since
1.1.0

7.2.49 ANN-W-0616

Severity
WARNING

Message
`The check '{name}' failed.`

Category
Checks

Cause
A condition that the program examined was not true.

Remedy
Read the fields of the record to find the values that failed. If the value is intended, no action is necessary.

Since
1.1.0

7.3 Problems without a message code

Table 7.3: Problems, probable causes and remedies

Problem	Probable cause	Remedy
The terminal cannot find the command `annunciator`.	The directory of the Python scripts is not in the search path of the terminal.	Use `python -m annunciator`. If you use a virtual environment, activate it.
Python shows `ModuleNotFoundError: No module named 'annunciator'`.	The library is not installed in the Python environment that runs the program.	Do the procedure in *Installation* with the same Python environment.
The console shows no events.	The program did not call `configure`, or the option `level` is too high, or the option `console` is false.	Call `configure` near the start of the program. Decrease `level` or `console_level`.
The console shows each event two times.	A different handler is installed on the root logger, for example by `logging.basicConfig`.	Remove the call of `logging.basicConfig`. Use only `configure`.
The log file does not exist.	The library makes the file when it writes the first record. No record had the necessary severity.	Decrease `level` or `file_level`.
The log file does not contain the last events.	The option `asynchronous` is true and the program stopped abnormally.	Call `shutdown` before the program stops. Set `asynchronous` to false if the problem continues.
The console shows no colour.	The output is not a terminal, or the variable `NO_COLOR` is set.	Set `console_format` to `"colour"`.
The console shows characters such as `[32m` in the text.	The terminal does not support ANSI colour codes.	Set `console_format` to `"plain"`.
A field shows `[REDACTED]` but the value is not secret.	The field name contains a part that is in `redact_keys`, for example `token`.	Change the field name, or remove the part from `redact_keys`.
A Python warning is in the log only one time.	The standard `warnings` module shows the same warning from the same line only one time.	No action is necessary. To record each occurrence, call `warnings.simplefilter("always")`.
The source position of an event is a line in your own helper function.	The helper function calls the log method.	Give the parameter `stacklevel=2` to the log method in the helper function.
The summary of `summarise()` contains fewer events than the log file.	The event store is full and removed the oldest events.	Increase `store_capacity`, or make the summary from the log file.
The log file contains `DEBUG` records but the option `level` is `"INFO"`.	The flight recorder wrote its kept records after an error. These records have the field `flight_recorder`.	No action is necessary. To stop this function, set `flight_recorder` to 0.
The message of `once` does not occur in a second run of the same process.	The library keeps the state of `once` until the process stops.	Use `start_run` for each run, or call `reset_occurrences`.
The command `verify` fails for a file that nobody changed.	The program used a key and the command does not have the same key. A different cause is an incomplete last line after an abnormal stop.	Set `ANNUNCIATOR_CHAIN_KEY` to the key of the run. Examine the last line of the file.

continues on next page

Table 7.3 – continued from previous page

Problem	Probable cause	Remedy
The records of worker processes are not in the log file.	The worker processes did not call `init_worker`, or the program stopped before `stop_listener`.	Do the procedure in *Make the records robust*.
A watchdog records `ANN-W-0332` but the operation is in progress.	The code does not call `beat` or `update` frequently, or the limit is too short.	Call `beat` in the inner loop, or increase the limit.
A TOML file cannot be read on Python 3.9 or 3.10. The reason contains `tomli`.	The package `tomli` is not installed.	Install the extra `toml`. Refer to *Installation*.

7.4 Report a problem

Purpose

To give the necessary data to the person who supplies the software.

Procedure

1. Set the option `level` to `"DEBUG"` and the option `file_format` to `"json"`.
2. Do the operation that causes the problem again.
3. Collect these items:
   - the log file,
   - the output of `annunciator --version`,
   - the output of `python --version`,
   - the name and the version of the operating system,
   - the steps that cause the problem.

   Caution

   Read the log file before you send it. Make sure that it contains no secret data.

4. Send the items to the person who supplied the software to you.

Result

The supplier has the data that are necessary to find the cause.

CHAPTER

EIGHT

GLOSSARY

catalogue
message catalogue
: A set of message definitions that have the same prefix.

cause
: The text that gives the probable reason for an event. It is a part of a message definition.

chain
: The sequence of hashes that connects the records of a log file. Each record contains the hash of the record before it.

coded error
: An exception that has a message code, a cause and a remedy. All coded errors are subclasses of `CodedError`.

coded warning
: A Python warning that has a message code, a cause and a remedy. All coded warnings are subclasses of `CodedWarning`.

context
: Key and value pairs that the library adds to all events of one thread or one task.

correlation identifier
: A unique text in the context. It lets you find all events of one operation.

error policy
: The rule that tells a handling tool what to do with an exception after the record. The tool can raise it again, change it or stop it.

event
: One occurrence that the library recorded. An event has a time, a severity, a message and optional structured data.

event store
: A handler that keeps the last events in memory.

field
structured field
: A key and value pair that is a part of an event.

filter
: An object that accepts, rejects or changes a record before a handler writes it.

fingerprint
: A short identifier of a type of failure. Exceptions with the same class and the same sequence of functions have the same fingerprint.

flight recorder
: A handler that keeps the last detailed records in memory and writes them when a failure occurs.

formatter
: An object that changes a record into text.

handler
An object that sends records to a destination, for example the console or a file.

hook
A function that Python calls when an exception has no handler or when a module issues a warning.

JSON Lines
A text file format. Each line of the file is one JSON object.

logger
structured logger
An object with methods that record events. Each logger has a name.

manifest
run manifest
The data that identify the environment of a run: the software versions, the source revision, the configuration and the seed.

message code
The unique identifier of a message definition, for example `APP-E-0042`.

message definition
The code, the text, the cause and the remedy of one message.

prefix
The first part of a message code. It identifies the catalogue.

record
The object that the standard `logging` module makes for one log call. The library changes a record into an event.

redaction
The replacement of secret data in a record with the text `[REDACTED]`.

remedy
The text that gives the corrective action for an event. It is a part of a message definition.

replacement field
A name in braces in a message text, for example `{path}`. The library puts a value in its position.

rotation
The procedure that closes a full log file, gives it a new name and starts a new file.

run
One complete operation of a program, for example one measurement or one simulation.

severity
The importance of an event. The library has seven severities, from `TRACE` to `CRITICAL`.

soft check
A function that examines a condition on data. It raises an error or records a warning when the condition is not true.

stall
The condition of an operation that makes no progress but does not raise an exception.

traceback
The list of function calls that were active when an exception occurred.

watchdog
An object that records a warning when an operation makes no progress in a permitted time.

8.1 Abbreviations

Table 8.1: Abbreviations

Abbreviation	Meaning
ANSI	American National Standards Institute
API	Application programming interface
ASD	AeroSpace, Security and Defence Industries Association of Europe
CSV	Comma-separated values
HMAC	Hash-based message authentication code
HTML	HyperText Markup Language
JSON	JavaScript Object Notation
PDF	Portable Document Format
SHA	Secure Hash Algorithm
STE	Simplified Technical English
TOML	Tom's Obvious, Minimal Language
UTC	Coordinated Universal Time

CHAPTER

NINE

RELATED INFORMATION

9.1 Standards

Table 9.1: Standards

Identifier	Title
ISO/IEC/IEEE 26514:2022	Systems and software engineering. Design and development of information for users.
ASD-STE100, Issue 9	Simplified Technical English. International specification for the preparation of technical documentation in a controlled language.
ISO 8601	Date and time. Representations for information interchange.

9.2 Python documentation

Table 9.2: Python documentation

Subject	Address
Module `logging`	https://docs.python.org/3/library/logging.html
Module `logging.handlers`	https://docs.python.org/3/library/logging.handlers.html
Module `warnings`	https://docs.python.org/3/library/warnings.html
Module `contextvars`	https://docs.python.org/3/library/contextvars.html
Errors and exceptions	https://docs.python.org/3/tutorial/errors.html
Installation of packages	https://packaging.python.org/en/latest/tutorials/installing-packages/

9.3 Other documentation

Table 9.3: Other documentation

Subject	Address
Sphinx	https://www.sphinx-doc.org/
JSON Lines	https://jsonlines.org/
TOML	https://toml.io/
`NO_COLOR` convention	https://no-color.org/

9.4 Files in the software package

Table 9.4: Files in the software package

File or directory	Contents
`README.md`	A short description and a quick start.
`CHANGELOG.md`	The changes in each version of the software.
`LICENSE`	The text of the licence.
`examples/`	A complete example program.
`docs/`	The source files of this manual.
`tools/ste_check.py`	The tool that examines the text of this manual.

CHAPTER

TEN

CONFORMITY TO STANDARDS

This chapter tells you how the manual applies the two standards in Table 9.1. It also gives the limits of the conformity statement.

10.1 Structure and content: ISO/IEC/IEEE 26514:2022

Table 10.1 shows where the manual gives each information component that the standard defines for information for users.

Table 10.1: Information components and their position in this manual

Component	Position
Identification data	Title page and *Document identification*
Table of contents	Contents list on the first page. Navigation panel in the HTML form.
List of illustrations	List of figures and list of tables in the PDF form. All figures and tables have a number and a title.
Introduction	First page and *Information for use of this manual*
Information for use of the documentation	*Information for use of this manual*
Concept of operations	*Concept of operations*
Procedures	*Installation* and *Procedures*
Information on software commands	*Command-line interface*, *Configuration options* and *Programming interface*
Error messages and problem resolution	*Messages and problem resolution*
Critical information (warnings, cautions, notes)	Definitions in *Information for use of this manual*. Each caution and note is immediately before the step or the text that it applies to.
Glossary	*Glossary*
Related information sources	*Related information*
Navigational features	Section numbers, cross-references, numbered figures and tables. Links to the previous page and the subsequent page in the HTML form.
Index	General index in the two forms
Search capability	Search function in the HTML form. Text search in the PDF form.

The manual also applies these principles of the standard:

- The procedures are task-oriented. Each procedure has a purpose, initial conditions, numbered steps and a result.
- The reference data are separate from the procedures.
- The same term always has the same meaning. *Glossary* defines the terms.

- The message reference comes directly from the software. Thus the manual and the software agree.

10.2 Language: ASD-STE100 Issue 9

The text of the manual applies the writing rules of the specification. Table 10.2 gives the primary rules and their application.

Table 10.2: Application of the writing rules

Rule	Application in this manual
Use approved words with their approved meanings.	Applied. Technical names and technical verbs are in Table 2.3 and Table 2.4.
Keep procedural sentences to a maximum of 20 words.	Applied. The tool `ste_check.py` counts the words.
Keep descriptive sentences to a maximum of 25 words.	Applied. The tool `ste_check.py` counts the words.
Keep paragraphs to a maximum of 6 sentences.	Applied. The tool `ste_check.py` counts the sentences.
Write only one instruction in each step.	Applied, unless two actions occur at the same time.
Write instructions in the imperative form.	Applied in all numbered steps.
Use the active voice.	Applied in procedures. Applied in descriptive text where possible.
Do not make noun clusters of more than three nouns.	Applied, with the exception of technical names.
Use a vertical list for complex text.	Applied.
Start a caution with a clear command or condition.	Applied.

The documentation text in the source code applies the same rules. That text is the source of *Programming interface*.

10.3 Limits of the conformity statement

Note

The author applied the two standards during the preparation of this manual. An independent organisation did not examine the manual.

These limits apply:

- The tool `ste_check.py` examines sentence length, paragraph length, contractions and a list of words that are not approved. It does not contain the full dictionary of the specification.
- The tool cannot examine the meaning of a word or the voice of a sentence. A person must examine these items.
- Text that comes from other software is not in Simplified Technical English. Examples are Python tracebacks and the names in program code.
- The licence text in the file `LICENSE` is a legal text. The writing rules do not apply to it.

If your organisation must show formal conformity, a qualified person must examine the manual against the two standards.

PYTHON MODULE INDEX

a

annunciator.catalogue, 73
annunciator.checks, 125
annunciator.cli, 132
annunciator.config, 90
annunciator.context, 72
annunciator.errors, 78
annunciator.events, 87
annunciator.filters, 98
annunciator.formatters, 97
annunciator.handlers, 94
annunciator.handling, 102
annunciator.hooks, 108
annunciator.htmlreport, 132
annunciator.integrity, 121
annunciator.levels, 64
annunciator.lint, 128
annunciator.logger, 66
annunciator.multiprocess, 123
annunciator.provenance, 118
annunciator.recorder, 114
annunciator.report, 109
annunciator.sphinxext, 133
annunciator.testing, 129
annunciator.warns, 84
annunciator.watch, 115

INDEX

A

add() (*annunciator.catalogue.MessageCatalogue method*), 75
add() (*annunciator.handling.ErrorCollector method*), 106
add_warning() (*annunciator.handling.ErrorCollector method*), 106
AggregateError, 84
ANN-C-0201, 139
ANN-C-0202, 139
ANN-D-0010, 137
ANN-D-0306, 142
ANN-D-0307, 143
ANN-E-0001, 136
ANN-E-0002, 136
ANN-E-0003, 136
ANN-E-0004, 137
ANN-E-0101, 137
ANN-E-0102, 138
ANN-E-0103, 138
ANN-E-0104, 138
ANN-E-0105, 139
ANN-E-0203, 140
ANN-E-0204, 140
ANN-E-0302, 141
ANN-E-0303, 141
ANN-E-0304, 142
ANN-E-0310, 143
ANN-E-0403, 146
ANN-E-0501, 147
ANN-E-0510, 147
ANN-E-0511, 147
ANN-E-0601, 148
ANN-E-0602, 148
ANN-E-0603, 148
ANN-E-0604, 149
ANN-E-0605, 149
ANN-E-0606, 149
ANN-I-0308, 143
ANN-I-0320, 144
ANN-I-0330, 144
ANN-I-0331, 144
ANN-I-0333, 145
ANN-I-0340, 145
ANN-N-0401, 146
ANN-N-0402, 146
ANN-W-0205, 140
ANN-W-0301, 141
ANN-W-0305, 142
ANN-W-0309, 143
ANN-W-0332, 145
ANN-W-0611, 150
ANN-W-0612, 150
ANN-W-0613, 150
ANN-W-0614, 151
ANN-W-0615, 151
ANN-W-0616, 151
annunciator.catalogue module, 73
annunciator.checks module, 125
annunciator.cli module, 132
annunciator.config module, 90
annunciator.context module, 72
annunciator.errors module, 78
annunciator.events module, 87
annunciator.filters module, 98
annunciator.formatters module, 97
annunciator.handlers module, 94
annunciator.handling module, 102
annunciator.hooks module, 108
annunciator.htmlreport module, 132
annunciator.integrity module, 121
annunciator.levels module, 64
annunciator.lint module, 128
annunciator.logger module, 66

annunciator.multiprocess
    module, 123
annunciator.provenance
    module, 118
annunciator.recorder
    module, 114
annunciator.report
    module, 109
annunciator.sphinxext
    module, 133
annunciator.testing
    module, 129
annunciator.warns
    module, 84
annunciator.watch
    module, 115
API reference, 64
ASD-STE100, 160
assert_logged() (*annunciator.testing.EventCapture method*), 131
assert_no_problems() (*annunciator.testing.EventCapture method*), 131
assert_not_logged() (*annunciator.testing.EventCapture method*), 131
asyncio, 35
AsyncQueueHandler (*class in annunciator.handlers*), 96
audience, 6

B

beat() (*annunciator.watch.Watchdog method*), 117
bind() (*annunciator.logger.StructuredLogger method*), 66
bind() (*in module annunciator.context*), 72
bound (*annunciator.logger.StructuredLogger property*), 66
build_parser() (*in module annunciator.cli*), 132
by_code (*annunciator.report.EventReport attribute*), 110
by_fingerprint (*annunciator.report.EventReport attribute*), 110
by_logger (*annunciator.report.EventReport attribute*), 110
by_severity (*annunciator.report.EventReport attribute*), 110

C

CallbackHandler (*class in annunciator.handlers*), 95
capture_events() (*in module annunciator.testing*), 131
catalogue, **155**
catalogue file, 58
CatalogueDirective (*class in annunciator.sphinxext*), 133
CatalogueError, 83
catalogues() (*in module annunciator.catalogue*), 78
catch() (*annunciator.handling.ErrorCollector method*), 107
cause, **155**
chain, **155**
chain_key_from_env() (*in module annunciator.integrity*), 123
ChainFormatter (*class in annunciator.integrity*), 121
ChainResult (*class in annunciator.integrity*), 122
change record, 4
changed_codes (*annunciator.report.RunComparison attribute*), 113
check() (*in module annunciator.checks*), 125
check_finite() (*in module annunciator.checks*), 126
check_not_empty() (*in module annunciator.checks*), 127
check_one_of() (*in module annunciator.checks*), 127
check_range() (*in module annunciator.checks*), 126
check_type() (*in module annunciator.checks*), 127
clear() (*annunciator.handlers.EventStoreHandler method*), 95
clear() (*annunciator.testing.EventCapture method*), 130
clear_context() (*in module annunciator.context*), 72
close() (*annunciator.recorder.FlightRecorderHandler method*), 115
close() (*annunciator.watch.Progress method*), 116
code (*annunciator.lint.LintFinding attribute*), 129
CODE_PATTERN (*in module annunciator.catalogue*), 73
coded error, **155**
    procedure, 29
    reference, 56
coded warning, **155**
    procedure, 31
    reference, 56
CodedError, 78
CodedWarning, 84
CodeFilter (*class in annunciator.filters*), 99
codes (*annunciator.testing.EventCapture property*), 130
coerce() (*annunciator.levels.Severity class method*), 65
collect_manifest() (*in module annunciator.provenance*), 118
ColourFormatter (*class in annunciator.formatters*), 97
command-line interface, 61
commands, 61
compare_runs() (*in module annunciator.report*), 114
comparison of runs, 43

concept of operations, 9
configuration
    procedure, 22
    reference, 53
ConfigurationError, 79
ConfigurationWarning, 85
configure() (*in module annunciator.config*), 93
conformity, 160
ConsoleHandler (*class in annunciator.handlers*), 94
context, **155**
    procedure, 25
context() (*in module annunciator.context*), 72
ContextFilter (*class in annunciator.filters*), 99
conventions, 6
correlation identifier, **155**
count() (*annunciator.report.EventReport method*), 110
counts() (*annunciator.handlers.EventStoreHandler method*), 95
CRITICAL (*annunciator.levels.Severity attribute*), 65
critical() (*annunciator.logger.StructuredLogger method*), 68
CSV, 36
    format, 58
current_config() (*in module annunciator.config*), 94

D

data flow, 9
DataError, 80
DataQualityWarning, 85
DEBUG (*annunciator.levels.Severity attribute*), 64
debug() (*annunciator.logger.StructuredLogger method*), 67
default_code (*annunciator.errors.AggregateError attribute*), 84
default_code (*annunciator.errors.OperationTimeoutError attribute*), 82
default_code (*annunciator.errors.RetryExhaustedError attribute*), 82
default_code (*annunciator.warns.DeprecatedFeatureWarning attribute*), 86
DEFAULT_REDACT_KEYS (*in module annunciator.filters*), 99
default_severity (*annunciator.errors.CodedError attribute*), 79
default_severity (*annunciator.errors.InternalError attribute*), 83
default_severity (*annunciator.warns.CodedWarning attribute*), 85
define() (*annunciator.catalogue.MessageCatalogue method*), 75
DependencyError, 82
deprecated() (*in module annunciator.handling*), 107
DeprecatedFeatureWarning, 86
deprecation, 31
describe() (*annunciator.errors.AggregateError method*), 84
detail (*annunciator.lint.LintFinding attribute*), 129
discard() (*annunciator.recorder.FlightRecorderHandler method*), 115
document identification, 4
done (*annunciator.watch.Progress attribute*), 116
dropped (*annunciator.filters.RateLimitFilter attribute*), 101
DuplicateFilter (*class in annunciator.filters*), 100

E

elapsed (*annunciator.handling.timed attribute*), 105
elapsed (*annunciator.watch.Progress property*), 116
emit() (*annunciator.handlers.CallbackHandler method*), 96
emit() (*annunciator.handlers.EventStoreHandler method*), 95
emit() (*annunciator.recorder.FlightRecorderHandler method*), 115
emit() (*annunciator.testing.EventCapture method*), 130
ENV_PREFIX (*in module annunciator.config*), 94
environment variable, 22
    reference, 53
ERROR (*annunciator.levels.Severity attribute*), 65
error collector, 32
error messages, 134
error policy, **155**
    procedure, 32
error() (*annunciator.logger.StructuredLogger method*), 68
ErrorCollector (*class in annunciator.handling*), 105
errors (*annunciator.errors.AggregateError attribute*), 84
errors (*annunciator.handling.ErrorCollector attribute*), 106
errors (*annunciator.provenance.Run property*), 119
event, **155**
Event (*class in annunciator.events*), 87
event store, **155**
    procedure, 36

event() (*annunciator.logger.StructuredLogger method*), 70
EventCapture (*class in annunciator.testing*), 130
EventReport (*class in annunciator.report*), 109
events (*annunciator.handlers.EventStoreHandler property*), 95
events (*annunciator.testing.EventCapture attribute*), 130
EventStoreHandler (*class in annunciator.handlers*), 94
every() (*annunciator.logger.StructuredLogger method*), 69
exception (*annunciator.handling.guard attribute*), 103
exception() (*annunciator.logger.StructuredLogger method*), 69
ExceptionInfo (*class in annunciator.events*), 88
exit status, 29
    commands, 61
    reference, 56
exit_code (*annunciator.errors.CatalogueError attribute*), 84
exit_code (*annunciator.errors.CodedError attribute*), 79
exit_code (*annunciator.errors.ConfigurationError attribute*), 79
exit_code (*annunciator.errors.DataError attribute*), 80
exit_code (*annunciator.errors.DependencyError attribute*), 83
exit_code (*annunciator.errors.InternalError attribute*), 83
exit_code (*annunciator.errors.OperationTimeoutError attribute*), 82
exit_code (*annunciator.errors.ResourceAccessError attribute*), 81
exit_code (*annunciator.errors.ResourceError attribute*), 80
exit_code (*annunciator.errors.ResourceNotFoundError attribute*), 81
exit_code (*annunciator.errors.RetryExhaustedError attribute*), 82
exit_code (*annunciator.errors.ValidationError attribute*), 80
exit_code (*annunciator.provenance.Run property*), 119
exit_code() (*annunciator.report.EventReport method*), 111
explain() (*in module annunciator.catalogue*), 78
export_csv() (*in module annunciator.report*), 111

F

failed (*annunciator.handling.guard property*), 104
field, **155**
field_names (*annunciator.catalogue.MessageDefinition property*), 74
filter, **155**
filter() (*annunciator.filters.CodeFilter method*), 100
filter() (*annunciator.filters.ContextFilter method*), 99
filter() (*annunciator.filters.DuplicateFilter method*), 100
filter() (*annunciator.filters.LocalsFilter method*), 101
filter() (*annunciator.filters.RateLimitFilter method*), 101
filter() (*annunciator.filters.RedactionFilter method*), 100
filter() (*annunciator.filters.SeverityRangeFilter method*), 99
find() (*annunciator.catalogue.MessageCatalogue method*), 75
fingerprint, **155**
    procedure, 39
fingerprint() (*in module annunciator.events*), 90
finish() (*annunciator.provenance.Run method*), 120
first (*annunciator.report.EventReport attribute*), 110
first_seq (*annunciator.integrity.ChainResult attribute*), 122
flight recorder, **155**
    procedure, 39
FlightRecorderHandler (*class in annunciator.recorder*), 114
flush() (*annunciator.recorder.FlightRecorderHandler method*), 115
flushes (*annunciator.recorder.FlightRecorderHandler attribute*), 115
format() (*annunciator.catalogue.MessageDefinition method*), 74
format() (*annunciator.formatters.JsonFormatter method*), 98
format() (*annunciator.formatters.PlainFormatter method*), 97
format() (*annunciator.integrity.ChainFormatter method*), 122
format_event() (*annunciator.formatters.PlainFormatter method*), 97
formatter, **155**
from_dict() (*annunciator.catalogue.MessageCatalogue class method*), 76
from_dict() (*annunciator.catalogue.MessageDefinition class*

method), 74
from_dict() (*annunciator.config.LogConfig class method*), 92
from_dict() (*annunciator.events.Event class method*), 88
from_dict() (*annunciator.events.ExceptionInfo class method*), 89
from_dict() (*annunciator.events.SourceLocation class method*), 89
from_env() (*annunciator.config.LogConfig class method*), 92
from_events() (*annunciator.report.EventReport class method*), 110
from_exception() (*annunciator.events.ExceptionInfo class method*), 89
from_file() (*annunciator.config.LogConfig class method*), 92
from_json() (*annunciator.events.Event class method*), 88
from_letter() (*annunciator.levels.Severity class method*), 65
from_record() (*annunciator.events.Event class method*), 88

G

GENESIS (*in module annunciator.integrity*), 123
get() (*annunciator.catalogue.MessageCatalogue method*), 75
get_catalogue() (*in module annunciator.catalogue*), 77
get_context() (*in module annunciator.context*), 72
get_logger() (*in module annunciator.logger*), 71
get_store() (*in module annunciator.config*), 94
git_revision() (*in module annunciator.provenance*), 119
gone_codes (*annunciator.report.RunComparison attribute*), 113
gone_failures (*annunciator.report.RunComparison attribute*), 113
guard, 32
guard (*class in annunciator.handling*), 103

H

handle_errors() (*in module annunciator.handling*), 102
handler, **156**
has_content (*annunciator.sphinxext.CatalogueDirective attribute*), 133
has_errors (*annunciator.report.EventReport property*), 111
has_warnings (*annunciator.report.EventReport property*), 110
hook, **156**
hooks
    procedure, 35
hooks_installed() (*in module annunciator.hooks*), 108
HTML report
    procedure, 48

I

identical (*annunciator.report.RunComparison property*), 113
INFO (*annunciator.levels.Severity attribute*), 64
info() (*annunciator.logger.StructuredLogger method*), 68
init_worker() (*in module annunciator.multiprocess*), 124
install_asyncio_handler() (*in module annunciator.hooks*), 109
install_hooks() (*in module annunciator.hooks*), 108
installation, 16
integrity
    procedure, 43
InternalError, 83
is_configured() (*in module annunciator.config*), 93
is_enabled_for() (*annunciator.logger.StructuredLogger method*), 67
is_error (*annunciator.lint.LintFinding property*), 129
is_failure (*annunciator.levels.Severity property*), 65
is_problem (*annunciator.levels.Severity property*), 65
ISO/IEC/IEEE 26514, 160

J

JSON Lines, **156**
    format, 58
json_default() (*in module annunciator.events*), 89
JsonFormatter (*class in annunciator.formatters*), 98

K

KEY_VARIABLE (*in module annunciator.integrity*), 123
kind (*annunciator.lint.LintFinding attribute*), 129

L

last (*annunciator.report.EventReport attribute*), 110
last_seq (*annunciator.integrity.ChainResult attribute*), 122
letter (*annunciator.levels.Severity property*), 65
level, 53
line (*annunciator.lint.LintFinding attribute*), 128
lint
    procedure, 48

lint_paths() (*in module annunciator.lint*), 129
LintFinding (*class in annunciator.lint*), 128
load() (*annunciator.catalogue.MessageCatalogue class method*), 76
local variables
    procedure, 45
LocalsFilter (*class in annunciator.filters*), 101
log file, 22
log() (*annunciator.logger.StructuredLogger method*), 67
log_calls() (*in module annunciator.handling*), 105
LogConfig (*class in annunciator.config*), 90
logger, **156**
    procedure, 25
lookup() (*in module annunciator.catalogue*), 78

M

main() (*in module annunciator.cli*), 132
make_code() (*in module annunciator.catalogue*), 77
make_file_handler() (*in module annunciator.handlers*), 96
manifest, **156**
    procedure, 43
manifest (*annunciator.provenance.Run attribute*), 119
message catalogue, **155**
    procedure, 27
message code, 9, **156**
    procedure, 27
message definition, **156**
message reference, 38
MessageCatalogue (*class in annunciator.catalogue*), 74
MessageDefinition (*class in annunciator.catalogue*), 73
messages, 134
messages (*annunciator.testing.EventCapture property*), 130
module
    annunciator.catalogue, 73
    annunciator.checks, 125
    annunciator.cli, 132
    annunciator.config, 90
    annunciator.context, 72
    annunciator.errors, 78
    annunciator.events, 87
    annunciator.filters, 98
    annunciator.formatters, 97
    annunciator.handlers, 94
    annunciator.handling, 102
    annunciator.hooks, 108
    annunciator.htmlreport, 132
    annunciator.integrity, 121
    annunciator.levels, 64
    annunciator.lint, 128
    annunciator.logger, 66
    annunciator.multiprocess, 123
    annunciator.provenance, 118
    annunciator.recorder, 114
    annunciator.report, 109
    annunciator.sphinxext, 133
    annunciator.testing, 129
    annunciator.warns, 84
    annunciator.watch, 115
modules, 64
msg_role() (*in module annunciator.sphinxext*), 133
multiprocessing
    procedure, 45

N

name (*annunciator.logger.StructuredLogger property*), 66
name (*annunciator.provenance.Run attribute*), 119
necessary software, 16
new_codes (*annunciator.report.RunComparison attribute*), 113
new_correlation_id() (*in module annunciator.context*), 73
new_failures (*annunciator.report.RunComparison attribute*), 113
NOTICE (*annunciator.levels.Severity attribute*), 65
notice() (*annunciator.logger.StructuredLogger method*), 68
number (*annunciator.catalogue.MessageDefinition property*), 74

O

ok (*annunciator.handling.ErrorCollector property*), 106
ok (*annunciator.integrity.ChainResult property*), 122
once() (*annunciator.logger.StructuredLogger method*), 69
OperationError, 81
OperationTimeoutError, 82
option_names() (*annunciator.config.LogConfig class method*), 92
option_spec (*annunciator.sphinxext.CatalogueDirective attribute*), 133
optional_arguments (*annunciator.sphinxext.CatalogueDirective attribute*), 133
options, 53

P

parse_code() (*in module annunciator.catalogue*), 76
path (*annunciator.integrity.ChainResult attribute*), 122
path (*annunciator.lint.LintFinding attribute*), 128
PerformanceWarning, 86

plain text format, 58
PlainFormatter (*class in annunciator.formatters*), 97
prefix, **156**
prefix (*annunciator.catalogue.MessageDefinition property*), 74
prepare() (*annunciator.handlers.AsyncQueueHandler method*), 96
prepare() (*annunciator.multiprocess.ProcessQueueHandler method*), 124
problem resolution, 134
problems (*annunciator.integrity.ChainResult attribute*), 122
procedures, 19
ProcessQueueHandler (*class in annunciator.multiprocess*), 124
progress
    procedure, 39
Progress (*class in annunciator.watch*), 116
pytest, 48

Q

quick start, 21

R

raise_if_errors() (*annunciator.handling.ErrorCollector method*), 107
RateLimitFilter (*class in annunciator.filters*), 101
read_events() (*in module annunciator.report*), 111
record, **156**
record_suppressed_summary() (*in module annunciator.logger*), 71
records (*annunciator.integrity.ChainResult attribute*), 122
redaction, 25, **156**
RedactionFilter (*class in annunciator.filters*), 100
reference information, 51
register_catalogue() (*in module annunciator.catalogue*), 77
register_levels() (*in module annunciator.levels*), 65
regression (*annunciator.report.RunComparison property*), 113
related information, 157
remedy, **156**
removal, 16
repeated messages, 39
replace() (*annunciator.config.LogConfig method*), 93
replacement field, **156**
report
    procedure, 36
report() (*annunciator.logger.StructuredLogger method*), 70
required_arguments (*annunciator.sphinxext.CatalogueDirective attribute*), 133
reset_occurrences() (*in module annunciator.logger*), 71
ResourceAccessError, 81
ResourceError, 80
ResourceNotFoundError, 81
retry, 32
retry() (*in module annunciator.handling*), 104
RetryExhaustedError, 82
rotation, 22, **156**
run, **156**
    procedure, 43
Run (*class in annunciator.provenance*), 119
run manifest, **156**
run_id (*annunciator.provenance.Run attribute*), 119
RunComparison (*class in annunciator.report*), 112

S

save() (*annunciator.catalogue.MessageCatalogue method*), 76
save_manifest() (*annunciator.provenance.Run method*), 120
select() (*annunciator.catalogue.MessageCatalogue method*), 75
select() (*annunciator.handlers.EventStoreHandler method*), 95
select() (*annunciator.testing.EventCapture method*), 130
select_events() (*in module annunciator.report*), 112
set_level() (*annunciator.logger.StructuredLogger method*), 67
setup() (*in module annunciator.sphinxext*), 133
severity, 9, **156**
    reference, 53
severity (*annunciator.catalogue.MessageDefinition property*), 74
Severity (*class in annunciator.levels*), 64
severity_changes (*annunciator.report.RunComparison attribute*), 113
SeverityLike (*in module annunciator.levels*), 65
SeverityRangeFilter (*class in annunciator.filters*), 99
shutdown() (*in module annunciator.config*), 93
Simplified Technical English, 6
soft check, **156**
    procedure, 45
SourceLocation (*class in annunciator.events*), 89
Sphinx extension

procedure, 38
stall, 39, **156**
stalls (*annunciator.watch.Watchdog attribute*), 117
standards, 157
start() (*annunciator.watch.Watchdog method*), 117
start_listener() (*in module annunciator.multiprocess*), 124
start_run() (*in module annunciator.provenance*), 120
status (*annunciator.provenance.Run attribute*), 119
stdlib (*annunciator.logger.StructuredLogger property*), 66
stop() (*annunciator.watch.Watchdog method*), 117
stop_listener() (*in module annunciator.multiprocess*), 124
stream (*annunciator.handlers.ConsoleHandler property*), 94
structured field, **155**
structured fields, 25
structured logger, **156**
StructuredLogger (*class in annunciator.logger*), 66
summarise() (*in module annunciator.report*), 112
summary() (*annunciator.handling.ErrorCollector method*), 107
supports_colour() (*in module annunciator.formatters*), 98
suppressed_counts() (*in module annunciator.logger*), 71

T

tamper-evident log, 43
target_name() (*in module annunciator.sphinxext*), 134
tests
procedure, 48
thread, 35
throttle() (*annunciator.logger.StructuredLogger method*), 69
timed, 32
timed (*class in annunciator.handling*), 105
to_dict() (*annunciator.catalogue.MessageCatalogue method*), 76
to_dict() (*annunciator.catalogue.MessageDefinition method*), 74
to_dict() (*annunciator.config.LogConfig method*), 93
to_dict() (*annunciator.events.Event method*), 88
to_dict() (*annunciator.events.ExceptionInfo method*), 89
to_dict() (*annunciator.events.SourceLocation method*), 89
to_dict() (*annunciator.report.EventReport method*), 111
to_dict() (*annunciator.report.RunComparison method*), 113
to_json() (*annunciator.events.Event method*), 88
to_text() (*annunciator.integrity.ChainResult method*), 122
to_text() (*annunciator.lint.LintFinding method*), 129
to_text() (*annunciator.report.EventReport method*), 111
to_text() (*annunciator.report.RunComparison method*), 113
total (*annunciator.report.EventReport attribute*), 109
TRACE (*annunciator.levels.Severity attribute*), 64
trace() (*annunciator.logger.StructuredLogger method*), 67
traceback, **156**
track() (*in module annunciator.watch*), 117
troubleshooting, 134

U

unbind() (*in module annunciator.context*), 72
unhandled exception, 35
uninstall_hooks() (*in module annunciator.hooks*), 108
unregister_catalogue() (*in module annunciator.catalogue*), 77
update() (*annunciator.watch.Progress method*), 116
upgrade, 16

V

ValidationError, 79
verify_chain() (*in module annunciator.integrity*), 123

W

warn() (*in module annunciator.warns*), 86
WARNING (*annunciator.levels.Severity attribute*), 65
warning() (*annunciator.logger.StructuredLogger method*), 68
warnings (*annunciator.handling.ErrorCollector attribute*), 106
warnings (*annunciator.provenance.Run property*), 119
watchdog, 39, **156**
Watchdog (*class in annunciator.watch*), 117
worst (*annunciator.report.EventReport attribute*), 110
write_events() (*in module annunciator.report*), 111
write_html_report() (*in module annunciator.htmlreport*), 132